

logo.png

logo.png

People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research
University of Science and Technology Houari Boumediene (USTHB)
Faculty of Computer Science
Department of Computer Systems (SIQ)

Master's Thesis

Field: Computer Science
Specialization: High Performance Computing (HPC)

Topic:

**A Hybrid Optimization Method for Protein Complex
Extraction from Large PPI Networks on HPC Platforms.**

Presented by:

KEFSI Nourhane
MEKHAZNI Ryham

Proposed by:

Dr. MEHDI Malika
Dr. SAADI Hocine

Defended on: .././....

Submitted to the jury composed of:

Dr. BOUIBEDE Karima	President
Dr. BELHADI Hiba	Member

Group N° : HPC-002 / 2025

Acknowledgements

First and foremost, we give thanks to God for granting us the strength, patience, and perseverance required to complete this work.

We express our sincere gratitude to our supervisor, Dr. Mehdi, for allowing us to work on this stimulating research topic and for providing access to her Grid5000 account. Her guidance, trust, and support throughout the project have been of immense value to us.

We would also like to thank Dr. Saadi for his constant availability, his responsiveness to our numerous questions, and his insightful advice, which greatly helped us progress in our work.

*Finally, we extend our heartfelt thanks to the members of the jury: **Dr. Karima Bouibede**, President of the jury, and **Dr. Hiba Belhadi**, Member of the jury, for the honor of evaluating our work, and for their constructive and enriching feedback.*

Dedication

In the name of Allah, the Most Gracious, the Most Merciful.

I dedicate this work to Allah Almighty, who granted me the strength, patience, and guidance to persevere throughout this journey.

All praise is due to Him alone.

Dedication

In the name of Allah, the Most Gracious, the Most Merciful.

All praise is due to Allah for guiding, supporting, and protecting me throughout every step of this journey.

I dedicate this work:

To my beloved family — the foundation of my strength — for their sincere love, silent prayers, and unwavering support.

To my dear friends, whose kindness, encouragement, and understanding have uplifted me in times of doubt.

And to myself, for not giving up despite the challenges.

May Allah make this work a source of goodness.

Alhamdulillah Rabbil 'Alamin.

Abstract

Protein-protein interactions (PPIs) play a fundamental role in cellular functioning, being involved in a wide range of biological processes such as signaling, gene regulation, and DNA replication. Studying these interactions helps uncover the molecular mechanisms behind both normal biological functions and complex diseases.

Protein-protein interaction networks (PPI networks) are graphical representations of these interactions, where nodes represent proteins and edges represent physical or functional interactions. Detecting protein complexes — groups of proteins that interact in a stable and coherent manner — is essential for understanding the modular organization of cellular systems and predicting novel biological assemblies.

In this final-year project, we propose a method for extracting protein complexes based on hybrid metaheuristic optimization. Our approach combines a genetic algorithm with a local search using Tabu Search, and includes a biologically informed weighting of the PPI network to increase robustness and result quality.

To address the challenges posed by the size and complexity of real-world PPI networks, we implemented a parallel version of our method based on the island model, where multiple subpopulations evolve independently with periodic information exchange. This parallel version was successfully executed on the Grid5000 high-performance computing platform, allowing us to significantly reduce execution time and improve scalability.

Keywords: protein complexes, PPI networks, hybrid optimization, parallel computing, Grid5000, bioinformatics, metaheuristics.

Résumé

Les interactions entre protéines jouent un rôle fondamental dans le fonctionnement cellulaire, intervenant dans une multitude de processus biologiques tels que la signalisation, la régulation de l'expression génique ou la réplication de l'ADN. Leur étude permet de mieux comprendre les mécanismes moléculaires complexes qui sous-tendent les fonctions biologiques normales ainsi que l'apparition de pathologies.

Les réseaux d'interactions protéine-protéine (PPI) constituent une représentation graphique de ces relations : les nœuds y représentent les protéines, et les arêtes les interactions physiques ou fonctionnelles observées. L'extraction de complexes protéiques, c'est-à-dire de groupes de protéines interagissant de manière stable et cohérente, est une tâche essentielle pour l'analyse de ces réseaux. Elle permet notamment de prédire des assemblages biologiques fonctionnels encore inconnus et de mieux comprendre l'organisation modulaire des systèmes cellulaires.

Dans ce projet de fin d'études, nous avons développé une méthode d'extraction de complexes protéiques basée sur des techniques d'optimisation métaheuristiques hybrides. L'approche proposée combine un algorithme génétique avec une recherche tabou locale, et intègre une pondération préalable du réseau PPI fondée sur des critères biologiques. Cette combinaison vise à renforcer la robustesse de la recherche tout en améliorant la qualité des complexes extraits.

Afin de répondre aux défis liés à la taille et à la complexité croissantes des réseaux biologiques, nous avons parallélisé notre approche selon un modèle de type « îles », où plusieurs sous-populations évoluent de manière concurrente, avec des échanges contrôlés d'individus. Cette version parallèle a été implémentée et testée avec succès sur la plateforme Grid5000, dans le but d'améliorer l'efficacité de traitement et la scalabilité de l'algorithme.

Mots-clés : complexes protéiques, réseaux PPI, optimisation hybride, calcul parallèle, Grid5000, bioinformatique, métaheuristiques.

Contents

Contents

List of figures	II
List of tables	III
List of algorithms	V
List of abbreviations	VI
General Introduction	1
1 Biological and Computational Context	3
1.1 Introduction	3
1.2 Biological Concepts	3
1.2.1 Protein Synthesis and Structure	3
1.2.2 Protein-Protein Interactions (PPI)	4
1.2.3 Protein Complexes and Their Extraction from PPI Networks	7
1.2.4 Experimental Limitations and Computational Solutions	11
1.3 Computational Concepts	11
1.3.1 Computational Challenges in PPI Networks	11
1.3.2 Optimization Methods	12
1.3.3 Parallel Computing and HPC Platforms	19
1.4 Conclusion	23
2 State of the Art	25
2.1 Introduction	25
2.2 Classical Exact Methods	25
2.2.1 Cluster Quality-Based Methods	26
2.2.2 Node Affinity-Based Methods	26
2.2.3 Network Embedding-Based Methods	26
2.3 Metaheuristic Optimization Approaches	27

2.3.1	Representative Works by Metaheuristic Category	27
2.3.2	Genetic Algorithm-Based Methods	28
2.3.3	Genetic Algorithm Methods: PPI-GA and GAGC	28
2.4	Optimization and Parallelism	30
2.4.1	Parallel Genetic Algorithms (PGAs)	30
2.4.2	Parallel Tabu Search (PTS)	31
2.5	Conclusion	32
3	Proposed Methodology	33
3.1	Introduction	33
3.2	Global Solution Architecture	33
3.2.1	Methodology Overview	33
3.2.2	Methodology Workflow	34
3.3	Data Collection and Preprocessing	35
3.3.1	Data Sources	35
3.3.2	Data Filtering and Normalization	37
3.3.3	PPI Network Weighting	37
3.4	Hybrid Optimization Model Design	37
3.4.1	Solution Representation	38
3.4.2	Multi-Objective Fitness Function	38
3.4.3	Population Initialization	38
3.4.4	Search Strategy	39
3.4.5	Hybrid Algorithm	42
3.5	Parallel Computing Integration	43
3.5.1	Parallel Population Initialization	43
3.5.2	Island Model	44
3.5.3	Parallel Tabu Search	45
3.5.4	Neighborhood Parallelism	46
3.5.5	Master-Slave Metric Computation	47
3.5.6	Global Parallel Algorithm	48
3.6	Conclusion	49
4	Implementation and Results	50
4.1	Introduction	50
4.2	Development and Execution Environments	50
4.2.1	Software Environment	50
4.2.2	Hardware Infrastructure	51
4.3	Experiments	52
4.3.1	Datasets	52
4.3.2	Algorithm Parameters	53

4.3.3	Parameter Justification	54
4.4	Results	54
4.4.1	Extracted Complex Analysis	54
4.4.2	Visualization with Cytoscape	55
4.4.3	Execution Time and Scalability	56
4.5	Discussion and Analysis	58
4.5.1	Interpretation of Results	58
4.5.2	Limitations and Perspectives	65
4.6	Conclusion	67
General Conclusion		68
Bibliographie		70
Annexe A : Initialization Methods, Comparison and their Implementation		A-1
A.1	Protein Complex Detection Methods	A-1
A.1.1	Markov Clustering - MCL	A-1
A.1.2	Molecular Complex Detection Method - MCODE	A-2
A.1.3	Clustering with Overlapping Neighborhood Expansion - ClusterONE	A-3
A.1.4	Core-Attachment Based Method - COACH	A-4
A.1.5	Edge Weight Algorithm and Core Attachment Structure - EWCA	A-4
A.1.6	Seed Extended Algorithm Based on Density and Modularity with Topo- logical Structure and GO Annotations - SEDMTG	A-8
A.1.7	MPC-C	A-10
A.2	Implementation and results	A-12
A.2.1	Markov clustering - MCL	A-12
A.2.2	Molecular Complex Detection method - MCODE	A-12
A.2.3	Clustering with Overlapping Neighborhood Expansion - ClusterONE	A-13
A.2.4	Core Attachment Based Method - COACH	A-14
A.2.5	An Extended Algorithm Based on Density and Modularity with Topo- logical Structure and GO Annotations - SEDMTG	A-15
A.2.6	Edge Weight Algorithm and Core Attachment Structure - EWCA	A-15
A.2.7	Protein Complex Extraction Using a Clustering Model - MPCC	A-16
Annexe B : Design, Implementation and Results		B-1
B.1	Detailed Functions	B-1

List of Figures

1.1	Illustration of protein synthesis and structures.	4
1.2	Examples of two golden complexes.	8
1.3	Protein complex extraction from PPI network.	9
1.4	Classification of metaheuristics.	14
1.5	Illustration of the three types of parallelism: data, task, and pipeline	20
1.6	Architecture of a computational grid with multiple interconnected sites, each composed of full clusters	22
1.7	Comparison of parallel programming models: distributed memory, shared memory, and hybrid model.	23
2.1	Classification of protein complex detection methods.	27
2.2	Different PGA models: (a) master-slave model, (b) island model, (c) cellular model.	31
2.3	Hierarchical taxonomy of parallel tabu search models according to Crainic et al. [109]. Each dimension (control, coordination, differentiation) includes several modalities.	32
3.1	Global workflow of the methodology: from initialization to parallel optimization	35
4.1	Four extracted protein complexes with different topological structures generated via Cytoscape.	56
4.2	Execution time (in minutes) for each network, in sequential and parallel versions.	57
4.3	Parallel efficiency (%) according to the number of cores, for five PPI networks.	57
4.4	Speedup observed on five protein interaction networks.	58
4.5	Visual comparison between the reference complex AP-1 and the complexes predicted by different methods (Part 1).	60
4.5	Visual comparison between the reference complex AP-1 and the complexes predicted by different methods (Part 2).	60
4.6	Comparison of total scores of different methods on various networks.	64
A.1	Variation of total score with respect to VWP	A-13
A.2	Variation of total score with respect to fluff_density	A-13

A.3	Visualization of ClusterONE performance from two complementary perspectives	A-14
A.4	Variation of total score with respect to COACH parameters	A-15
A.5	Variation of total score with respect to number of iterations of SEDMTG	A-15
A.6	Variation of total score with respect to COACH parameters	A-16
A.7	Variation of total score with respect to COACH parameters	A-16

List of Tables

1.1	Protein-protein interaction detection methods.	5
1.2	PPI network size classification.	7
1.3	Summary of major PPI databases [7, 8].	7
1.4	Summary of major golden complex databases.	9
3.1	Metric Coverage of Biological Properties	38
4.1	PPI network statistics	53
4.2	Gold standard complex statistics	53
4.3	Genetic algorithm parameters	53
4.4	Tabu search parameters	54
4.5	Parallel configuration	54
4.6	Topological characteristics of predicted complexes	55
4.7	Complex overlap with gold standards	55
4.8	Biological metrics for the methods on the BIOGRID network	61
4.9	Biological metrics for the methods on the STRING networks	61
4.10	Biological metrics for the methods on the DIP_yeast network	62
4.11	Results of the methods on human networks	62
4.12	Results of the methods on yeast networks	63
A.1	Network performance for different inflation values	A-12
A.2	Variation of <i>Total Score</i> according to MCODE parameters	A-14

List of Algorithms

1.1	Tabu Search	15
1.2	Genetic Algorithm	16
3.1	POPULATIONINITIALIZATION	39
3.2	BINARYTOURNAMENTSELECTION	40
3.3	CROSSOVER	40
3.4	LOCALTABUSEARCH	41
3.5	UPDATEPOPULATION	42
3.6	OPTCLUST-H	43
3.7	PARALLELRANDOMINITIALIZATION	44
3.8	EXECUTEISLAND	45
3.9	MUTATEANDIMPROVE	46
3.10	PARALLELTABUSEARCH	47
3.11	PARALLELMETRICCOMPUTATION	48
3.12	OPTCLUST-Hp($G, R, n_{\text{islands}}, g, \text{mig_inter}, \text{mig_size}, n_{\text{alea}}$)	49
A.1	Markov Clustering (MCL)	A-2
A.2	MCODE - Molecular Complex Detection	A-3
A.3	ClusterONE - Protein Complex Detection	A-5
A.4	COACH - Protein Complex Core Mining	A-6
A.5	EWCA - Protein Complex Detection	A-8
A.6	SEDMTG - Protein Complex Detection	A-10
A.7	MPCC - Weighted Protein Complex Detection	A-11
B.2	GENERATEINDIVIDUAL	B-1
B.1	RANDOMINITIALIZATION	B-2
B.3	CREATECONNECTEDCOMPLEX	B-3
B.4	INITGLOBALS	B-3
B.5	COMPUTEMETRICS(P, R, τ)	B-3
B.6	COMPUTEPPV(P, R)	B-4
B.7	COMPUTERECALL(P, R)	B-4
B.8	COMPUTE FMEASURE(ppv, sn)	B-4

B.9	COMPUTEMMR(P, R)	B-4
B.10	COMPUTEJACCARD(P, R)	B-4
B.11	COMPUTECOVEREDRATE(P, R, τ)	B-5
B.12	COMPUTEFITNESSPOPULATION(P, G)	B-5
B.13	FITNESS(I, G)	B-5
B.14	FF(C, G)	B-5
B.15	COHESION(C, G)	B-6
B.16	DENSITY(C, G)	B-6
B.17	AIEW(C, G)	B-6
B.18	ABEW(C, G)	B-7
B.19	AWM(C, G)	B-7
B.20	NEIGHBORSBYDELETION(C, G)	B-7
B.21	NEIGHBORSBYINSERTION(C, G)	B-7
B.22	NEIGHBORSBYREPLACEMENT(C, G)	B-8
B.23	NEIGHBORSBYDENSECOMPLEXES(C, G, R)	B-8
B.24	DIVERSIFICATIONBYRARELYVISITEDPROTEINS(G)	B-8
B.25	PARALLELPOPULATIONINITIALIZATION	B-9
B.26	DIVIDEPOPULATIONINTOISLANDS	B-9
B.27	SELECTBESTINDIVIDUALS	B-9
B.28	SENDTORIGHTNEIGHBOR	B-9
B.29	RECEIVEFROMLEFTNEIGHBOR	B-10
B.30	SELECTWORSTINDIVIDUALS	B-10

List of abbreviations

ADN	Acide Désoxyribonucléique
ARN	Acide Ribonucléique
ARNm	Messenger Ribonucleic Acid (mRNA)
ARNt	Transfer Ribonucleic Acid (tRNA)
ARNr	Ribosomal Ribonucleic Acid (rRNA)
PPI	Protein-Protein Interaction
PPIN	Protein-Protein Interaction Network
TAP-MS	Tandem Affinity Purification – Mass Spectrometry
Co-IP	Co-immunoprecipitation
NMR	Nuclear Magnetic Resonance Spectroscopy
Y2H	Yeast Two-Hybrid System
FRET	Förster Resonance Energy Transfer
BRET	Bioluminescence Resonance Energy Transfer
DIP	Database of Interacting Proteins
BioGRID	Biological General Repository for Interaction Datasets
STRING	Search Tool for the Retrieval of Interacting Genes/Proteins
IntAct	IntAct Molecular Interaction Database
MINT	Molecular INTeraction Database
MIPS	Munich Information Center for Protein Sequences
CORUM	Comprehensive Resource of Mammalian Protein Complexes
CYC2008	Comprehensive Yeast Complexes 2008
Cryo-EM	Cryogenic Electron Microscopy
AP-MS	Affinity Purification – Mass Spectrometry
BioGRID-ORCS	BioGRID Open Repository of CRISPR Screens
Hu.MAP 2.0	Human Multi-protein Assembly Prediction version 2.0
PDB	Protein Data Bank
MCL	Markov Clustering Algorithm
MCODE	Molecular Complex Detection
TSP	Travelling Salesman Problem
ACO	Ant Colony Optimization

GRASP	Greedy Randomized Adaptive Search Procedure
GA	Genetic Algorithm
TS	Tabu Search
DF	Depth-First (Search)
PSO	Particle Swarm Optimization
DE	Differential Evolution
SA	Simulated Annealing
VNS	Variable Neighborhood Search
PPV	Positive Predictive Value
Sn	Sensitivity (Recall)
MMR	Mean Matching Rate
HPC	High Performance Computing
CPU	Central Processing Unit
GPU	Graphics Processing Unit
POSIX	Portable Operating System Interface
OpenMP	Open Multi-Processing
MPI	Message Passing Interface
NUMA	Non-Uniform Memory Access
GPGPU	General-Purpose Graphics Processing Unit
SIMD	Single Instruction, Multiple Data
ID	Identifier
CUDA	Compute Unified Device Architecture
ClusterONE	Clustering with Overlapping Neighborhood Expansion
LCMA	Local Clique Merging Algorithm
mIPALM	Improved Protein complex detection Algorithm based on Local Module density
CMC	Clustering based on Maximal Cliques
CFinder	Clique Finder (based on the k-clique percolation method)
CORE	Clustering based On Recursive Expansion
COACH	Core-Attachment based method
PECC	Protein complex Extraction by Core-attachment and Clustering coefficient
RNSC	Restricted Neighborhood Search Clustering
GANe	Genetic Algorithm based on Node and Edge information
SE-DMTG	Seed-Extended algorithm for detecting protein complexes based on Density and Topological structure and GO annotations
EWCA	Edge Weight method and Core-Attachment structure
MPC-C	Mining protein complexes using a new clustering model

DEMO	Differential Evolution Multi-Objective
SLA	Stochastic Learning Automaton
ACOPIN	Ant Colony Optimization for Protein Interaction Networks
PSP	Protein Structure Prediction
GAGC	Genetic Algorithm-based Graph Clustering
PPI-GA	Genetic Algorithm for Protein-Protein Interaction Network Clustering
IPCA	Initial Population Construction Algorithm
EIC	Edge Importance Calculation
PGA	Parallel Genetic Algorithm
PTS	Parallel Genetic Algorithm
SPSS	Single Point Single Strategy
SPDS	Single Point Diverse Strategies
MPSS	Multiple Points Single Strategy
MPDS	Multiple Points Diverse Strategies
SGD	Saccharomyces Genome Database
GO	Gene Ontology
HCN	High Common Neighbor
FS	Functional Similarity
GEO	Gene Expression Omnibus
PCC	Pearson Correlation Coefficient
RAM	Random Access Memory
OS	Operating System
Slurm	Simple Linux Utility for Resource Management
VWP	Vertex Weight Percentage
OptClust-H	Hybrid Optimized Clustering
OptClust-Hp	Parallel Hybrid Optimization Clustering

General Introduction

The Human Genome Project, launched in 1990 and marked by key milestones under the presidency of Bill Clinton (2000) before its completion in 2003, revolutionized life sciences by establishing, for the first time, the complete sequence of the 3.2 billion base pairs of human DNA ¹. This historic achievement, further refined in 2022 through the sequencing of previously missing regions, laid the foundation for the emergence of modern proteomics ². Much like a compiler translating source code into an executable program, this project made it possible to convert genetic information into a functional understanding of proteins.

Proteins, the true workers of the cell, collaborate with each other to carry out specific tasks by forming interactions, thereby creating protein complexes. These complexes are specialized in precise biological processes, and the entirety of proteins and their interactions can be modeled using a PPI (Protein-Protein Interaction) network. Identifying complexes within these networks is crucial in many fields. For instance, in a PPI network derived from a cancerous organ, it enables researchers to determine which complexes are responsible for the core cancer-related functions, paving the way for the design of targeted drugs to inhibit them.

Current methods for identifying protein complexes suffer from three major limitations that hinder their large-scale application:

- (i) Prohibitive computational complexity for large networks, with execution times increasing non-linearly with graph size;
- (ii) Increased sensitivity to experimental noise: nearly 40% of reported interactions may be false positives [110];
- (iii) Structural specificity: each method focuses on a particular complex architecture, excluding those with different structures.

These challenges, clearly identified in recent assessments, create a critical need for innovative algorithmic solutions capable of simultaneously ensuring:

- Exhaustive coverage of biologically relevant complexes;
- Computational performance suitable for very large-scale graphs;

¹DNA: A molecule that contains the basic genetic code (A, T, C, G) used by cells to produce proteins.

²Proteomics studies all the proteins in a cell: their shapes, functions, and how they work together.

- High tolerance to imperfections in experimental data.

This multidimensional problem calls for the development of a hybrid approach that synergizes the complementary strengths of advanced metaheuristics and current methods, all optimized for modern parallel computing architectures. Such a methodological combination, still insufficiently explored according to recent literature, would represent a significant step forward for high-throughput analysis of interaction networks, with important potential benefits for therapeutic discovery.

This document is structured into four main chapters:

- **Chapter 1:** Presentation of the biological and computational foundations required to understand PPI networks and our proposed method;
- **Chapter 2:** Critical review of the state of the art, identification of the limitations of existing approaches, and a detailed discussion of the parallelism models from the literature used to optimize the metaheuristics in our approach;
- **Chapter 3:** Description of the proposed hybrid methodology, combining:
 - Integration of multimodal data (network topology and biological annotations);
 - Use of adapted existing methods as starting points for metaheuristics;
 - Hybrid optimization approach combining genetic algorithm and tabu search;
 - Parallel approach deploying the hybrid methodology on an HPC platform (Grid'5000).
- **Chapter 4:** Presentation of the environment, parameters, experimental results, and validations on real biological data, with a discussion of the contributions and limitations of our two approaches.
- **General Conclusion:** Presentation of the main contributions of this work, summary of the obtained results, discussion of identified limitations, and proposal of possible improvements or future extensions.

The document is complemented by technical appendices and biological visualizations generated using Cytoscape ³, offering both theoretical and practical insights into the proposed advances.

This general introduction sets the scientific framework of the thesis while highlighting the originality of our approach in the face of current bioinformatics challenges. The following chapters will elaborate on each of these aspects in detail, from theoretical foundations to experimental validations. Furthermore, the entire source code associated with this project is openly available on GitHub at the following address: https://github.com/nourhanekefsi/Master_final_project.

³Cytoscape: an open-source software specialized in the visualization and analysis of biological networks, particularly suited to the study of protein complexes and protein-protein interaction (PPI) networks.

Chapter 1

Biological and Computational Context

1.1 Introduction

The use of hybrid optimization methods to solve our problem of extracting protein complexes from large PPI networks in an HPC environment relies on a solid understanding of both biological foundations and algorithmic solution approaches. This chapter aims to introduce these essential concepts.

It is divided into two main parts. First, we present fundamental biological concepts along with a formal problem definition. Second, we discuss optimization approaches, focusing on metaheuristics and parallel computing principles that form the core of our methodology.

1.2 Biological Concepts

This section establishes the biological foundations necessary for understanding the protein complex extraction problem. We adopt a progressive approach: starting with protein synthesis and structure, then examining their interactions leading to protein complex formation. This leads us to a rigorous problem definition. Finally, we highlight the essential contribution of computational approaches to overcome experimental method limitations.

1.2.1 Protein Synthesis and Structure

Living organisms, whether humans, animals, or plants, exhibit complex and finely regulated biological organization ensuring their survival. This organization relies on essential physiological processes such as cellular respiration, digestion, or photosynthesis¹. To execute these processes, cells synthesize proteins - the molecular effectors of these functions.

The protein synthesis scheme and their structures are shown in Figure 1.1 and will be detailed below:

¹**Cellular respiration** converts sugar and O₂ into CO₂ and usable energy; **digestion** breaks down nutrients via protein enzymes; and **photosynthesis** converts light into chemical energy through protein complexes.

A. **Gene expression:** The biological process that synthesizes proteins, divided into two main steps [1, 2]:

- (a) **Transcription:** During this step, a copy of the protein-coding sequence in DNA (gene) is produced. This copy is called messenger RNA (mRNA)².
- (b) **Translation:** The mRNA is then read by a ribosome³, which assembles amino acids⁴ corresponding to their codons in the mRNA to form a polypeptide chain.

B. **Protein structure:** The polypeptide chain adopts four structures [2, 3]:

- **Primary structure:** Linear amino acid sequence.
- **Secondary structure:** Local polypeptide chain folding (α -helices, β -sheets).
- **Tertiary structure:** Overall three-dimensional protein organization.
- **Quaternary structure:** Association of multiple polypeptide chains (subunits).

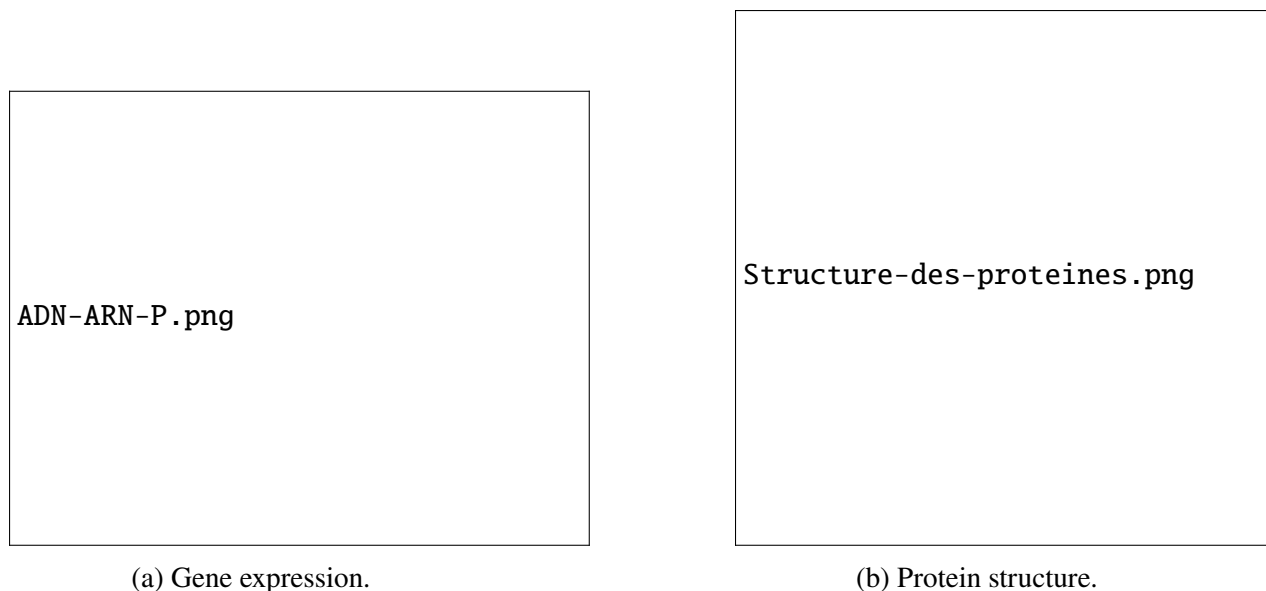


Figure 1.1: Illustration of protein synthesis and structures.

Once synthesized, these proteins can interact with each other, forming interactions detailed in the following subsection.

1.2.2 Protein-Protein Interactions (PPI)

Protein-protein interactions (PPIs) refer to physical contacts between two or more proteins, generally governed by multiple contributions from individual molecular fragments [4]. Two main types are distinguished [5]:

²DNA (deoxyribonucleic acid): molecule containing hereditary genetic information; RNA (ribonucleic acid): intermediary molecule involved in protein synthesis.

³The ribosome is a cellular "machine" that builds proteins by following mRNA instructions.

⁴Amino acids are protein building blocks. Some come from diet, others are synthesized by the cell.

- Stable interactions (strong and long-lasting).
- Transient interactions (weak and temporary).

This subsection presents methods for detecting these interactions, introduces a commonly used modeling approach, and describes available databases.

1.2.2.1 PPI Detection Methods

Several categories of methods are used to detect these interactions [6], as detailed in Table 1.1.

Table 1.1: Protein-protein interaction detection methods.

Category	Method
<i>In Vitro</i> ⁵	- Tandem affinity purification mass spectrometry (TAP-MS) - Affinity chromatography - Co-immunoprecipitation (Co-IP) - Protein microarrays - Protein fragment complementation - Phage Display - X-ray crystallography - NMR spectroscopy - Yeast two-hybrid system (Y2H)
<i>In Vivo</i> ⁶	- Synthetic lethality - FRET (Förster resonance energy transfer) - BRET (Bioluminescence resonance energy transfer)
<i>In Silico</i> ⁷	- Molecular docking - Co-evolution analysis - Machine learning - Interaction network analysis

To analyze these interactions using computational tools and methods, appropriate modeling is necessary, formally defined below.

1.2.2.2 PPI Networks

Protein-protein interactions in a given organism, obtained through various detection methods, are grouped and modeled as a graph called a "*PPI network*" [7]. Figure 1.3a shows an example PPI network with 10 proteins $P_1, P_2, \dots, P_{10}$.

Definition 1.2.1. A PPI network is an undirected graph $G = (V, E)$ where:

- V is the set of vertices representing proteins.
- $E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$ is the set of edges representing physical interactions between protein pairs.

Key concepts used in our report [11]:

- **Protein neighborhood:** The neighbors of protein u are all proteins connected to it in the PPI network:

$$N(u) = \{v \mid v \in V, \{u, v\} \in E\}. \quad (1.2.1)$$

- **Network density:** Measuring the proportion of existing interactions relative to the maximum possible. For a graph $G = (V, E)$ with $|V|$ nodes and $|E|$ edges, it is given by:

$$\text{Density}(G) = \frac{\sum_{e \in E} w(e)}{|V|(|V| - 1)/2} \quad (1.2.2)$$

where $w(e)$ represents edge weight, with $w(e) = 1$ if the interaction is unweighted. This measure quantifies global network connectivity considering both presence and interaction intensity.

PPI networks can be classified into two categories based on their nature [8]:

- **Static networks:** Static models representing protein-protein interactions at a given time under stable biological conditions.
- **Dynamic networks:** Evolving models incorporating temporal dimension to capture protein interaction variability.

PPI networks vary in size based on the number of proteins covered and interactions modeled, as shown in Table 1.2.

Network type	#proteins	#interactions
Small	< 1,000	< 10,000
Medium	1,000 – 10,000	10,000 – 100,000
Large	> 10,000	> 50,000

Table 1.2: PPI network size classification.

For this project, we primarily work on static large-scale PPI networks. Medium-sized networks will also be used to evaluate our approach’s scalability.

1.2.2.3 PPI Network Databases

Table 1.3 below presents important PPI network databases for different species:

Name	Version	Species	#Proteins	#PPIs	URL
DIP	Feb 2017	All	28,255	76,881	http://dip.doe-mbi.ucla.edu/dip/Main.cgi
BioGRID	3.5.172	All	71,619	1,683,670	https://thebiogrid.org/
IntAct	2022	Human	45,000+	1,000,000+	https://www.ebi.ac.uk/intact/
MINT	2019	All	26,099	130,733	https://mint.bio.uniroma2.it
STRING	11.0	All	24,584,628	3,123,056,667	https://string-db.org/

Table 1.3: Summary of major PPI databases [7, 8].

Community structure detection represents an important property of complex networks, particularly PPI networks. The following subsection presents the extraction of PPI network components called "*protein complexes*".

1.2.3 Protein Complexes and Their Extraction from PPI Networks

We approach protein complex extraction in three stages: conceptual definition, formal modeling, then biological validation using specific metrics.

⁵Methods performed in test tubes

⁶Methods performed in living organisms

⁷Computer simulation methods

1.2.3.1 Protein Complexes

Proteins typically don't function alone but collaborate with others, forming what are called protein complexes. These are stable biomolecular entities resulting from non-covalent association of multiple proteins, constituting distinct functional units [9].

Definition 1.2.2. Let $P = \{p_1, p_2, \dots, p_n\}$ be a set of proteins and $E_c = \{(p_i, p_j) \mid p_i, p_j \in P\}$ the set of interactions between protein pairs. Set P forms a protein complex if and only if:

- The number of proteins is strictly greater than two ($n > 2$);
- The interaction graph $C = (P, E_c)$ is connected ($\forall (p_i, p_j) \in P^2, \exists$ a path between p_i and p_j in C);
- The set has a unified biological function (e.g., cooperative enzymatic activity).

Well-characterized protein complexes experimentally validated and referenced in specialized biological databases like MIPS, CORUM or CYC2008 are called **golden complexes**. They typically serve as biological references (or *gold standards*) to evaluate complex extraction methods from PPI networks [7]. Figure 1.2 shows two examples visualized in Cytoscape.

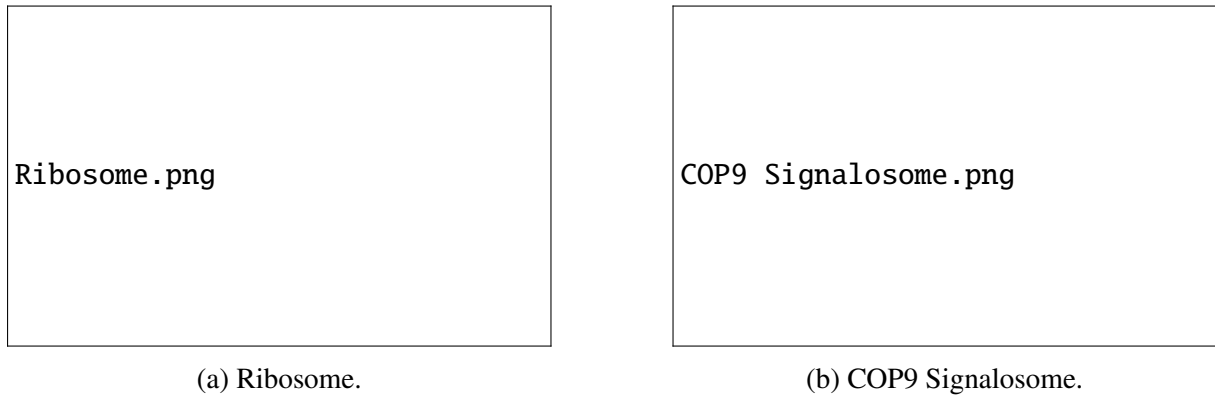


Figure 1.2: Examples of two golden complexes.

Several experimental validation methods are used including [9]:

- Immunoprecipitation (IP) and Co-Immunoprecipitation (Co-IP).
- Tandem affinity purification mass spectrometry (TAP-MS).
- Cryo-electron microscopy (Cryo-EM).
- Yeast two-hybrid (Y2H).
- Affinity purification mass spectrometry (AP-MS).

Table 1.4 summarizes major golden complex databases, indicating for each the species covered, number of proteins and complexes listed, and online access [7, 8].

Table 1.4: Summary of major golden complex databases.

Name	Version	Species	#Proteins	#Complexes	URL
CORUM	4.1	Mammals	8,000+	5,200+	https://mips.helmholtz-muenchen.de/corum
Complex Portal	2024	Multi-species	25,000+	5,000+	https://www.ebi.ac.uk/complexportal
BioGRID-ORCS	3.5	Multi-species	20,000+	1,500+	https://orcs.thebiogrid.org
Hu.MAP 2.0	2020	Human	4,600+	4,600+	http://humap2.proteincomplexes.org
PDB	2024	Multi-species	200,000+	200,000+	https://www.rcsb.org

1.2.3.2 Protein Complex Extraction

This involves identifying existing protein complexes in a PPI network. Formally: Let $G = (V, E)$ be a PPI network where V represents proteins and E their interactions (see Definition 1.2.1). Complex extraction consists of identifying subgraphs $C = (P, E_c)$ where $P \subseteq V$ and $E_c \subseteq E$, which may overlap (i.e., a protein may belong to multiple complexes). These subgraphs represent protein complexes (see Definition 1.2.2). Figure 1.3 illustrates a PPI network example and three extracted protein complexes, highlighting the overlap phenomenon. [10]

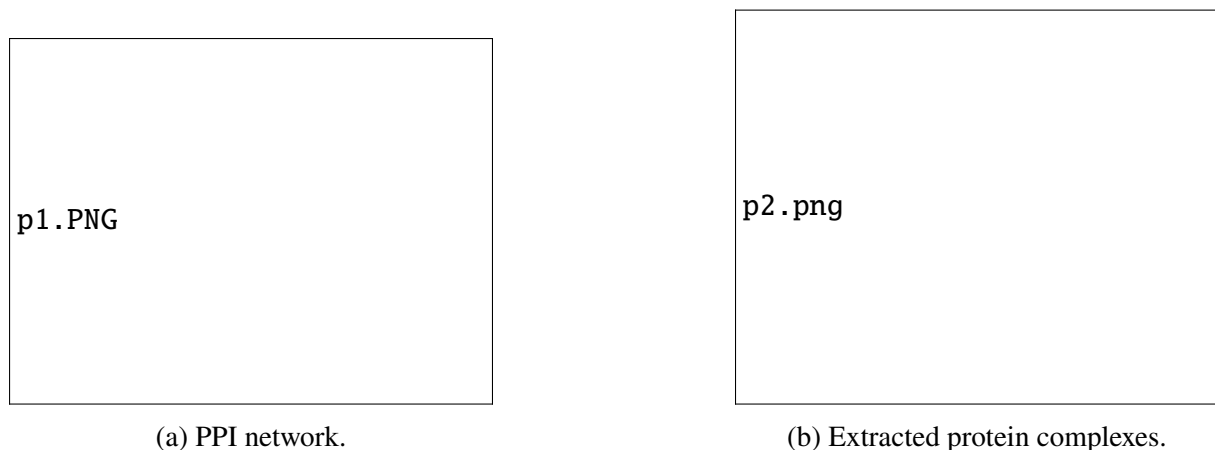


Figure 1.3: Protein complex extraction from PPI network.

We define key concepts for protein complex extraction used in subsequent chapters:

- **Cluster neighborhood:** PPI network proteins having at least one interaction with a cluster protein. Formally:

$$V(C) = \{u \mid \exists v \in C, (u, v) \in E\}. \quad (1.2.3)$$

- **Modularity (Q):** A key measure for evaluating complex network partitioning quality, comparing intra-cluster interaction density to that expected in a random network. Expressed as:

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j), \quad (1.2.4)$$

where $A_{ij} \in \{0, 1\}$ indicates interaction presence between nodes i and j , $k_i = \sum_j A_{ij}$ is node i 's degree, $m = \frac{1}{2} \sum_i k_i$ the total edge count, and $\delta(c_i, c_j)$ the cluster membership indicator function ($\delta = 1$ if $c_i = c_j$, 0 otherwise).

Typically, $Q \in [-1, 1]$, with $Q > 0.3$ indicating significant modular structure, particularly relevant for biological PPI network analysis.

1.2.3.3 Biological Evaluation of Protein Complexes

When evaluating predicted protein complexes, researchers rely on key biological indicators to determine whether detected protein groups truly function together in cells. The two most important indicators are [11, 12]:

- **p-value (biological significance):** The biological significance of a protein complex is evaluated by a **p-value** calculated from a hypergeometric distribution, quantifying the probability of randomly observing overlap between complex proteins and specific functional annotations. The formula is:

$$p = 1 - \sum_{i=0}^{k-1} \frac{\binom{F}{i} \binom{N-F}{C-i}}{\binom{N}{C}} \quad (1.2.5)$$

where N is the total annotated proteins in the reference set, F the number of proteins with the studied functional annotation, C the detected complex size, and k the number of complex proteins sharing this annotation. A p-value below 0.01 indicates statistically significant association between the complex and functional annotation, suggesting biological relevance.

- **Colocalization score (CS):** Measures the percentage of complex proteins sharing the same subcellular localization. A high CS suggests these proteins function together in the same cellular region. Calculated as:

$$CS = \frac{\sum_{j=1}^m \max_{i=1}^n (l_{i,j})}{\sum_{j=1}^m N_j} \quad (1.2.6)$$

where: $l_{i,j}$ is the number of proteins in complex j found at location i , N_j is the total protein count in complex j , m is the number of detected complexes, and n is the number of location groups.

1.2.3.4 Importance of Protein Complex Analysis

Protein complexes play essential roles in several domains:

- **Molecular biology:** They participate in key cellular functions like cell division or DNA replication ensured by a prokaryotic complex [13].
- **Biophysics:** Oxygen transport is ensured by the hemoglobin protein complex [14].
- **Medicine:** They help identify therapeutic targets, like the γ -secretase complex involved in Alzheimer's disease, enabling better pathology understanding and treatment development such as semagacestat [15, 16].

1.2.4 Experimental Limitations and Computational Solutions

Experimental methods for identifying protein complexes are often very time-consuming and expensive, making computational methods an effective solution.

Experimental methods (like affinity chromatography followed by mass spectrometry, Co-IP/MS) may require several days to weeks of processing at a cost of \$200 to \$1,000 per sample [17, 18, 19]. In comparison, computational methods (such as MCL or MCODE) provide results within hours to a day [20, 21] often for less than \$100 [78].

These computational methods enable prediction of potential new complexes, which can then be experimentally validated, reducing experimentation cost and time. Key concepts of these methods are detailed in the following section.

1.3 Computational Concepts

Having presented the biological foundations of the problem and highlighted the need for computational methods, we now address associated computational aspects. PPI network analysis raises significant computational challenges due to their structural complexity and often substantial data size. This section aims to highlight these difficulties and present computational concepts to effectively overcome them, particularly optimization methods (especially metaheuristics) and parallelization approaches.

1.3.1 Computational Challenges in PPI Networks

Protein complex extraction presents several major computational challenges:

- **Noisy and incomplete data:** Interactions come from various experimental identification methods potentially containing errors or omissions, complicating reliable network modeling [21].
- **Structural diversity:** Protein complexes show significant structural variability, ranging from dense structures like the proteasome [83] to modular network organizations. This heterogeneity requires considering various topological properties during identification [23].
- **High computational cost:** Exhaustive exploration of all possible graph partitions quickly becomes infeasible as network size increases [79].
- **Need for efficient, scalable solutions:** Methods must handle large-scale networks while ensuring reliable, precise results [80].

These challenges justify using advanced computational approaches adapted to this problem type's specific constraints. In this context, optimization methods enable simultaneous consideration of multiple criteria related to identified complex quality [107], while parallelism provides an effective solution to control computational cost. The following subsection presents fundamental concepts related to optimization methods.

1.3.2 Optimization Methods

Optimization methods aim to identify the best possible solution, known as the optimal solution, from a set of candidate solutions called the *search space*. A solution is considered optimal when it maximizes or minimizes a specific characteristic or measure, known as the *objective*. This objective is formalized by a function f , referred to as the *objective function*, which we aim to optimize (either minimize or maximize) [45].

As a classic example, consider the Traveling Salesman Problem (TSP), whose goal is to determine the shortest path that allows a salesman to visit a given list of cities, exactly once each, while minimizing the total distance traveled [45]:

$$\min_{S \in \mathcal{E}} f(S) \quad \text{where} \quad \begin{cases} f(S) = \sum \text{of distances between successive cities in } S \\ \mathcal{E} = \text{set of all possible permutations of cities} \end{cases}$$

In this subsection, we begin by formulating our main problem as an optimization task. We then introduce the concept of a metaheuristic, followed by a presentation of the main categories of such methods. We describe in detail the two metaheuristics used in our approach. Finally, we highlight the importance of parallelism as a key driver for improving computational performance.

1.3.2.1 Problem Formulation as an Optimization Task

The problem of extracting protein complexes from a protein–protein interaction network can be formalized as an optimization problem defined by the triplet $(\mathcal{S}, f, C)$, where:

- $\mathcal{S}$ is the search space, consisting of all possible configurations of subgraph sets from the graph $G = (V, E)$, representing groups of proteins that may form complexes;
- $f : \mathcal{S} \rightarrow \mathbb{R}$ is an objective function that assigns to each solution $x \in \mathcal{S}$ (i.e., a set of subgraphs) a real value assessing the global quality of the solution based on structural or biological properties of the subgraphs (complexes);
- C is the set of constraints imposed on each subgraph $C_i \in x$, such as:
 - $|V(C_i)| \geq 3$ (the subgraph contains at least three vertices);
 - C_i is connected in G .

The objective is to determine an optimal solution $x^* \in \mathcal{S}$ such that:

$$x^* = \arg \max_{x \in \mathcal{S}, x \text{ satisfies } C} f(x)$$

This abstract formulation provides a clear theoretical framework for complex extraction and paves the way for the use of optimization techniques [31]. Chapter 3 will detail the specific choices regarding solution representation, the definition of the objective function, and the exploratory search mechanisms.

1.3.2.2 Metaheuristic

The term *metaheuristic*, introduced by GLOVER [41] in 1986, combines the Greek prefix *meta-* (“beyond”) with the word *heuristic* (“to search”). According to GLOVER and SÖRENSEN, a metaheuristic is a *high-level algorithmic framework*, problem-independent, that provides general strategies to design specific heuristic algorithms⁸. A concrete implementation of this framework for a given problem is also called a metaheuristic.

Metaheuristics are intelligent optimization methods that explore the search space without exhaustively enumerating all possibilities. They start from one or more initial solutions and iteratively improve them while considering available resources (time, memory, etc.). The quality of the results thus directly depends on the time devoted to exploration. These methods aim to balance the exploration of promising new areas and the exploitation of good solutions already found [85, 86].

⁸A framework here refers to an adaptable structure guiding the design of algorithms for various problems.

1.3.2.3 Metaheuristic Classification

According to Glover, they can be divided into four main categories, as shown in Figure 1.4 [41]:

- **Local Search Methods:** These apply iterative modifications to a single solution, generating neighboring solutions (hence “local”) to reach an optimal one. Different search strategies determine which neighboring solution is selected. A solution that is better than all its neighbors is called a “local optimum”. Metaheuristics aim to escape such optima.
- **Population-Based Metaheuristics:** These aim to obtain an optimal solution by selecting and combining solutions from a set called a “population”. Evolutionary algorithms are the best-known representatives.
- **Constructive Metaheuristics:** These build a solution progressively from its basic components, rather than improving an existing one, by adding one element at a time to the current partial solution.
- **Hybrid Metaheuristics:** These approaches combine different metaheuristics to exploit the specific advantages of each.

This classification, along with typical examples, is illustrated in

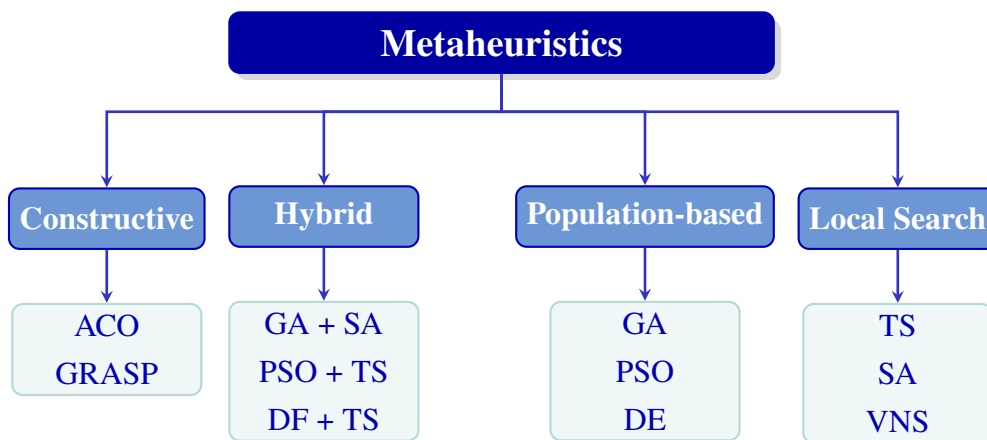


Figure 1.4: Classification of metaheuristics.

1.3.2.4 Tabu Search (TS)

Tabu Search, proposed by GLOVER in 1986, is an extension of local search methods. The term *tabu*, borrowed from Tongan, refers to what is forbidden or sacred [42]. This method allows escape from local optima by temporarily accepting non-improving solutions while preventing returns to recently visited solutions using a tabu list.

A typical implementation relies on two basic concepts — neighborhood and tabu list — along with two complementary mechanisms — intensification and diversification [42]:

- **Neighborhood:** The set $N(S)$ of solutions obtained by a local transformation of solution S (permutation, move, insertion).
- **Tabu List:** A short-term memory storing the latest moves or solutions to prevent cycles and encourage exploration. The set of admissible neighbors is denoted $\tilde{N}(S)$.
- **Intensification:** Aims to deepen promising regions, for instance by restarting the search from the best solution or altering the neighborhood to focus the exploration.
- **Diversification:** Avoids local stagnation using frequency memory to generate a radically different solution and restart the search in an unexplored area.

Tabu Search stops based on one of the following criteria [42]:

- Maximum number of iterations or timeout reached;
- No improvement over a set number of iterations;
- Objective function reaches a target value.

Algorithm 1.1 presents the standard pseudocode of Tabu Search.

Algorithm 1.1: Tabu Search

```

1 Choose or construct an initial solution  $S_0$ ;
2 Initialization :  $S \leftarrow S_0$ ;  $f^* \leftarrow f(S_0)$ ;  $S^* \leftarrow S_0$ ;  $T \leftarrow \emptyset$ ;
3 while Stopping criterion not satisfied do
4   Generate the neighborhood  $N(S)$  of the current solution  $S$  ;
5   Build  $\tilde{N}(S)$  by excluding tabu solutions ;
6   Select  $S' \in \tilde{N}(S)$  such that  $f(S')$  is minimal ;
7   if  $f(S') < f^*$  then
8      $f^* \leftarrow f(S')$  ;  $S^* \leftarrow S'$  ;
9   end
10  Update the tabu list  $T$  with the move or solution  $S'$  ;
11  if  $|T| > T_{max}$  then
12    Remove the oldest entry from  $T$  ;
13  end
14   $S \leftarrow S'$  ;
15 end
16 return  $S^*$  ;

```

1.3.2.5 Genetic Algorithm (GA)

The term *Genetic Algorithm*, now almost universally abbreviated as *GA*, was first used by John Holland. It is a population-based metaheuristic inspired by the natural process of evolution [42]. The core concepts of GA [43] are:

- **Population:** A set of strings encoding candidate solutions, called chromosomes or individuals, each composed of a series of genes. This population is used as a starting point to explore the search space. A more diverse initial population increases the chance of discovering better solutions.
- **Search Strategy:** Based on four key pillars:
 - **Selection:** Choosing two chromosomes (parents) randomly or via a specific method to apply crossover and mutation operations to generate new individuals (offspring).
 - **Crossover:** Involves defining crossover points to split each parent into parts and then exchanging them to create new offspring.

Parent 1 : [1 0 0 | 1 0 1]

Enfant 1 : [1 0 0 0 1 0]

Parent 2 : [1 1 0 | 0 1 0]

Enfant 2 : [1 1 0 1 0 1]

- **Mutation:** Similar to moves in Tabu Search, this modifies chromosomes by adding, deleting, inserting, or swapping alleles (gene values).
- **Population Reconstruction:** Adding offspring (some or all) or replacing certain solutions in the population with them, according to predefined rules.

Algorithm 1.2 presents the standard pseudocode of the Genetic Algorithm [42].

Algorithm 1.2: Genetic Algorithm

```

1 Choose an initial population of chromosomes ;
2 while Termination criterion not satisfied do
3   repeat
4     if Crossover condition is satisfied then
5       Select parent chromosomes ;
6       Choose crossover parameters (e.g., crossover point) ;
7       Perform crossover to generate offspring ;
8     end
9     if Mutation condition is satisfied then
10      Choose mutation points ;
11      Perform mutation on offspring ;
12    end
13    Evaluate the fitness of the offspring ;
14  until A sufficient number of offspring is generated;
15  Select the new population from offspring and parents (e.g., via elitist or roulette
    selection) ;
16 end
17 return Best solution found ;

```

1.3.2.6 Evaluation Metrics

The evaluation metrics used in our problem differ slightly from those typically employed to assess metaheuristics in prediction problems, as we clarify below [104].

- **Overlap Score (OS):** Measures the degree of overlap between a predicted complex M_j and a real complex C_i .

$$OS(M_j, C_i) = \frac{|M_j \cap C_i|}{\sqrt{|M_j| \times |C_i|}} \quad (1.3.1)$$

This measure ranges from 0 (no overlap) to 1 (identical). A threshold of 0.2 is used to determine whether a predicted complex sufficiently matches a real one [104].

- **Positive Predictive Value (PPV or Precision):** Measures the proportion of predicted complexes that are correct.

$$PPV = \frac{\sum_{j=1}^m \max_i |C_i \cap M_j|}{\sum_{j=1}^m |M_j|} \quad (1.3.2)$$

Where M_j is a predicted complex and C_i is a real complex.

- **Sensitivity (Sn or Recall):** Measures the proportion of real complexes that have been correctly identified.

$$Sn = \frac{\sum_{i=1}^n \max_j |C_i \cap M_j|}{\sum_{i=1}^n |C_i|} \quad (1.3.3)$$

- **F-measure (F-score):** Combines Precision (PPV) and Recall (Sn) into a single metric using the harmonic mean.

$$F\text{-score} = 2 \times \frac{PPV \times Sn}{PPV + Sn} \quad (1.3.4)$$

- **Accuracy:** Measures the proportion of correct predictions relative to the total number of predictions. Here, it is defined as the square root of the product of Precision and Recall:

$$Accuracy = \sqrt{Sn \times PPV} \quad (1.3.5)$$

- **Mean Matching Rate (MMR):** Assesses overall prediction performance by computing the average overlap scores between each predicted and real complex.

$$MMR = \frac{1}{m} \sum_{j=1}^m \max_i (OS(M_j, C_i)) \quad (1.3.6)$$

Where m is the total number of predicted complexes.

- **Jaccard Index:** Measures the similarity between two sets. For complexes, it is defined as:

$$Jaccard = \frac{|M_j \cap C_i|}{|M_j \cup C_i|} \quad (1.3.7)$$

Where M_j is a predicted complex and C_i a real one.

- **Covered Rate:** Measures the proportion of real complexes that are covered by the predicted complexes. In other words, it is the ratio of real complexes correctly predicted to the total number of real complexes.

$$\text{Covered Rate} = \frac{\text{Number of correctly predicted real complexes}}{\text{Total number of real complexes}} \quad (1.3.8)$$

This rate indicates the model's ability to identify true complexes.

- **Total Score:** Represents the overall model performance by summing F-score, Covered Rate, MMR, Accuracy, and Jaccard Index. It is computed as:

$$\text{Total Score} = F\text{-score} + \text{Covered Rate} + MMR + \text{Accuracy} + Jaccard \quad (1.3.9)$$

1.3.2.7 Need for Metaheuristic Parallelism

Metaheuristics offer efficient solutions to complex optimization problems where exhaustive search is impractical. Their success is acknowledged in many fields, particularly in bioinformatics. However, their main drawback lies in a high computational cost, especially for high-dimensional problems.

For example, *tabu search* can be slowed down by managing a large tabu list, while *genetic algorithms* often require a large number of iterations to converge.

These limitations are explained by several factors [107, 118]:

- **Complexity of the objective function:** A nonlinear or multimodal function requires numerous evaluations.
- **Large search space:** The size of the space requires intensive exploration.
- **High dimensionality:** This slows convergence due to the “curse of dimensionality”.
- **Suboptimal conceptual choices:** For instance, fully storing solutions in the tabu list increases comparison overhead.

Parallelism thus emerges as a natural solution to accelerate the execution of metaheuristics and improve the quality of explored solutions. The basic principles of parallelism are presented in the next subsection, while a detailed study of its integration into GA and TS is proposed in ??.

1.3.3 Parallel Computing and HPC Platforms

As previously mentioned, parallelism provides an effective solution to the computational challenges posed by metaheuristic approaches. In this subsection, we first present the principles of parallel computing and its main categories. We then introduce performance metrics used to evaluate it, followed by an overview of the main parallel programming models. Finally, we conclude with a presentation of high-performance computing (HPC) platforms.

1.3.3.1 Parallel Computing

In the early days of modern computing, machines were single-processor systems (CPUs), able to execute only one instruction at a time, sequentially. Their computational capacity followed Moore's Law, predicting a doubling of the number of transistors every two years. However, since the 1990s, physical limits of processors have made it difficult to maintain this trend [47].

Nevertheless, the growing need for computational power in various fields required an alternative approach to overcome these limitations [48]. Parallel computing emerged as a viable solution, allowing multiple independent tasks to be executed simultaneously, reducing computation time. Thanks to the development of parallel hardware—including multiprocessors, supercomputers, clusters, and computational grids—this approach has become an essential reality [49].

1.3.3.2 Types of Parallelism

There are three main types of parallelism illustrated in Figure 1.5 and detailed below:

- **Data Parallelism:** The data is split into independent subsets, each processed simultaneously by a distinct processing unit. This type is particularly effective for repetitive operations on large datasets, such as image processing or scientific simulations [49].
- **Task Parallelism:** The program is divided into independent or semi-independent tasks that can be executed in parallel. This approach is often used to handle distinct operations in a computational pipeline [49].
- **Pipeline Parallelism:** An execution flow is broken down into successive stages, processed simultaneously by different compute units. This type is typically exploited in pipelined architectures, such as modern CPUs or GPUs, where each compute unit handles one step before passing the result to the next [49].

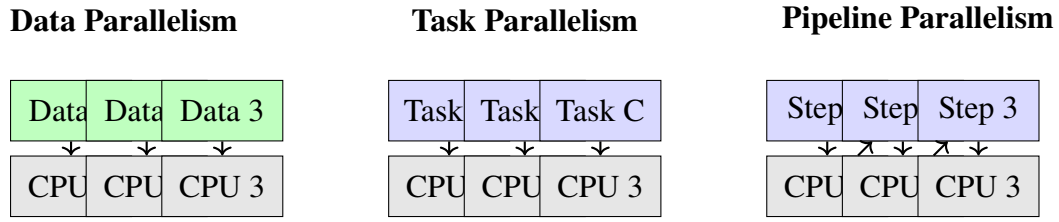


Figure 1.5: Illustration of the three types of parallelism: data, task, and pipeline

1.3.3.3 Performance Metrics

Two fundamental metrics are used to evaluate the performance of a parallel algorithm: **speedup** and **efficiency** [49].

- **Speedup:** It reflects the performance gain due to parallelism. It is defined as the ratio of the execution time of the sequential version, $T(1)$, to the parallel version using p processors, $T(p)$:

$$S(p) = \frac{T(1)}{T(p)} \quad (1.3.10)$$

An ideal speedup would be $S(p) = p$, but this is rarely achieved in practice due to synchronization, communication costs, or sequential code sections.

- **Efficiency:** It indicates how productively processors are used. It is calculated by normalizing the speedup by the number of processors:

$$E(p) = \frac{T(1)}{p \times T(p)} = \frac{S(p)}{p} \quad (1.3.11)$$

Efficiency ranges from 0 to 1: a value close to 1 indicates good resource utilization, while low efficiency reveals performance loss due to parallelization overhead.

1.3.3.4 HPC Platforms

High-performance computing platforms are designed to run compute-, memory-, and bandwidth-intensive applications. They integrate thousands of cores, high-throughput parallel storage, and low-latency networks. These environments are essential in fields like climate modeling, genomics, molecular dynamics, and deep learning [52], as illustrated in Figure 1.6.

- **HPC Clusters:** A high-performance computing cluster is a centralized infrastructure composed of multiple servers, called *nodes*, interconnected by a high-speed network (e.g., InfiniBand, Omni-Path). These nodes work together to run large-scale parallel applications. A cluster typically includes:
 - **Compute Nodes:** The main execution units, equipped with multicore CPUs and sometimes accelerators like GPUs or FPGAs. They handle heavy scientific or technical computations, often using models like MPI or OpenMP.

- **Management Nodes:** They manage the cluster through job scheduling tools (e.g., Slurm, OAR, PBS), allocating resources according to priorities or quotas.
- **Storage Nodes:** Dedicated to the parallel file system (e.g., Lustre, GPFS), they manage massive data access and ensure result persistence.
- **Login Nodes:** User access interface. These nodes allow users to connect, develop, compile code, and submit jobs to the scheduler.

Thanks to their homogeneous architecture and resource proximity, clusters enable high computational efficiency. They are commonly used in computing centers for numerical simulations, data analysis, or large-scale AI [116].

- **Grid Computing:** Unlike a cluster, a computational grid is a decentralized and distributed infrastructure. It federates several remote computing centers (clusters, supercomputers, cloud servers), often located in different countries or institutions. The goal is to pool heterogeneous resources to handle complex tasks collectively. A grid includes:
 - **Computing Sites:** Each site has its own cluster or set of machines and operates autonomously. Sites are connected via the Internet or a dedicated academic network.
 - **Middleware:** Software platforms such as Globus, gLite, or DIRAC handle global orchestration: single authentication (X.509 certificates), resource allocation, job submission, file transfer, and execution monitoring.
 - **Abstraction Layer:** The heterogeneity of environments is hidden by unified interfaces (containers, VMs, or REST APIs), allowing tasks to be executed without hardware-specific knowledge.
 - **Cooperative Sharing:** Resources are accessed through inter-institutional agreements defined by Service-Level Agreements (SLAs). Thus, a user can run a job on remote machines without direct access.

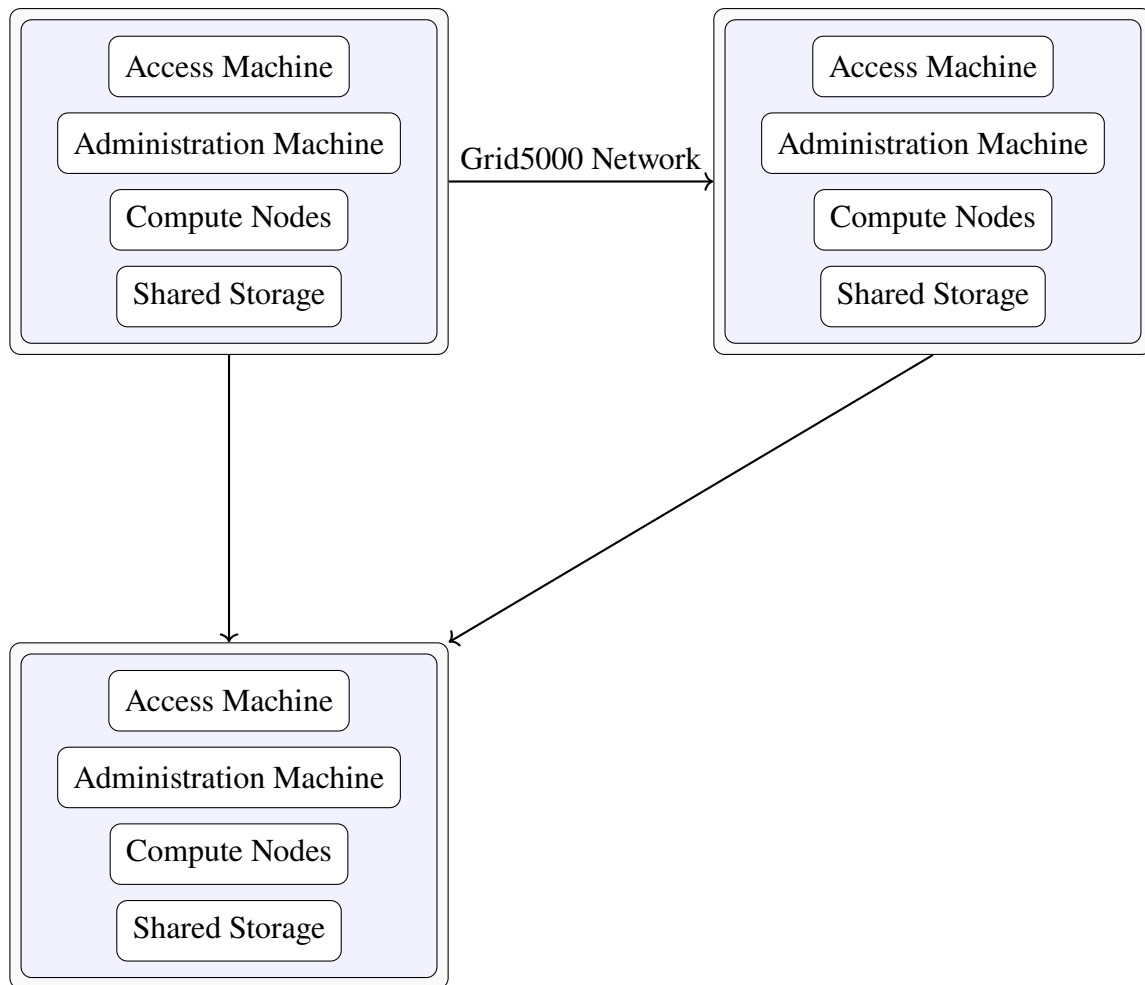


Figure 1.6: Architecture of a computational grid with multiple interconnected sites, each composed of full clusters

Computational grids are suited to large-scale collaborative projects (e.g., WLCG for CERN, EGI in Europe), requiring load balancing and high resilience [117]. In our project, we use the **Grid5000** grid, which exemplifies this model.

1.3.3.5 Parallel Programming Models

Parallel programming allows efficient use of multicore and distributed architectures to accelerate intensive tasks. Three main models are distinguished based on memory organization and process distribution: shared memory, distributed memory, and hybrid. These models are illustrated in Figure 1.7.

- i. **Shared Memory Model:** All execution units (threads) access a common memory space. Typical of multicore systems, this model allows fast communication via shared read/write operations but requires careful concurrency control. In Python, this is illustrated by the `ThreadPool` module from the `multiprocessing` package, which creates a pool of threads executing tasks in parallel using `pool.map` or `pool.apply` [115].

- ii. **Distributed Memory Model:** Each process runs on a node with its own memory, and communication occurs via message passing. The Ray framework [?] embodies this model in Python, enabling distributed function execution via `@ray.remote` and `.remote()`, with efficient data sharing through an object store and centralized initialization via `ray.init()`.
- iii. **Hybrid Model:** This model combines the previous two: Ray distributes tasks across cluster nodes, while each node uses `ThreadPool` to parallelize computation locally on its cores. This scheme is especially suited to HPC environments, as it leverages both local resources (multicore) and inter-node scalability.

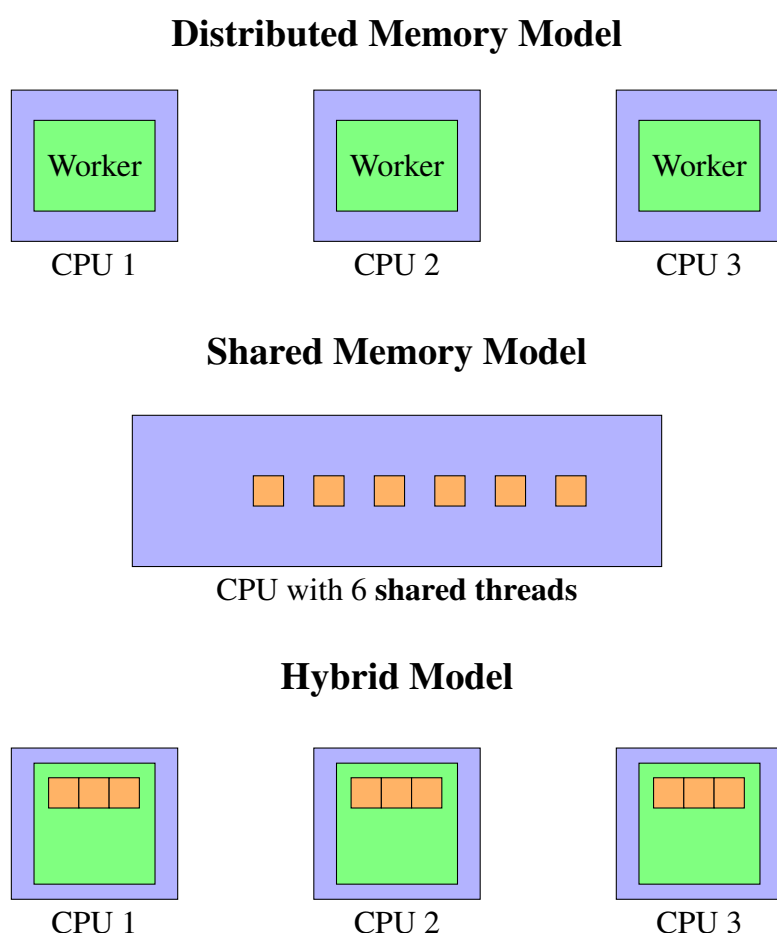


Figure 1.7: Comparison of parallel programming models: distributed memory, shared memory, and hybrid model.

1.4 Conclusion

Protein complex extraction relies on a combination of biological and computational approaches. Using metaheuristics and high-performance computing enhances the efficiency and accuracy of existing methods. This chapter introduced the essential elements for understanding the

challenges of this task. In the next chapter, we will analyze the various methods proposed in the literature and assess their effectiveness.

Chapter 2

State of the Art

2.1 Introduction

The identification of protein complexes within protein-protein interaction (PPI) networks has garnered significant interest in the scientific community, leading to diverse approaches ranging from exact methods to optimization techniques. Some approaches focus on detecting specific structural patterns of complexes, while others leverage optimization strategies—particularly metaheuristics—to improve the quality of identified clusters.

This chapter presents a state-of-the-art review of the main methods for protein complex extraction. First, we examine classical exact approaches, analyzing them through various evaluation criteria. Next, we explore metaheuristics applied to this domain, followed by parallel computing models adapted for these optimization techniques. Special attention is given to genetic algorithms and tabu search, which form the foundation of the hybrid approach proposed in this work.

2.2 Classical Exact Methods

Several computational methods exist for predicting protein complexes. Most assume that complexes are embedded in densely interconnected protein groups within PPI networks. These methods vary significantly in algorithmic methodologies and the types of biological data used for detection. Conceptually, they can be divided into two main groups: (1) those relying solely on PPI network topology, and (2) those incorporating additional biological data (e.g., core-attachment structures, evolutionary information, functional coherence, or cooperative/competitive interactions) to enhance prediction accuracy [24].

Furthermore, these methods employ different network exploration strategies. Some iteratively grow small "seed" clusters by adding/removing proteins or merging subgraphs based on similarity metrics. Others partition the network into subnetworks using divisive criteria. Recent approaches reduce the network to a mathematical space where complexes are identified using

traditional AI techniques. Based on this, we categorize topological methods into three classes: (i) cluster quality-based, (ii) node affinity-based, and (iii) network embedding-based. Below, we detail these categories and their underlying principles.

2.2.1 Cluster Quality-Based Methods

These methods extract protein complexes by optimizing a predefined cluster quality function. Starting from initial nodes, they iteratively modify clusters (adding/removing nodes) while evaluating cohesion and biological relevance using metrics like modularity or interaction density. Examples include CLUSTERONE (adaptive density-based clustering) and LCMA (density-modularity optimization). However, they often overlook deep node relationships, limiting prediction accuracy [7, 8].

2.2.2 Node Affinity-Based Methods

These techniques form clusters by leveraging inherent node connections, using affinity scores to guide expansion from seed nodes. Examples include MCL (random walks with inflation/expansion) and MCODE (local connection density scoring). Unlike cluster quality methods, they better capture biologically meaningful interactions but remain sensitive to seed selection and affinity score quality, risking incomplete/redundant clusters [7, 8].

2.2.3 Network Embedding-Based Methods

Network embedding transforms PPI networks into low-dimensional vector spaces, preserving node/edge attributes. Similarity in this space approximates graph-theoretic similarity, enabling clustering, link prediction, or classification via machine learning. Typical pipelines: (1) map nodes to d -dimensional vectors, (2) compute pairwise similarity to derive a weighted adjacency matrix, and (3) apply weighted clustering (e.g., GANE). While powerful for capturing complex structures, their computational demands limit scalability to large PPI networks [7].

Examples are illustrated in Figure 2.1.

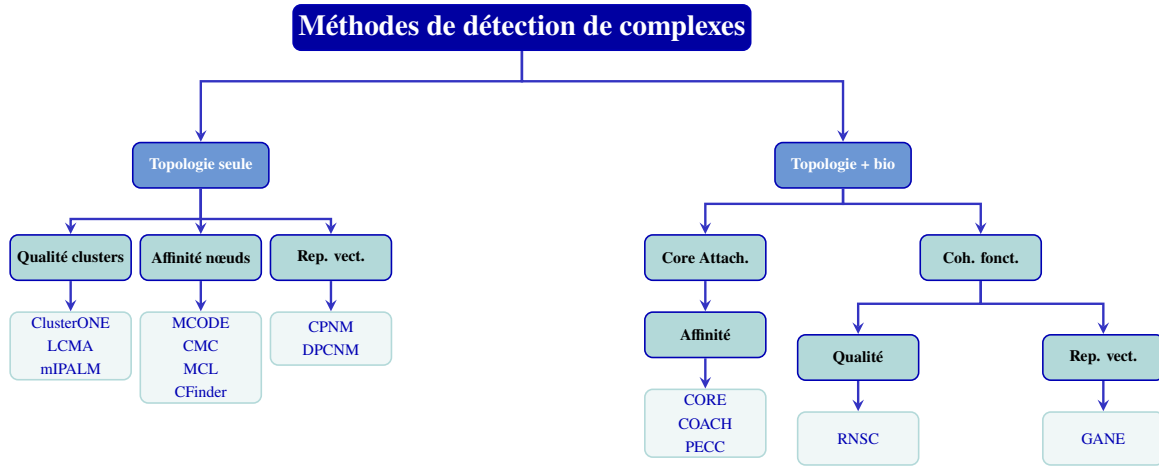


Figure 2.1: Classification of protein complex detection methods.

2.3 Metaheuristic Optimization Approaches

Complementing classical exact methods, metaheuristic-based optimization approaches have gained traction due to the NP-hard nature of protein complex detection—a combinatorial problem involving exhaustive subgraph exploration.

This section reviews key metaheuristics applied to this domain, categorized as: particle swarm optimization, ant colony algorithms, differential evolution, and multi-objective evolutionary algorithms. We emphasize genetic algorithm-based methods, which underpin our proposed hybrid approach.

2.3.1 Representative Works by Metaheuristic Category

Metaheuristics applied to protein complex extraction in PPI networks can be classified into several families, the most prominent being swarm-based approaches (PSO, ACO), differential evolution methods, and multi-objective algorithms. While these approaches have demonstrated promising results, recurring limitations persist—particularly in structured solution representation, handling experimental noise, and simultaneously addressing biological and topological criteria.

Particle Swarm Optimization (PSO) :

Sharafuddin et al. (2013) [62] proposed a PSO-based method to identify dense subgraphs. The algorithm models each solution as a particle navigating the search space, using modularity as the fitness function. This approach enables rapid convergence and improves functional module detection. However, PSO’s continuous nature requires adaptation mechanisms for discrete graphs, which may compromise topological precision. Moreover, the lack of explicit mechanisms to handle complex overlap and PPI data uncertainty limits its applicability to realistic biological contexts.

Ant Colony Optimization (ACO) :

Sallim et al. (Universiti Sains Malaysia) [64] introduced ACOPIN, an ACO-based algorithm for protein clustering in PPI networks. Incorporating topological and functional weights enhanced cluster relevance. However, this method relies on local probabilistic heuristics whose efficacy heavily depends on parameter tuning. Without global correction or structural validation mechanisms, cluster robustness remains sensitive to initial graph noise.

Hybrid Differential Evolution Methods :

Chowdhury et al. (2023) [63] proposed a hybrid model combining multi-objective differential evolution (DEMO) with a stochastic learning automaton (SLA). The approach simultaneously optimizes biological criteria: common neighbors, functional similarity, and solvent-accessible surface area. While this multi-objective formulation strengthens biological relevance, the search space complexity and continuous representation necessitate a trade-off between precision and computational cost. Furthermore, the absence of post-processing topological correction limits structural quality of extracted complexes.

Multi-Objective Evolutionary Algorithms :

Cutello et al. (2005) [65] explored protein structure prediction (PSP) via multi-objective evolutionary optimization. Their work demonstrates that simultaneous optimization of local and non-local energies improves predicted structure quality. Although conceptually extendable to PPIs, this method primarily targets 3D structure modeling. Direct application to noisy network complex detection is hindered by unaddressed local topology, overlap, and scalability.

Collectively, these approaches have advanced flexible bioinformatic modeling but share recurring limitations: difficulty handling explicit complex overlap, lack of robust false positive/negative controls, and insufficient integration of topological exploration with biological validation. These challenges motivate the design of hybrid metaheuristics adaptable to the imperfect, multidimensional nature of real biological networks.

2.3.2 Genetic Algorithm-Based Methods

Given that our methodology hybridizes genetic algorithms (GAs) with tabu search, we detail two major GA-based contributions for protein complex extraction. While promising, their limitations inspire improvements in our work.

2.3.3 Genetic Algorithm Methods: PPI-GA and GAGC

Among evolutionary approaches for protein complex detection, two methods stand out: PPI-GA by Shirmohammady et al. (2021) [66] and GAGC by Liu et al. (2023) [60]. We critically analyze their structural, biological, and algorithmic limitations.

2.3.3.1 Chromosome Representation

- PPI-GA [66]: Each chromosome is a permutation of N integers representing proteins. Cluster assignment compares values to indices: if a value exceeds its index, the protein initiates a new cluster; otherwise, it joins an existing one.
 - Example: For $P = [9, 3, 2, 1, 4, 6, 10, 7, 5, 8]$, the first value ($9 > 1$) creates cluster 1, the second ($3 > 2$) creates cluster 2, the third (2) assigns protein 3 to cluster 2, etc.

This encoding reduces search space (N^N to $N!$), improving convergence speed. However, it enforces strict partitioning, prohibiting biologically realistic cluster overlap.

- GAGC [60]: Individuals are lists of complexes (each a protein list), explicitly allowing overlap. While biologically realistic, this flexibility increases crossover/mutation complexity to maintain subgraph coherence.

2.3.3.2 Population Initialization

- PPI-GA: Random permutations ignore PPI graph topology, potentially inefficient for search space exploration.
- GAGC: Uses IPCA to seed clusters via topology-guided expansion from weighted seed nodes (EIC algorithm). While initial solutions are graph-aware, population diversity may suffer from density bias.

2.3.3.3 Fitness Function

- PPI-GA: Multi-objective function maximizes intra-cluster density (μ_i), minimizes inter-cluster connections (ε_{ij}), and regulates cluster count K :

$$Of = \frac{CQ}{K} + \frac{\sum \mu_i - \sum \varepsilon_{ij}}{E}, \quad (2.3.1)$$

where CQ measures cluster compactness and E is total interactions. Structurally sound but omits key biological dimensions (e.g., cellular localization).

- GAGC: Uses F-measure against reference complexes, measuring precision but ignoring network modularity or noise robustness.

2.3.3.4 Search Strategy

1. Selection:

- PPI-GA: Tournament selection favors elites but may reduce diversity.
- GAGC: Fitness-proportional selection with elitism risks premature convergence.

2. Crossover:

- **PPI-GA:** Single-point crossover may disrupt functional substructures.
- **GAGC:** Overlap-score-based crossover preserves connectivity at higher computational cost.

3. Mutation:

- **PPI-GA:** Random perturbations lack topological guidance.
- **GAGC:** Random complex mutations lack biological constraints.

4. Population Update:

- **PPI-GA:** Full generational replacement may discard promising solutions.
- **GAGC:** Elite retention improves stability but lacks adaptive exploration-exploitation balance.

Neither method addresses PPI data uncertainty (false positives/negatives) or integrates auxiliary biological data (GO annotations, localization). Scalability to large networks remains unaddressed, particularly regarding parallelization.

2.4 Optimization and Parallelism

Parallel computing models tailored to metaheuristics have been proposed to address implementation challenges. Solutions range from search space partitioning to compute-intensive task parallelization. We focus on parallel genetic algorithms (GAs) and tabu search (TS), as they underpin our hybrid approach.

2.4.1 Parallel Genetic Algorithms (PGAs)

PGA models differ in population management (partitioned/unified) and parent selection topology. Major models (Figure 2.2) include:

1. **Global (Master-Slave) Model:** A central population managed by a master node delegates compute-intensive tasks (e.g., fitness evaluation) to slaves. Effective when evaluation dominates runtime but suffers from master bottleneck.
2. **Cellular Model:** A spatially structured population where individuals interact only with neighbors (e.g., Von Neumann/Moore topologies). Promotes local diversity but limits global information flow.
3. **Island Model:** Independent subpopulations (islands) evolve in parallel, periodically exchanging migrants. Key parameters:

- **Exchange Topology:** Ring, hypercube, mesh, random, or fully connected.
- **Migration Policy:** Frequency, migrant selection (elites/random), and replacement strategies.

Balances exploration and exploitation, ideal for distributed architectures.

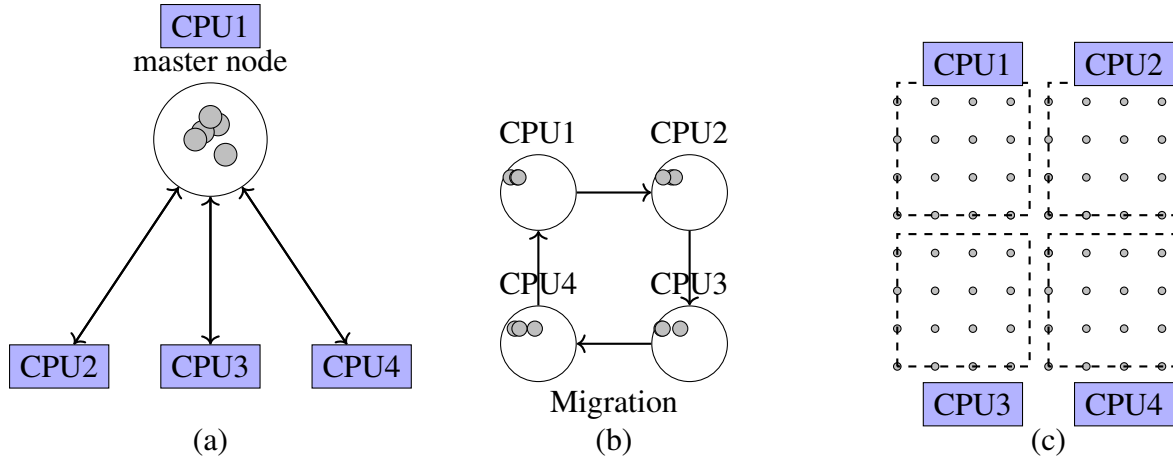


Figure 2.2: Different PGA models: (a) master-slave model, (b) island model, (c) cellular model.

2.4.2 Parallel Tabu Search (PTS)

Crainic et al. [109] classified PTS strategies along three dimensions (Figure 2.3):

2.4.2.1 Taxonomy Dimensions

1. Control Cardinality:

- **1-control:** Centralized master-slave architecture.
- **p-control:** Decentralized independent searches ($p > 1$).

2. Coordination Type:

- **Rigid synchronization:** Fixed information exchange.
- **Knowledge-sharing synchronization:** Periodic structured exchanges (e.g., best solutions, tabu memories).
- **Collegial/Advanced collegial modes:** Asynchronous communication with solution or global guidance.

3. Search Differentiation:

- **SPSS/SPDS:** Single/multiple starting points with single/multiple strategies.
- **MPSS/MPDS:** Multiple points with single/multiple strategies (most general).

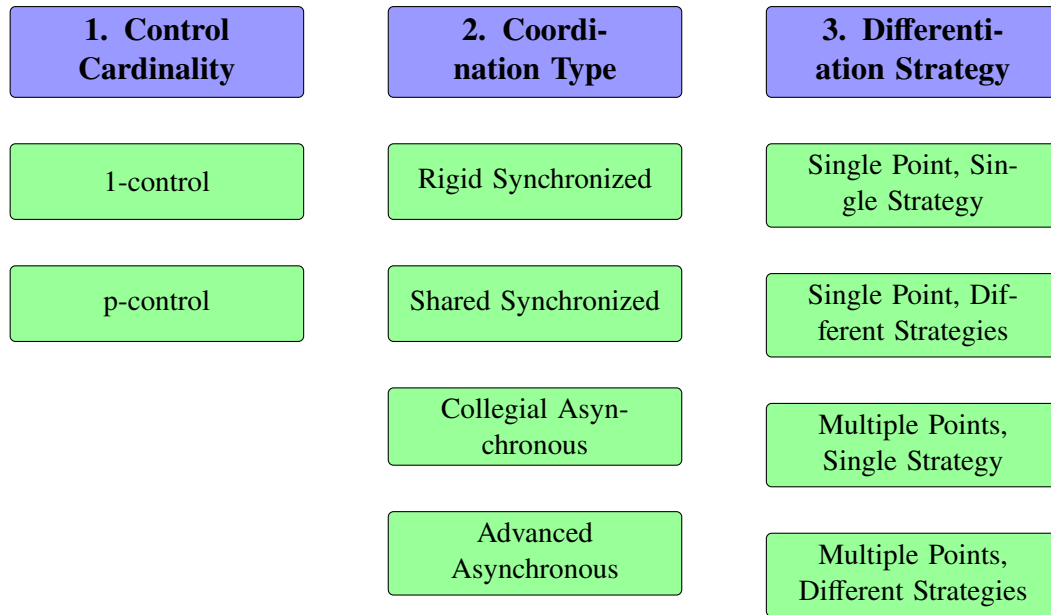


Figure 2.3: Hierarchical taxonomy of parallel tabu search models according to Crainic et al. [109]. Each dimension (control, coordination, differentiation) includes several modalities.

2.5 Conclusion

Existing approaches exhibit persistent limitations in precision and computational complexity. Classical methods overlook complex structural variability, while metaheuristics require careful initialization and substantial resources. A hybrid approach leveraging high-performance computing emerges as a promising solution. The next chapter details our proposed methodology to address these gaps.

Chapter 3

Proposed Methodology

3.1 Introduction

Having reviewed existing methods and their limitations in the literature, this chapter details our proposed methodology to address these shortcomings. We begin by presenting the overall architecture of our approach, followed by data collection and preprocessing steps. Next, we describe the design of our hybrid model, concluding with its parallel implementation.

3.2 Global Solution Architecture

This section outlines the key components of our methodology before delving into specifics. We first provide a textual description, followed by a synthetic workflow diagram.

3.2.1 Methodology Overview

Our methodology combines optimization algorithms (genetic and tabu search), classical literature methods, parallelization techniques, and integrated topological/biological data analysis to efficiently extract protein complexes from large PPI networks. The approach consists of:

1. **Data Collection:** PPI networks from *STRING*, *BIOGRID*, and *DIP* databases, supplemented with organism-specific reference complexes (gold standards) and auxiliary biological data.
2. **Preprocessing/Normalization:** Data cleaning, formatting, and normalization to construct a weighted PPI network where each interaction carries a confidence score reflecting biological reliability.
3. **Optimization Model Design:**
 - Novel solution representation schema overcoming limitations of existing approaches

- Multi-objective fitness function incorporating topological features and biological weights
- **Population Initialization:** Hybrid seeding using three literature methods (EWCA, MPC-C, SEDMTG) plus random initialization (RND) to ensure initial solution diversity/quality
- **Evolutionary Search Strategy:**
 - Binary tournament parent selection
 - Crossover of selected parents
 - Local optimization via tabu search applied to child complexes
 - Elitist population updating

This GA-TS hybridization leverages global exploration (GA) and local refinement (TS) capabilities.

4. **Parallelization Framework:** Hybrid shared/distributed memory model:

- **Island Model:** Implemented via *Ray* framework, where each worker process represents an island executing full GA cycles. Periodic migration occurs under defined policies.
- **Parallel Tabu Search:** Within-island workers optimize individual complexes via *Ray*, accelerated further by `multiprocessing.ThreadPool` for neighborhood generation.
- **Master-Slave Evaluation:** Per-island workers compute solution metrics (F-measure, Jaccard index) post-generation.

3.2.2 Methodology Workflow

Figure 3.1 illustrates this pipeline from data acquisition to parallel optimization:

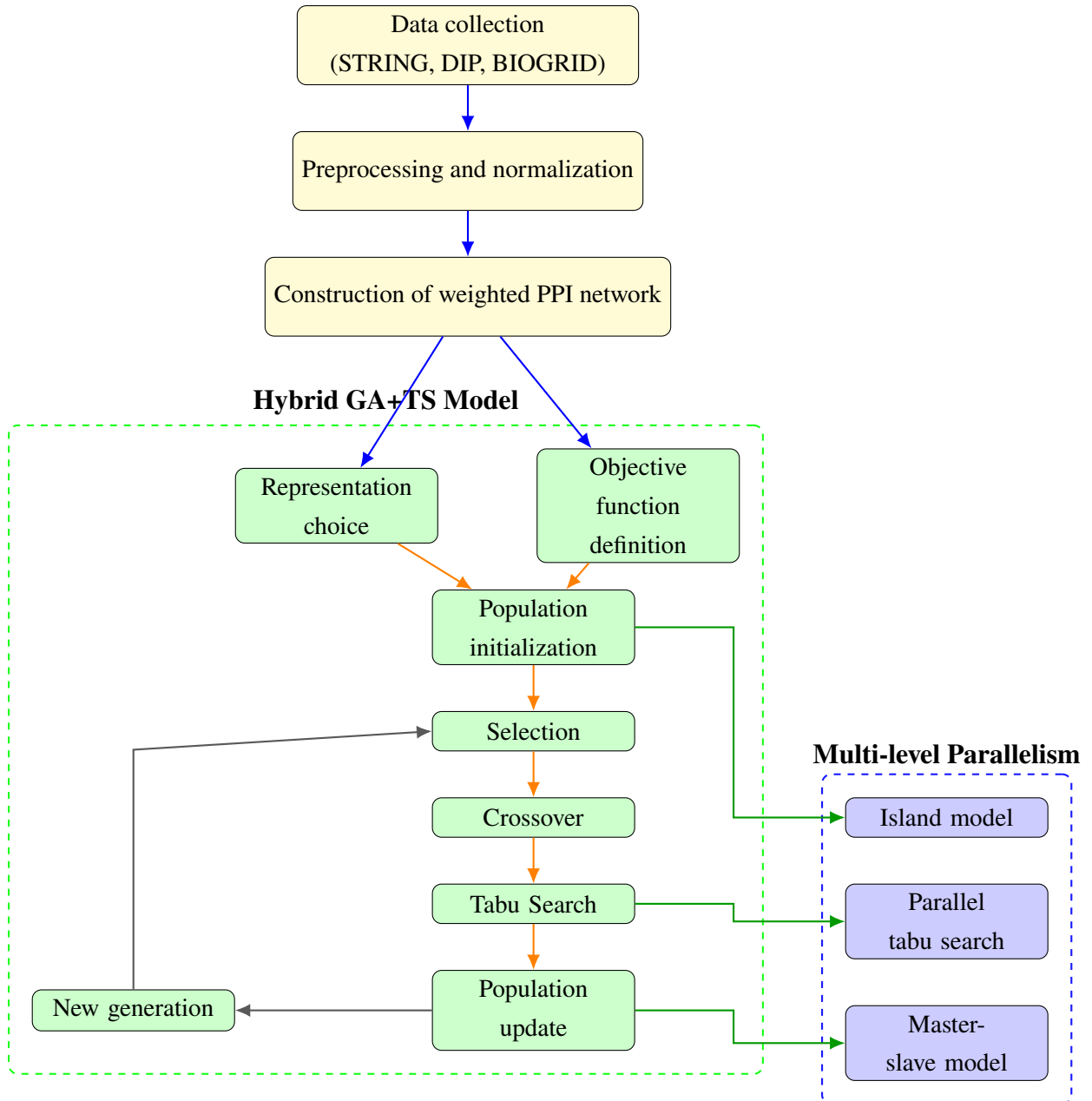


Figure 3.1: Global workflow of the methodology: from initialization to parallel optimization

3.3 Data Collection and Preprocessing

Our methodology begins with comprehensive data preparation from multiple sources, followed by rigorous filtering and normalization.

3.3.1 Data Sources

We integrate three data categories:

3.3.1.1 Protein Interaction Databases

- **STRING** [67]: Comprehensive PPI database featuring:
 - Coverage of 5,000+ organisms
 - Multi-evidence interactions (experimental, predicted, literature-mined)
 - Confidence scores (scale from 0 to 1)
 - Access: <https://string-db.org> (v12.0, 2024)
- **BIOGRID** [68]: Experimentally validated PPIs:
 - Focus on *S. cerevisiae*, *H. sapiens*, *M. musculus*, and 70+ other models
 - Monthly updates (50,000 new interactions/month)
 - Access: <https://thebiogrid.org> (v4.4, 2024)
- **DIP** [69]: Curated experimental PPIs:
 - Specialization in *S. cerevisiae* (150+ species supported)
 - Structural annotations (PDB links when available)
 - Access: <http://dip.doe-mbi.ucla.edu> (v2024.01)

3.3.1.2 Gold Standard Complexes

- **CORUM** [70]: 4,300+ human/300+ mouse complexes with full metadata
- **Complex Portal** [71]: EMBL-EBI manually annotated structural complexes
- **hu.MAP** [102]: 12,000+ human complexes with functional annotations
- **SGD** [105]: *S. cerevisiae* complexes from literature curation

3.3.1.3 Auxiliary Biological Data

- **GO Slim** [72]: Simplified Gene Ontology terms for functional annotation
- **Compartments** [73]: Subcellular localization predictions
- **GEO** [74]: Gene expression profiles (RNA-seq/microarray)
- **UniProt ID Mapping** [75]: Protein identifier unification

3.3.2 Data Filtering and Normalization

We applied sequential cleaning steps:

- Removal of redundant/self-interactions
- Organism-specific filtering
- High-confidence interaction selection (STRING/BIOGRID: score > 0.7; DIP: experimental only)
- Protein name standardization using UniProt IDs

3.3.3 PPI Network Weighting

To address experimental noise [87], we weighted interactions using four biological metrics:

- **Higher-order Common Neighbors (HCN):**

$$HCN(v, u) = \frac{|N(v) \cap N(u)|^2}{|N(v)| \times |N(u)| \times |N(v) \cup N(u)|} \quad (3.3.1)$$

- **Expression Correlation (PCC):**

$$PCC_{\text{norm}}(v, u) = \frac{\text{Pearson}(v, u) + 1}{2} \quad (3.3.2)$$

- **Functional Similarity (FS):**

$$FS(v, u) = \frac{\sum_{k=1}^M o_{vk} \cdot o_{uk}}{\sqrt{\sum_{k=1}^M o_{vk}^2} \cdot \sqrt{\sum_{k=1}^M o_{uk}^2}} \quad (3.3.3)$$

- **Subcellular Co-localization (SL):**

$$SL(v, u) = \frac{|SL(v) \cap SL(u)|^2}{|SL(v)| \cdot |SL(u)|} \quad (3.3.4)$$

Final edge weights combine these metrics:

$$W(v, u) = \frac{HCN + PCC + FS + SL}{4} \quad (3.3.5)$$

3.4 Hybrid Optimization Model Design

We present *OptClust-H*, our hybrid approach for protein complex extraction.

3.4.1 Solution Representation

Solutions are overlapping protein clusters reflecting biological multifunctionality:

- A *cluster* $C_i = \{P_{i_1}, \dots, P_{i_k}\}$ represents a potential complex
- A *solution* $S = \{C_1, \dots, C_m\}$ allows $P_i \in C_j \cap C_k$ ($j \neq k$)

3.4.2 Multi-Objective Fitness Function

Our fitness function $FF(C)$ integrates three key biological properties [76, 77]:

$$FF(C) = \underbrace{C(C)}_{\text{Cohesion}} + \underbrace{\mathcal{D}(C)}_{\text{Density}} + \underbrace{\mathcal{I}(C)}_{\text{Isolation}} - \underbrace{\mathcal{B}(C)}_{\text{Boundary}} + \underbrace{\mathcal{M}(C)}_{\text{Modularity}} \quad (3.4.1)$$

Table 3.1: Metric Coverage of Biological Properties

Metric	Density	Isolation	Function
Cohesion (C)	✓	✓	-
Density ($\mathcal{D}$)	✓	-	-
AIEW ($\mathcal{I}$)	-	-	✓
ABEW ($\mathcal{B}$)	-	✓	✓
AWM ($\mathcal{M}$)	✓	✓	✓

Key advantages:

- **Comprehensive:** Full coverage of critical complex properties (Table 3.1)
- **Balanced:** Uniform weighting of structural/biological criteria

3.4.3 Population Initialization

Protein complex detection faces a major challenge in the topological diversity of biological structures, which traditional unsupervised methods often fail to address due to their limitation to specific topologies. To overcome this, we propose a hybrid approach combining genetic algorithms with tabu search (GA+TS), requiring a high-quality initial population capable of covering diverse network structures. This initialization phase is detailed in Algorithm 3.1, illustrating our multi-strategy approach to solution space exploration.

3.4.3.1 Quality and Diversity Requirements

The initial population's quality is critical for effective GA evolution, guided by two principles:

1. **Quality:** Individuals must represent biologically plausible protein complexes with high precision.
2. **Topological diversity:** The population must cover varied structures for comprehensive search space exploration.

3.4.3.2 Initialization Strategies

We integrate three state-of-the-art complex detection methods:

- **EWCA** [99]: Specialized in core-attachment structures
- **SE-DMTG** [100]: Effective for dense, modular complexes
- **MPC-C** [101]: Detects community-structured complexes

(See Appendix A for methodological details.)

These are supplemented with **RND** (random initialization), generating connected complexes with sizes between $0.25 \times \text{golden_size}$ and $0.75 \times \text{golden_size}$, bounded by $\text{size}_{\min}$ and $\text{size}_{\max}$.

3.4.3.3 Implementation

Population construction employs:

- **EWCA:** o individuals via structural similarity threshold $ss \in [0.30, 0.62]$ (step 0.04)
- **SE-DMTG:** o individuals through random seed queue expansion
- **MPC-C:** o individuals with $\text{Score}(C) \in [0.1, 0.8]$ (step 0.1)
- **RND:** n_{alea} random individuals (Algorithm B.1)

The combined $3o + n_{\text{alea}}$ individuals form initial population P for our GATS hybrid approach.

Algorithm 3.1: POPULATIONINITIALIZATION

Data: G : PPI graph, R : references, o : individuals per method, n_{alea} : random individuals

Result: Initial population P

```

1  $P_1 \leftarrow \text{EWCA}(G, o)$  ; // Algorithm A.1.5
2  $P_2 \leftarrow \text{SE\_DMTG}(G, o)$  ; // Algorithm A.1.6
3  $P_3 \leftarrow \text{MPC\_C}(G, o)$  ; // Algorithm A.1.7
4  $P_4 \leftarrow \text{RANDOMINITIALIZATION}(G, n_{\text{alea}}, R, f, \text{max}, \text{min})$  ; // Algorithm B.1
5 return  $P_1 \cup P_2 \cup P_3 \cup P_4$ 

```

3.4.4 Search Strategy

This subsection details our evolutionary operators: selection, crossover, mutation, and population update.

3.4.4.1 Parent Selection

We employ *binary tournament selection* (Algorithm 3.2), randomly pairing individuals and selecting the fittest from each pair to promote solution quality.

Algorithm 3.2: BINARYTOURNAMENTSELECTION

Data: Population P (size N)

Result: Two selected parents

```

1 Randomly select  $i_1, i_2 \in P$ ;
2 if  $FITNESS(i_1) > FITNESS(i_2)$  then
    | ; // Algorithm B.13
3 | return  $i_1$ ;
4 else
5 | return  $i_2$ ;
```

3.4.4.2 Crossover Operator

Our fitness-based recombination (Algorithm 3.3) merges parental complexes while eliminating redundancies:

1. Create composite parent from both inputs
2. Determine child length between parental bounds
3. Evaluate and sort composite complexes
4. Select top complexes via tournament selection

Algorithm 3.3: CROSSOVER

Data: $parent_1, parent_2$

Result: Child individual

```

1  $composite \leftarrow parent_1 \cup parent_2$ ;
2  $childLen \leftarrow \text{RandomInt}(\min(|parent_1|, |parent_2|), \max(|parent_1|, |parent_2|));$ 
3 for  $c \in composite$  do
4 |  $fitness[c] \leftarrow \text{COMPUTE\_FITNESS}(c)$ ;
5 Sort  $composite$  by descending  $fitness$ ;
6 return BINARYTOURNAMENTSELECTION( $composite, childLen$ );
```

3.4.4.3 Mutation: Tabu Search

We enhance crossover-generated individuals through tabu search (Algorithm 3.4), applying four neighborhood moves:

- **Deletion:** Remove protein while maintaining connectivity

- **Insertion:** Add neighboring protein (Equation (1.2.3))
- **Replacement:** Simultaneous deletion/insertion
- **Creation:** Generate new complexes from dense proteins

The algorithm incorporates:

- Tabu memory to prevent cycling
- Diversification every $div_interval$ iterations
- Termination after $max_patience$ non-improving iterations or $max_iterations$

Algorithm 3.4: LOCALTABUSEARCH

Data: C_i : initial complex, G : PPI graph, R : reference complexes, $max_iterations$: maximum number of iterations, $max_patience$: iteration threshold without improvement, $div_interval$: diversification interval

Result: Optimized complex C_i^*

```

1   $C_i^* \leftarrow C_i$ ;
2   $\mathcal{T} \leftarrow \emptyset$ ;
3   $it \leftarrow 0$ ;
4   $noImprovement \leftarrow 0$ ;
5  while  $it < max\_iterations$  and  $noImprovement < max\_patience$  do
6       $\mathcal{V}_1 \leftarrow \text{NEIGHBORSBYREMOVAL}(C_i, G)$ ;           // Algo B.20
7       $\mathcal{V}_2 \leftarrow \text{NEIGHBORSBYINSERTION}(C_i, G)$ ;       // Algo B.21
8       $\mathcal{V}_3 \leftarrow \text{NEIGHBORSBYREPLACEMENT}(C_i, G)$ ;     // Algo B.22
9       $\mathcal{V}_4 \leftarrow \text{NEIGHBORSBYDENSECOMPLEXES}(C_i, G, R)$ ; // Algo B.23
10      $\mathcal{V} \leftarrow \mathcal{V}_1 \cup \mathcal{V}_2 \cup \mathcal{V}_3 \cup \mathcal{V}_4$ ;
11      $\forall v \in \mathcal{V}$ , compute  $FF(v)$ ;                             // Algo B.13
12      $v^* \leftarrow \arg \max_{v \in \mathcal{V} \setminus \mathcal{T}} FF(v)$ ;
13     if  $FF(v^*) > FF(C_i^*)$  then
14          $C_i^* \leftarrow v^*$ ;
15          $noImprovement \leftarrow 0$ ;
16     else
17          $noImprovement \leftarrow noImprovement + 1$ ;
18      $\mathcal{T} \leftarrow \text{UPDATETABULIST}(\mathcal{T}, v^*)$ ;
19     if  $it \bmod div\_interval = 0$  then
20          $\mathcal{V}_5 \leftarrow \text{DIVERSIFICATIONBYRAREPROTEINS}(G)$ ; // Algo B.24
21          $\mathcal{V} \leftarrow \mathcal{V} \cup \mathcal{V}_5$ ;
22      $it \leftarrow it + 1$ ;
23 return  $C_i^*$ 

```

3.4.4.4 Population Update

Our deterministic replacement (Algorithm 3.5) preserves elitism:

- Child replaces a parent *iff* its fitness is strictly superior
- Maintains constant population size
- Balances exploitation (selective replacement) and exploration (stochastic variation)

Algorithm 3.5: UPDATEPOPULATION

Data: Population P , child C , parents $\{P_1, P_2\}$

Result: Updated population P

```

1  $f_C \leftarrow \text{FITNESS}(C);$ 
2 for  $P_i \in \{P_1, P_2\}$  do
3    $f_i \leftarrow \text{FITNESS}(P_i);$ 
4   if  $f_C > f_i$  then
5     Replace  $P_i$  with  $C$  in  $P$ ;
6     Break from the loop
7   end
8 end
9 return  $P$ 

```

3.4.5 Hybrid Algorithm

Algorithm 3.6 presents **OptClust-H**, integrating global GA exploration with TS local optimization. Key phases:

1. Parallel population initialization
2. Generational evolution with:
 - Tournament selection
 - Fitness-based crossover
 - Tabu search optimization
 - Elitist population update
3. Continuous metric tracking

Algorithm 3.6: OPTCLUST-H

Data: PPI network $G = (V, E, W)$; reference complexes R ; population size N ;
maximum number of generations $G_{\max}$

Result: Best solution $best_indiv$ and associated evaluation metrics

```

1  $P_0 \leftarrow \text{PARALLELRANDOMINITIALIZATION}(G, R, o, n_{\text{alea}})$  ;
2  $best\_indiv \leftarrow \text{SELECTBESTINDIVIDUALS}(P_0)$  ;           // see Algorithm B.27
3  $metrics \leftarrow []$  ;
4  $metrics.append(\text{COMPUTEMETRICS}(s^*, R))$  ;           // see Algorithm 3.11
5 for  $g \leftarrow 1$  to  $G_{\max}$  do
6    $parent_1 \leftarrow \text{BINARYTOURNAMENTSELECTION}(P_0)$  ;
7    $parent_2 \leftarrow \text{BINARYTOURNAMENTSELECTION}(P_0)$  ;
8    $child \leftarrow \text{CROSSOVER}(parent_1, parent_2)$  ;
9    $sorted\_complexes \leftarrow$  complexes of  $I$  sorted in ascending order of  $FF(C, G)$  ;
10   $new\_child \leftarrow child - sorted\_complexes[nb\_improved\_complexes:]$  ;
11   $child \leftarrow sorted\_complexes[:nb\_improved\_complexes]$ 
12  foreach  $C \in child$  do
13     $new\_child.append(\text{LOCALTABUSEARCH}(C, G, R))$  ;
14  end
15   $P_0 \leftarrow \text{UPDATEPOPULATION}(P_0, new\_child, parent_1, parent_2)$  ;
16   $best\_indiv \leftarrow \text{SELECTBESTINDIVIDUALS}(P_0)$  ;           // see Algorithm B.27
17   $metrics.append(\text{COMPUTEMETRICS}(s^*, R))$  ;           // see Algorithm B.5
18 end
19 return  $best\_indiv, metrics$ 

```

3.5 Parallel Computing Integration

Our parallel implementation **OptClust-Hp** employs a hierarchical architecture:

3.5.1 Parallel Population Initialization

Utilizes `multiprocessing.pool.ThreadPool` for concurrent individual generation (Algorithm 3.7).

Algorithm 3.7: PARALLELRANDOMINITIALIZATION**Data:** G : PPI graph, N : population size, f : multiplicative factor, min , max , $maxC$ **Result:** Initial population P

```

1 INITGLOBALS( $G, min, max, maxC$ ) ;           // see Algorithm B.4
2  $n \leftarrow f \cdot N$ ;
3  $T \leftarrow$  number of available cores;
4 Create a thread pool  $Pool$  of size  $T$ ;
5  $R \leftarrow Pool.MAP(GENERATEINDIVIDUAL, \{1, \dots, n\})$  ;       // see Algorithm B.2
6  $I \leftarrow \{i \in R \mid i \neq \emptyset\}$  ;
7  $F \leftarrow$  fitness values associated with  $I$  ;
8  $\pi \leftarrow$  indices of  $F$  sorted in descending order ;
9  $P \leftarrow [I_{\pi_1}, I_{\pi_2}, \dots, I_{\pi_N}]$  ;
10 return  $P$ 

```

3.5.2 Island Model

Implemented via Ray with:

- Ring topology migrations every mig_inter generations
- Exchange of mig_size best individuals between neighbors
- Replacement of worst local individuals with migrants

Algorithm 3.8: EXECUTEISLAND

Data: P_i : local population of island i
 G : protein-protein interaction graph
 R : reference complexes
 g : total number of generations
 $migration_interval$: interval between two migrations
 $migration_size$: number of individuals exchanged
 i : island identifier
 n_cpu : number of available CPU cores

Result: Optimal individual $best_indiv$ and associated metrics

```

1  $metrics \leftarrow [ ]$  ;
2  $best\_indiv \leftarrow \text{SELECTBESTINDIVIDUALS}(P_i, 1)$  ;           // see Algorithm B.27
3  $metrics.append(\text{COMPUTEPARALLELMETRICS}(P_i, s^*, R))$  ;
4 for  $gen \leftarrow 1$  to  $g$  do
5    $parent_1 \leftarrow \text{BINARYTOURNAMENTSELECTION}(P_i)$  ;
6    $parent_2 \leftarrow \text{BINARYTOURNAMENTSELECTION}(P_i)$  ;
7    $child \leftarrow \text{CROSSOVER}(parent_1, parent_2)$  ;
8    $optimized\_child \leftarrow \text{MUTATEANDIMPROVE}(child, G, n\_cpu)$  ;
9    $P_i \leftarrow \text{UPDATEPOPULATION}(P_i, optimized\_child, parent_1, parent_2)$  ;
10   $best\_indiv \leftarrow \text{SELECTBESTINDIVIDUALS}(P_i, 1)$  ;
11   $metrics.append(\text{COMPUTEPARALLELMETRICS}(P_i, s^*, R))$  ;
12  if  $gen \bmod migration\_interval = 0$  then
13     $ind\_to\_send \leftarrow \text{SELECTBESTINDIVIDUALS}(P_i, migration\_size, G)$  ; // see
        Algorithm B.27
14     $\text{SENDTORIGHTNEIGHBOR}(i, ind\_to\_send)$  ;           // see Algorithm B.28
15     $received\_ind \leftarrow \text{RECEIVEFROMLEFTNEIGHBOR}(i)$  ; // see Algorithm B.29
16     $ind\_to\_replace \leftarrow \text{SELECTWORSTINDIVIDUALS}(P_i, migration\_size)$  ; // see
        Algorithm B.30
17     $P_i \leftarrow (P_i \setminus ind\_to\_replace) \cup received\_ind$  ;
18  end
19 end
20 return  $best\_indiv, metrics$ 

```

3.5.3 Parallel Tabu Search

Follows **1-control, rigid, MPSS** paradigm:

- Master (island) distributes complexes to Ray workers
- Independent optimization without inter-worker communication

- Multiple starting points with uniform strategy

Algorithm 3.9: MUTATEANDIMPROVE

Data: I : individual (list of complexes),
 n_cpu : number of available CPUs
Result: I' : improved individual

```

1 Initialize  $results \leftarrow []$ ;
2 for  $k \leftarrow 0$  to  $|I| - 1$  step  $n\_cpu$  do
3    $batch \leftarrow I[k : k + n\_cpu]$ ;    // extract a block of  $n\_cpu$  complexes
4    $pool \leftarrow []$ ;                  // list of Ray tasks for this batch
5   foreach  $C \in batch$  do
6      $pool.append(PARALLELTABUSEARCH.remote(C))$ ;
7   end
8    $batch\_results \leftarrow ray.get(pool)$ ;
9   Add  $batch\_results$  to  $results$ ;
10 end
11  $I' \leftarrow$  assembly of optimized complexes from  $results$ ;
12 return  $I'$ 

```

3.5.4 Neighborhood Parallelism

Within each TS worker, `ThreadPoolExecutor` accelerates:

- Deletion/insertion/replacement neighbors
- Dense complex injection

Algorithm 3.10: PARALLELTABUSEARCH**Data:** C_i : complex, G : PPI graph, R : reference complexes**Result:** Optimized complex C_i^*

```

1  $C_i^* \leftarrow C_i, \mathcal{T} \leftarrow \emptyset, it \leftarrow 0, noImprovement \leftarrow 0$ ;
2 while  $it < max\_iterations$  and  $noImprovement < max\_patience$  do
    // Launch neighborhood generation in parallel using a
    // ThreadPoolExecutor
3 In parallel:
    •  $\mathcal{V}_1 \leftarrow \text{NEIGHBORSBYDELETION}(C_i, G)$ 
    •  $\mathcal{V}_2 \leftarrow \text{NEIGHBORSBYINSERTION}(C_i, G)$ 
    •  $\mathcal{V}_3 \leftarrow \text{NEIGHBORSBYREPLACEMENT}(C_i, G)$ 
    •  $\mathcal{V}_4 \leftarrow \text{NEIGHBORSBYDENSECOMPLEXES}(C_i, G, R)$ 

     $\mathcal{V} \leftarrow \mathcal{V}_1 \cup \mathcal{V}_2 \cup \mathcal{V}_3 \cup \mathcal{V}_4$ ;
    // Evaluate generated neighbors
4 Evaluate  $FF(v)$  for each  $v \in \mathcal{V}$ ;
5  $v^* \leftarrow \arg \max_{v \in \mathcal{V} \setminus \mathcal{T}} FF(v)$ ;
6 if  $FF(v^*) > FF(C_i^*)$  then
7      $C_i^* \leftarrow v^*; noImprovement \leftarrow 0$ 
8 end
9 else
10      $noImprovement \leftarrow noImprovement + 1$ 
11 end
12 Update tabu list  $\mathcal{T}$  with  $v^*$ ;
13 if  $it \bmod div\_interval = 0$  then
14      $\mathcal{V}_5 \leftarrow \text{DIVERSIFICATIONBYRARELYVISITEDPROTEINS}(G)$ ;
15     Add  $\mathcal{V}_5$  if relevant;
16 end
17  $it \leftarrow it + 1$ 
18 end
19 return  $C_i^*$ 

```

3.5.5 Master-Slave Metric Computation

Distributes metric calculations across Ray workers, aggregating results post-evaluation.

Algorithm 3.11: PARALLELMETRICCOMPUTATION

Data: C : predicted complexes; R : reference complexes**Result:** Score vector s associated with C

```

1  $L \leftarrow \{\text{Accuracy, F-measure, Jaccard, MMR, PPV, Recall, Covered Rate}\};$ 
2  $pool \leftarrow [];$ 
3 foreach  $M \in L$  do
4    $pool.append(\text{compute\_M.remote}(C, R))$ 
5 end
6  $scores \leftarrow \text{ray.get}(pool);$ 
7  $Total\_Score \leftarrow \text{Accuracy} + \text{F-measure} + \text{Jaccard} + \text{MMR} + \text{Covered Rate};$ 
8  $s \leftarrow \text{dictionary } \{M_i : s_i\} \text{ with } s_i \in scores;$ 
9 return  $s \cup \{Total\_Score\}$ 

```

3.5.6 Global Parallel Algorithm

Algorithm 3.12 coordinates:

1. Parallel initialization
2. Island-based evolution
3. Result aggregation

Algorithm 3.12: OPTCLUST-HP($G, R, n_{\text{islands}}, g, \text{mig_inter}, \text{mig_size}, n_{\text{alea}}$)

Data: G : PPI network,

R : reference complexes,

n_{islands} : number of islands,

g : number of generations,

mig_inter : migration interval,

mig_size : migration size,

n_{alea} : number of randomly generated individuals

Result: Global best solution s^* and its associated metrics

```

1  $P \leftarrow \text{PARALLELPOPULATIONINITIALIZATION}(G, R, o, n_{\text{alea}})$ ; // see Algorithm B.25
2  $\mathcal{I} \leftarrow \text{SPLITPOPULATIONINTOISLANDS}(P, n_{\text{islands}})$ ; // see Algorithm B.26
3  $\text{workers} \leftarrow []$ ;
4 for  $i \leftarrow 0$  to  $n_{\text{islands}} - 1$  do
5   |  $\text{workers.append}(\text{RUNISLAND.REMOTE}(\mathcal{I}[i], G, R, g, \text{mig\_inter}, \text{mig\_size}, i))$ ;
6 end
7  $\text{results} \leftarrow \text{ray.get}(\text{workers})$ ;
8  $\text{solutions} \leftarrow$  extract  $\text{best\_indiv}$  from each result ;
9  $\text{metrics} \leftarrow$  concatenate all metrics from  $\text{results}$  ;
10  $s^* \leftarrow \text{SELECTBESTINDIVIDUALS}(\text{solutions}, 1, G)$ ; // see Algorithm B.27
11 return  $s^*, \text{metrics}$ 

```

3.6 Conclusion

This chapter introduced **OptClust-H**, combining GAs, TS, and multi-strategy initialization for protein complex detection. Its parallel variant **OptClust-Hp** employs hierarchical parallelism for scalability. The next chapter details implementation and experimental results on benchmark PPI networks.

Chapter 4

Implementation and Results

4.1 Introduction

Following the introduction of our two approaches, *OptClust-H* and *OptClust-Hp*, in the previous chapter, this chapter focuses on their implementation. We first present the experimental environment, both in terms of software and hardware. We then detail the experimental protocol and various test cases evaluated. Finally, we present the obtained results and provide a critical analysis.

4.2 Development and Execution Environments

The implementation of our two approaches, **OptClust-H** and **OptClust-Hp**, required the integration of different software and hardware environments. This section details the tools, libraries, frameworks developed, as well as the platforms used for development, experimentation, and evaluation.

4.2.1 Software Environment

All software components leveraged in this project rely on the Python ecosystem, ensuring both implementation modularity and execution efficiency, particularly for parallel processing. Standard libraries were combined with two custom frameworks designed specifically for this project, ensuring flexibility, reusability, and extensibility.

- **Programming language:** Python 3.12, chosen for its scientific libraries, readability, and widespread adoption in AI research.
- **Libraries used:**
 - **numpy** [28]: Foundational library for numerical computing with efficient multidimensional arrays.

- **pandas** [33]: Data manipulation and analysis toolkit.
 - **matplotlib** [?]: 2D/3D visualization library for convergence curves and PPI network visualization.
 - **ray** [29]: Distributed computing framework for parallel evolutionary algorithms and tabu search.
 - **cytoscape** [30]: External tool for biological network visualization and analysis.
 - **hpc_ga** [114]: Modular framework developed for genetic algorithms implementation (includes generic classes: **Population**, **Chromosome**, **Crossover**, **Mutator**, and parallel models). Installable via `pip install hpc_ga`.
 - **hpc_ts** [113]: Tabu search framework with modular components (neighborhood generation, tabu memory, evaluation) supporting multiple models. Installable via `pip install hpc_ts`.
- **Development tools:**
 - **Jupyter Notebook** [112]: Interactive environment for prototyping and reproducible research.
 - **Git** [111]: Version control system for collaborative code management. All data, code, results, and figures were versioned in a GitHub repository.

4.2.2 Hardware Infrastructure

Experiments were conducted across multiple hardware environments tailored to each approach's requirements: sequential for **OptClust-H** and parallel for **OptClust-Hp**.

4.2.2.1 Local Development Workstation

- **Device:** HP EliteBook 840 G4
- **CPU:** Intel Core i5-7300U @ 2.60GHz (2 cores, 4 threads)
- **RAM:** 8GB DDR4
- **Storage:** 256GB M.2 SATA SSD
- **OS:** Windows 10 Pro 64-bit

Used for initial development and **OptClust-H** execution, though thermal limitations restricted large-scale experiments.

4.2.2.2 Kaggle Cloud Platform

To overcome local limitations, partial experiments were migrated to Kaggle’s free cloud environment (12-hour session limit):

- **CPU:** Up to 2 Intel Xeon vCPUs
- **GPU:** Tesla P100/V100 (availability-dependent)
- **RAM:** 13GB

4.2.2.3 Grid5000 HPC Platform

Used for large-scale parallel execution of **OptClust-Hp**:

- **Site:** Nantes
- **Clusters:**
 - **econome (2012):** 22 nodes, 2× Intel Xeon E5-2660 (8 cores/CPU), 64GB RAM
 - **ecotype (2017):** 48 nodes, 2× Intel Xeon E5-2630L v4 (10 cores/CPU), 128GB RAM
- **Network:** 10Gb Ethernet
- **Scheduler:** OAR for job management

4.3 Experiments

This section describes the datasets and experimental parameters used for evaluation.

4.3.1 Datasets

Experiments utilized protein-protein interaction (PPI) networks and gold standard complexes for validation.

4.3.1.1 PPI Networks

Five networks were used (Table 4.1):

- Two large-scale human networks (STRING, BIOGRID)
- Three medium-scale yeast networks (DIP, STRING, BIOGRID)

Database	Organism	#Proteins	#Interactions	Density
STRING	Yeast	4,439	46,169	0.0047
STRING	Human	12,395	101,651	0.0013
BIOGRID	Yeast	4,233	24,464	0.0027
BIOGRID	Human	11,120	86,981	0.0014
DIP	Yeast	4,844	21,582	0.0018

Table 4.1: PPI network statistics

4.3.1.2 Gold Standard Complexes

Reference complexes were consolidated from multiple databases (Table 4.2):

Organism	Database	#Complexes	#Proteins	Avg. Size
Human	CORUM	2,562	4,004	5.08
Human	hu.MAP	4,777	9,205	5.49
Yeast	SGD	365	1,519	6.24

Table 4.2: Gold standard complex statistics

4.3.2 Algorithm Parameters

4.3.2.1 Genetic Algorithm Parameters

Parameter	Value
Individuals per method (o)	8
Random individuals (n_{alea})	26
Max generations (G_{max})	150

Table 4.3: Genetic algorithm parameters

4.3.2.2 Tabu Search Parameters

Parameter	Value
Max iterations	100
Patience threshold	20
Diversification interval	5
Complexes optimized/generation	400 (yeast), 600 (human)

Table 4.4: Tabu search parameters

4.3.2.3 Parallelization Parameters

Parameter	Value
CPUs per island	118
Number of islands	3

Table 4.5: Parallel configuration

4.3.3 Parameter Justification

Parameter values were empirically determined through extensive testing:

- Tabu search converges within 100 iterations (20 without improvement)
- Diversification every 5 iterations balances exploration/exploitation
- Parallel configuration maximizes Grid5000 resource utilization

4.4 Results

We executed the sequential approach on Kaggle/local machines and the parallel version on Grid5000. This section compares both approaches' outcomes.

4.4.1 Extracted Complex Analysis

Table 4.6 summarizes topological properties of predicted complexes:

Network	#Complexes	Avg. Size	Avg. Density	Avg. Overlap
OptClust-H				
DIP (yeast)	9,717	21.11	0.1786	53.53
BioGRID (human)	3,618	12.84	0.805	9.36
OptClust-Hp				
DIP (yeast)	9,780	21.43	0.1802	54.12
BioGRID (human)	3,650	12.97	0.807	9.62

Table 4.6: Topological characteristics of predicted complexes

Overlap with Gold Standards

Table 4.7 shows the percentage of predicted complexes matching reference complexes at varying overlap thresholds:

Threshold	BioGRID (human)		DIP (yeast)	
	OptClust-H	OptClust-Hp	OptClust-H	OptClust-Hp
≥ 1.0	3.48%	3.56%	1.92%	1.87%
≥ 0.7	15.34%	16.74%	4.63%	5.12%

Table 4.7: Complex overlap with gold standards

4.4.2 Visualization with Cytoscape

Our hybrid approach enables the detection of protein complexes with a wide diversity of topological structures. Figure 4.1 illustrates four examples of extracted complexes, each corresponding to a distinct structure: a sparse complex, a ring complex, a star complex, and a complete complex.

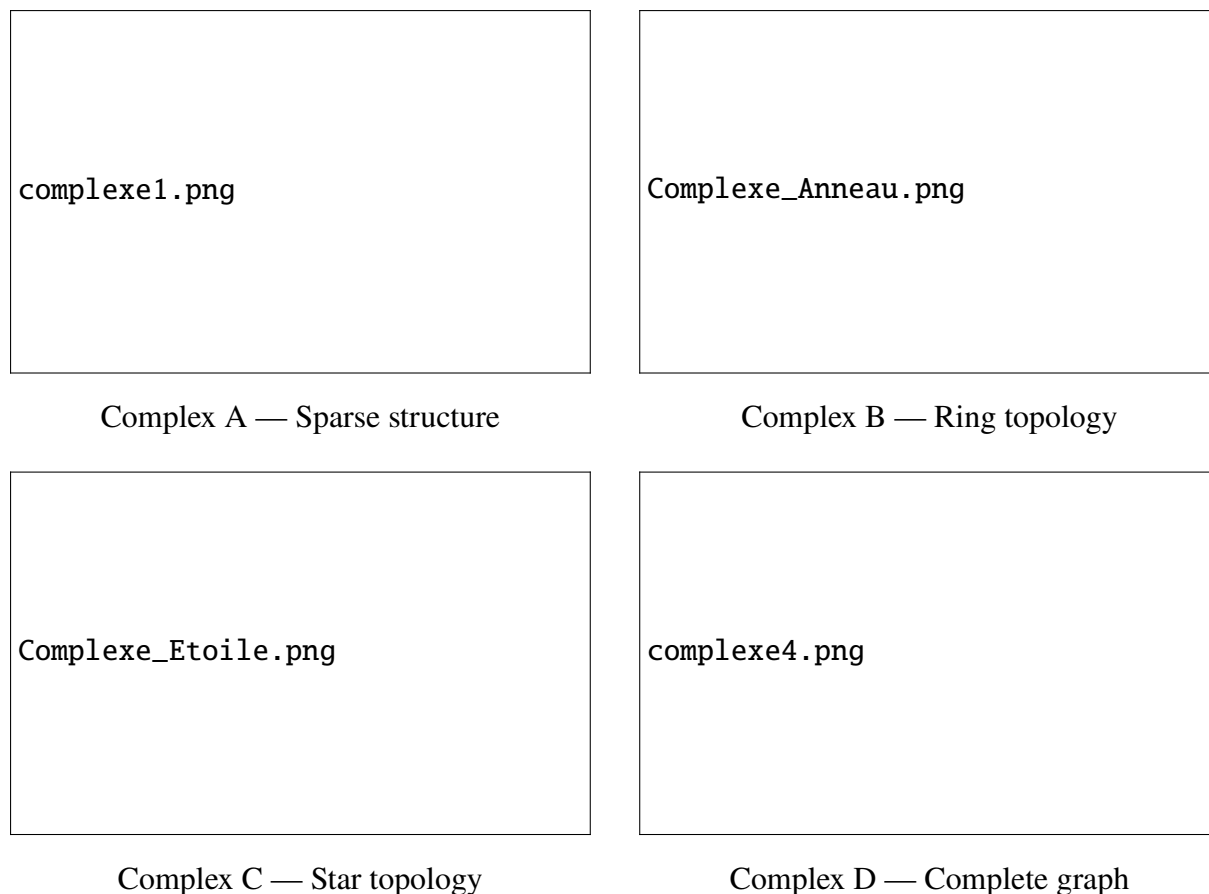


Figure 4.1: Four extracted protein complexes with different topological structures generated via Cytoscape.

4.4.3 Execution Time and Scalability

In this subsection, we analyze the performance of our approach in terms of execution time and scalability, particularly in the context of the parallel model OptClust-Hp.

4.4.3.1 Execution time.

The execution times of the sequential (OptClust-H) and parallel (OptClust-Hp) variants were measured on different protein-protein interaction (PPI) networks. Figure 4.2 presents the average times required to process each network. A significant reduction in execution time is observed due to parallelization, especially on large networks such as STRING_human and BioGRID_human. This confirms the efficiency of our parallel island model in a high-performance computing context.

Figure 4.3 shows the evolution of parallel efficiency as a function of the number of CPU cores used, for the five considered networks. It can be seen that efficiency gradually decreases as the number of cores increases, reflecting the classic trade-off between parallelism and communication overhead.

Finally, Figure 4.4 illustrates the *speedup* achieved on each network, highlighting the performance gains enabled by our parallel approach.



Figure 4.2: Execution time (in minutes) for each network, in sequential and parallel versions.

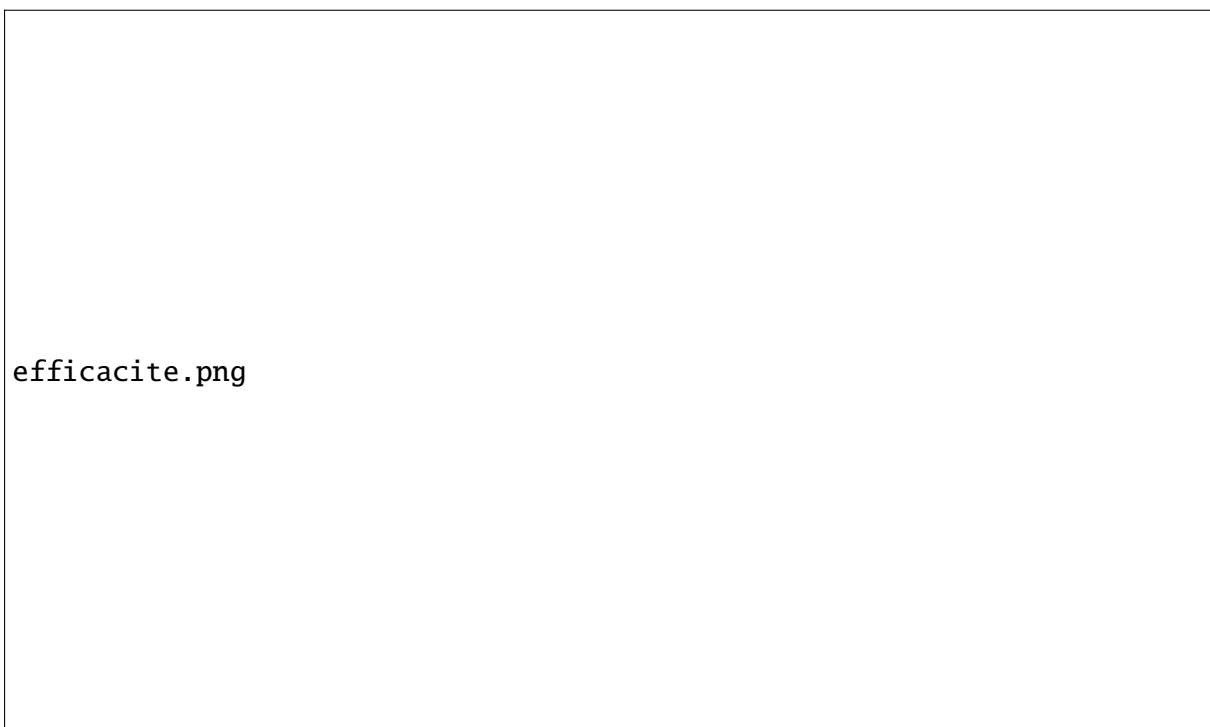


Figure 4.3: Parallel efficiency (%) according to the number of cores, for five PPI networks.

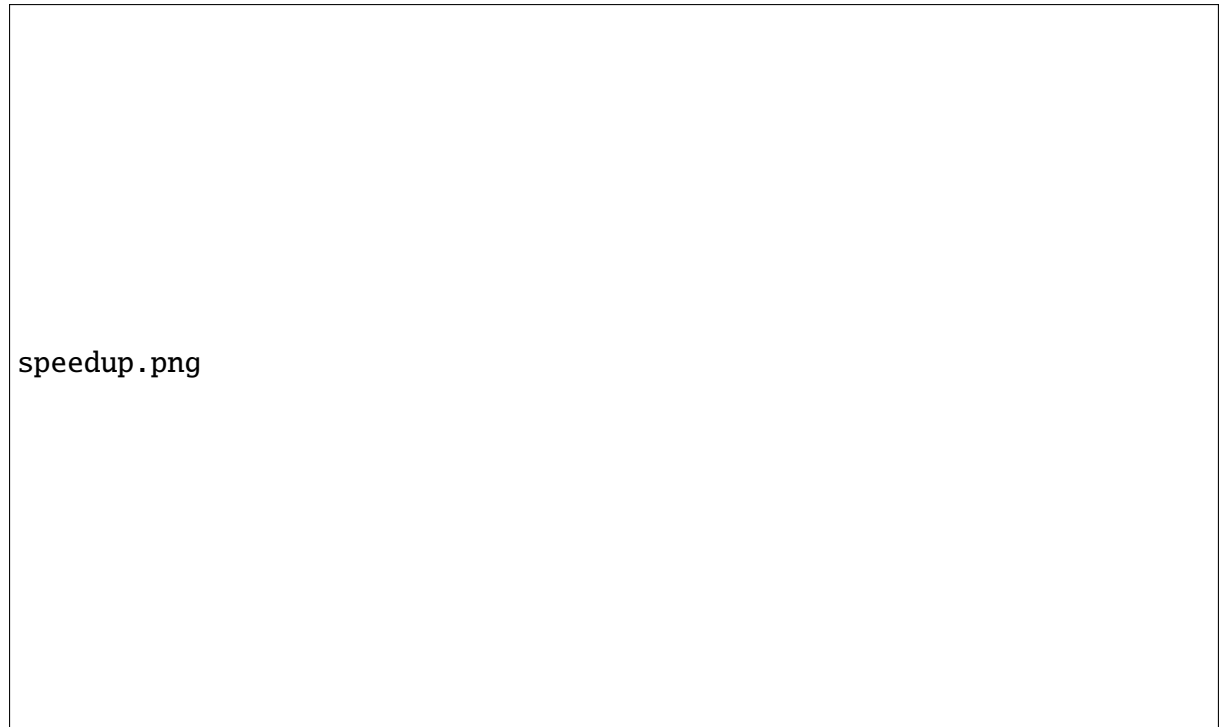


Figure 4.4: Speedup observed on five protein interaction networks.

4.4.3.2 Discussion.

These results show that the parallel architecture of our method is well-suited to HPC environments and that it effectively exploits available hardware resources. However, limitations appear beyond a certain number of cores, suggesting that additional optimizations (reduction of inter-island traffic, GPU porting, etc.) could further improve scalability.

4.5 Discussion and Analysis

In this section, we interpret the results obtained during the implementation of our two approaches and discuss their limitations.

4.5.1 Interpretation of Results

We begin by comparing our methods with state-of-the-art approaches, then present the contributions of our two approaches: the sequential hybrid approach and its parallelized version.

4.5.1.1 Comparison with state-of-the-art methods

To evaluate the performance of our approach compared to existing methods, we compared the results obtained by our two variants, *OptClust-H* and *OptClust-Hp*, with those of several state-of-the-art methods: MCL, MCODE, ClusterONE, COACH, EWCA, MPC-C, and SETMTG.

This comparison was performed on five PPI networks (yeast and human), using as reference complexes from biological databases such as CYC2008, MIPS, CORUM, among others.

Tables 4.8, 4.9, and 4.10 present the scores obtained according to different biological indicators, notably the *p-value* and the *co-localization score*, for the BioGRID, STRING, and DIP networks, respectively. Furthermore, Tables 4.11 and 4.12 summarize the values obtained for classical metrics such as F-measure, accuracy (ACC), Jaccard index, CR, MMR, as well as a total score, on human and yeast networks.

Overall, the two variants *OptClust-H* and *OptClust-Hp* produce competitive, if not superior, results, particularly on BioGRID networks (yeast and human) and STRING human. These performances reflect the ability of our approach to balance coverage (sensitivity) and specificity (precision). This robustness is mainly due to the combination of local tabu search and the parallel island model, which promotes a diversified and efficient exploration of the solution space.

In the same vein, we illustrate in this subsection a visual comparison of the reference complex **AP-1 (Adaptor Protein Complex)** and the corresponding complexes predicted by nine different methods, including our two variants. These predictions were extracted from the BioGRID_yeast network. Figure 4.5 highlights the topological diversity of the produced structures, as well as the proximity of our predictions to the expected biological structure.

Finally, Figure 4.6 provides a synthetic view of the total scores obtained by each method on the five networks, confirming the overall performance of our approaches compared to reference methods.

Overall, these results underscore the interest of our hybrid and scalable solution, capable of reconciling biological quality, topological diversity of modules, and computational efficiency on complex and large-scale networks.

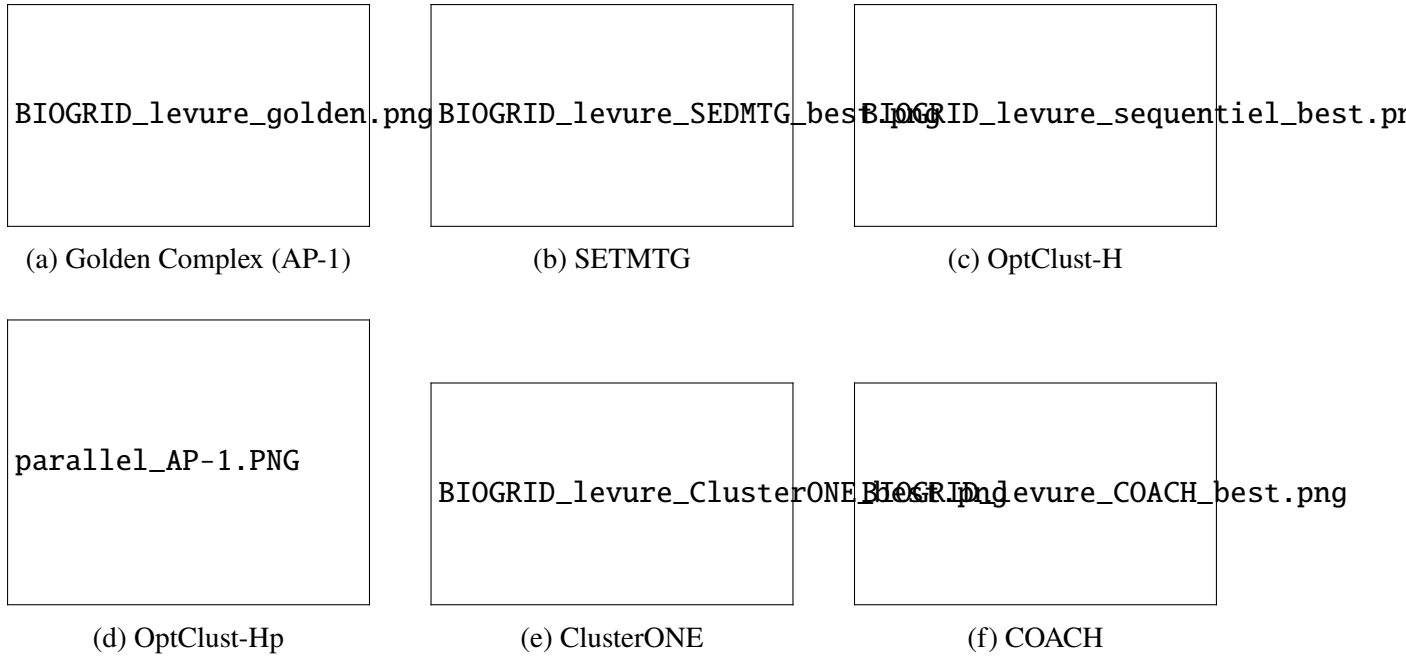


Figure 4.5: Visual comparison between the reference complex AP-1 and the complexes predicted by different methods (Part 1).

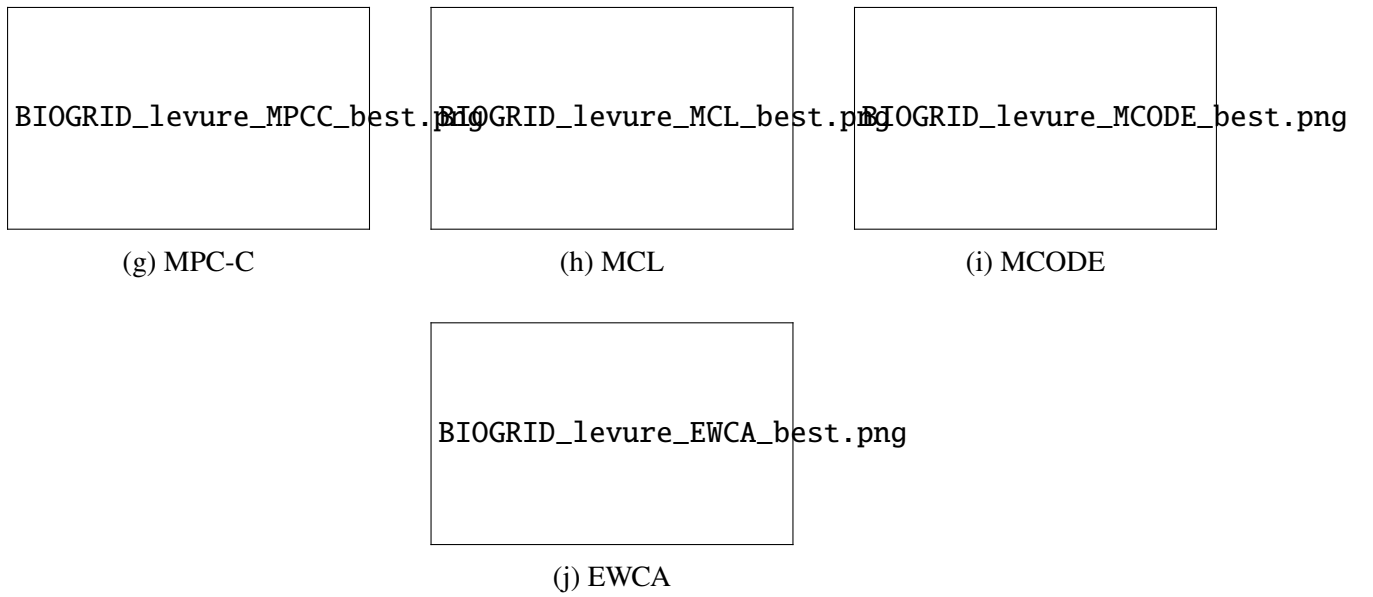


Figure 4.5: Visual comparison between the reference complex AP-1 and the complexes predicted by different methods (Part 2).

Table 4.8: Biological metrics for the methods on the BIOGRID network

BIOGRID_human			BIOGRID_yeast		
Method	p-value	Sc	Method	p-value	Sc
MCL	3.20e-03	1.0000	MCL	1.5917e-03	1.0000
MCODE	2.7814e-03	0.9960	MCODE	2.2342e-03	0.9980
ClusterONE	4.1599e-03	1.0000	ClusterONE	1.8200e-03	1.0000
COACH	3.5515e-03	1.0000	COACH	1.2095e-03	1.0000
EWCA	2.6000e-03	1.0000	EWCA	1.0000e-03	1.0000
SEDMTG	2.0000e-03	1.0000	SEDMTG	1.0000e-03	1.0000
MPC-C	3.2000e-03	1.0000	MPC-C	1.0000e-03	1.0000
OptClust-H	2.5096e-03	1.0000	OptClust-H	1.4773e-03	1.0000
OptClust-Hp	2.2646e-03	1.0000	OptClust-Hp	1.1378e-03	1.0000

Table 4.9: Biological metrics for the methods on the STRING networks

STRING_human			STRING_yeast		
Method	p-value	Sc	Method	p-value	Sc
MCL	2.6890e-03	1.0000	MCL	1.2463e-03	1.0000
MCODE	2.7235e-03	1.0000	MCODE	1.4673e-03	0.9323
ClusterONE	3.4879e-03	0.9995	ClusterONE	1.7465e-03	1.0000
COACH	2.8389e-03	1.0000	COACH	9.7272e-04	1.0000
EWCA	2.3000e-03	0.9998	EWCA	1.0000e-03	1.0000
SEDMTG	2.1000e-03	1.0000	SEDMTG	1.0000e-03	1.0000
MPC-C	2.6000e-03	1.0000	MPC-C	1.0000e-03	1.0000
OptClust-H	2.2356e-03	1.0000	OptClust-H	9.2788e-04	1.0000
OptClust-Hp	2.7613e-03	1.0000	OptClust-Hp	8.1293e-04	1.0000

Table 4.10: Biological metrics for the methods on the DIP_yeast network

DIP_yeast		
Method	p-value	Sc
MCL	1.4191e-03	1.0000
MCODE	1.9878e-03	0.9247
ClusterONE	1.8378e-03	1.0000
COACH	1.1719e-03	1.0000
EWCA	1.0000e-3	1.0000
SEDMTG	1.0000e-03	1.0000
MPC-C	1.0000e-03	1.0000
OptClust-H	1.2455e-03	1.0000
OptClust-Hp	1.1591e-03	1.0000

Table 4.11: Results of the methods on human networks

Method	F-measure	Acc	Jaccard	CR	MMR	Total Score
BIOGRID						
MCL	0.4903	0.4965	0.2761	0.8136	0.3592	2.4357
MCODE	0.3883	0.3954	0.2522	0.7598	0.3642	2.1599
ClusterONE	0.2989	0.3191	0.1425	0.9738	0.2362	1.9704
COACH	0.5022	0.5362	0.2544	0.9770	0.4008	2.6706
SE-DMTG	0.4404	0.4788	0.3932	0.5466	0.5548	2.4138
EWCA	0.5622	0.5943	0.3810	0.9396	0.5256	3.0027
MPC-C	0.4910	0.5310	0.3104	0.9482	0.4723	2.7529
OptClust-H	0.63	0.64	0.40	0.54	0.93	3.16
OptClust-Hp						
STRING						
MCL	0.3656	0.4235	0.1735	0.8565	0.2357	2.0548
MCODE	0.4026	0.4417	0.1982	0.8869	0.2893	2.2187
ClusterONE	0.2524	0.2922	0.1170	0.9587	0.1998	1.8201
COACH	0.4063	0.4792	0.1851	0.9838	0.3086	2.3630
SE-DMTG	0.3742	0.4351	0.2112	0.6906	0.3232	2.0343
EWCA	0.4042	0.4876	0.2167	0.9878	0.3551	2.4514

Method	F-measure	Acc	Jaccard	CR	MMR	Total Score
MPC-C	0.3640	0.4450	0.2040	0.9513	0.3234	2.2877
OptClust-H	0.43	0.50	0.22	0.33	0.99	2.47
OptClust-Hp						

Table 4.12: Results of the methods on yeast networks

Method	F-measure	Acc	Jaccard	CR	MMR	Total Score
BIOGRID						
MCL	0.3838	0.4293	0.1888	0.8436	0.2414	2.0869
MCODE	0.3294	0.3523	0.1607	0.8601	0.2417	1.9442
ClusterONE	0.2369	0.2841	0.1038	0.9753	0.1742	1.7743
COACH	0.4087	0.4847	0.1954	0.9877	0.3151	2.3915
SE-DMTG	0.3177	0.4044	0.1641	0.7984	0.2717	1.9562
EWCA	0.4314	0.5067	0.2341	0.9835	0.3446	2.5004
MPC-C	0.4179	0.4879	0.2376	0.9712	0.3556	2.4701
OptClust-H	0.45	0.51	0.23	0.31	0.99	2.51
OptClust-Hp						
DIP						
MCL	0.34	0.38	0.16	0.82	0.20	1.93
MCODE	0.33	0.35	0.18	0.81	0.25	1.94
ClusterONE	0.19	0.20	0.07	1	0.12	1.65
COACH	0.35	0.44	0.16	0.98	0.24	2.19
SE-DMTG	0.17	0.28	0.17	0.28	0.74	1.65
EWCA	0.37	0.45	0.23	0.32	0.98	2.35
MPC-C	0.34	0.42	0.22	0.32	0.95	2.27
OptClust-H	0.23	0.33	0.19	0.29	0.94	2.00
OptClust-Hp						
STRING						
MCL	0.3398	0.4340	0.1655	0.9205	0.2168	2.0767
MCODE	0.3640	0.4381	0.1808	0.9702	0.2483	2.2014
ClusterONE	0.2234	0.2922	0.0985	0.9139	0.1797	1.7076
COACH	0.3234	0.4304	0.1557	0.9603	0.2629	2.1327

Method	F-measure	Acc	Jaccard	CR	MMR	Total Score
SE-DMTG	0.3052	0.3784	0.1550	0.6490	0.2227	1.7103
EWCA	0.2991	0.4161	0.1830	1.0000	0.3004	2.1986
MPC-C	0.3575	0.4527	0.1952	0.9404	0.2833	2.2292
OptClust-H	0.32	0.42	0.19	0.28	0.99	2.21
OptClust-Hp						

comparison_scores.png

Figure 4.6: Comparison of total scores of different methods on various networks.

4.5.1.2 Contribution of the Hybrid Model

The hybrid approach *OptClust-H* combines a genetic algorithm with local tabu search to improve the extraction of protein complexes from noisy PPI networks. It offers several notable contributions:

- It improves solution quality by locally refining the results of genetic crossover through tabu search, leading to better fitness scores and more stable complexes.
- It establishes a balance between global exploration (through genetics) and local exploitation (through tabu), limiting the risks of premature convergence.
- It demonstrates robustness against experimental noise, thanks to biological weighting of interactions, which filters out unreliable data and strengthens relevant links.
- It introduces structural diversity during initialization by combining multiple population generation strategies, ensuring a richer exploration of the search space.

- It offers parametric flexibility, allowing easy adaptation to networks of different sizes and complexities.
- Finally, its modular architecture makes the model generalizable to other combinatorial optimization problems beyond bioinformatics.

4.5.1.3 Impact of Parallelization

The integration of parallelism in our approach led to significant performance improvements without degradation of solution quality. The parallel version, OptClust-Hp, is based on an island model, where multiple instances of the evolutionary process are executed in parallel with periodic exchanges of individuals, ensuring both diversity and convergence.

Experimental results highlight a substantial reduction in execution time, particularly on dense PPI networks like *STRING Human*, where speed gains of up to [to be completed] are observed, while maintaining or even improving biological scores.

This improvement is explained by:

- the simultaneous execution of multiple independent evolutionary processes,
- the structured distribution of the population across multiple islands,
- an asynchronous exchange strategy optimizing the trade-off between exploration and exploitation.

Beyond time savings, parallelization promotes diversity and mitigates premature convergence, improving search space coverage and enabling the detection of more precise complexes.

Compared to the few parallel approaches in the literature (such as MLPCD [119] or MapReduce methods [120]), our solution stands out for its specialization for biologically validated protein complexes and its lightweight and efficient architecture adapted to HPC environments.

OptClust-Hp thus represents an original and high-performance contribution for large-scale complex extraction in an intensive computing framework.

4.5.2 Limitations and Perspectives

This section presents a synthesis of the limitations identified in this work, then highlights the strengths of the proposed approach, before outlining the main improvement and extension perspectives.

4.5.2.1 Technical and methodological constraints

Although the proposed hybrid approach has shown satisfactory performance, several limitations were encountered, both technical and methodological:

- **Execution constraints:** The sequential version tested on the Kaggle platform was limited by a strict 12-hour execution ceiling, requiring fragmentation of experiments and manual processing of partial results. This factor slowed down experimentation and complicated analysis on large networks.
- **Restricted HPC access:** Although the parallel version (OptClust-Hp) was successfully validated on the Grid5000 platform, limited access time did not allow a complete evaluation of scalability (particularly regarding the number of cores or inter-island exchange topologies).
- **Sensitivity to hyperparameters:** The model remains dependent on several critical parameters (population size, tabu search intensity, mutation rate), whose tuning remains manual and empirically guided, which may limit the robustness and generalizability of the approach.
- **Weak biological integration:** The evaluation mainly relies on topological criteria and overlap with reference complexes. The lack of integration of enriched functional data, such as Gene Ontology annotations or gene expression profiles, limits the biological scope of the obtained results.

4.5.2.2 Scope and generality of the approach

One of the major strengths of the *OptClust-Hp* method lies in its generality. Although optimized for protein complex extraction, its algorithmic structure based on hybridization between evolutionary algorithms and tabu search, as well as its modular architecture, make it adaptable to various combinatorial optimization contexts.

The key elements of the model—subgraph representation, multi-criteria objective function, adaptive operators—are transferable to other domains, including:

- **Community detection** in social networks, including in the presence of noise and overlaps.
- **Transport or communication network optimization**, by searching for efficient substructures in terms of cost or reliability.
- **Variable selection** in machine learning, with each solution representing an optimal subset of features.
- **Task scheduling and resource management** in distributed or HPC environments.

This generality relies on the abstraction of data into graphs, the absence of strict biological assumptions, and parameterization flexibility. The parallel island model further enhances this portability to various computational environments.

4.5.2.3 Improvement perspectives

Several development axes can be considered to enhance the effectiveness and relevance of our approach:

- **Dynamic parameter adjustment:** The introduction of a self-adaptation mechanism (evolutionary strategies, online learning, meta-optimization) would reduce dependence on manual calibration while automatically optimizing performance according to the nature of the processed graph.
- **Enrichment of biological data:** The direct integration of functional annotations (Gene Ontology), gene expression profiles, or subcellular localization data would strengthen the biological relevance of extracted complexes, particularly by guiding evaluation toward functionally coherent structures.
- **Computational scalability:** A port to more intensive architectures, such as GPUs (CUDA/OpenCL) or large-scale multi-node platforms (via MPI or Spark), would enable processing of massive, even multi-species networks while maintaining reduced execution times.
- **Automation and deployment:** Integrating the solution into a reproducible bioinformatics pipeline (with Galaxy, Snakemake, or Nextflow) would facilitate its dissemination to the scientific community and its integration into high-throughput analytical workflows.

Thus, although some limitations remain, *OptClust-Hp* constitutes a robust, generic, and evolvable methodological foundation, paving the way for future contributions at the crossroads of bioinformatics, combinatorial optimization, and high-performance computing.

4.6 Conclusion

This chapter presented the implementation and experimental evaluation of the *OptClust-H* (sequential) and *OptClust-Hp* (parallel) approaches. After describing the software and hardware environment, as well as the experimental protocol, the results demonstrated the ability of these approaches to detect biologically coherent complexes while improving computational efficiency. The parallel version proved particularly effective in terms of execution time. Finally, the comparative analysis with the state of the art confirmed the competitiveness of the obtained performances, while highlighting avenues for improvement for future work.

General Conclusion

The automatic detection of protein complexes from protein-protein interaction (PPI) networks is a central challenge in systems bioinformatics. Due to the structural complexity of real PPI networks, often noisy and incomplete, and the combinatorial nature of the problem, it is necessary to design approaches that are robust, efficient, and scalable.

In this thesis, we proposed a protein complex extraction method called *OptClust-Hp*, based on a hybrid optimization framework combining evolutionary algorithms and tabu search, coupled with a parallel island architecture. This model was designed to simultaneously improve the biological quality of extracted complexes and runtime performance.

We first developed a sequential version of our approach, *OptClust-H*, implemented and tested on datasets from BioGRID, DIP, and STRING. This version uses preliminary weighting of the PPI graph to reduce noise effects and integrates a diversified initialization phase to generate varied complex structures. Tabu search then enables local refinement of solutions from genetic evolution.

In a second phase, we designed a parallel version, *OptClust-Hp*, successfully executed on the Grid5000 platform. This model distributes the population across multiple islands independently executing search phases, with controlled exchanges between islands. This structure increases explored diversity, accelerates convergence, and leverages HPC environment resources.

Experimental results demonstrate the relevance of our approach: the hybrid version improves F-measure, SC, and p-value scores compared to classical methods, while the parallel version provides notable computation time savings without quality degradation. Moreover, our graph weighting showed a positive effect in reducing the impact of noisy interactions.

Some limitations were nevertheless encountered. The execution of the sequential version was constrained by Kaggle platform restrictions (limited to 12 hours), requiring manual calculation splitting. Furthermore, although the parallel version was validated on Grid5000, larger-scale experiments could not be conducted due to unavailable resources. Methodologically, parameter sensitivity and the lack of functional information integration (such as Gene Ontology) partially limit biological interpretation of results.

Several short- and medium-term perspectives emerge. It would be relevant to make the algorithm more autonomous by integrating adaptive parameter optimization, to enrich PPI graphs with functional or expression data, and to port our solution to more massively parallel environments (GPU, MPI, Spark). Finally, integration into a bioinformatics platform like Galaxy

or Nextflow would enable broader dissemination of the approach to the scientific community.

In conclusion, this work enabled proposing an original, effective, and evolvable approach for protein complex extraction by combining metaheuristic optimization, biological robustness, and scalability. It constitutes a promising contribution to the analysis of complex biological networks and opens the way to broader and deeper future investigations. Possible improvements of the developed methods and future applications in other domains of computational biology.

Bibliography

- [1] Alberts, B., Johnson, A., Lewis, J., Morgan, D., Raff, M., Roberts, K., & Walter, P. (2014). *Molecular Biology of the Cell* (6th ed.). Garland Science, Taylor & Francis Group, New York. ISBN: 978-0-8153-4432-2.
- [2] Nelson, David L., & Cox, Michael M. (2013). *Lehninger Principles of Biochemistry* (6th edition). W.H. Freeman, New York. xxx + 1228 pp. ISBN: 978-1-4292-3414-6 (hardcover), 978-1-4641-0962-1 (eBook).
- [3] E. Buxbaum, *Fundamentals of protein structure and function*, 2nd ed. New York, NY, USA: Springer, 2015.
- [4] I. A. Sedov and Y. F. Zuev, "Recent advances in protein–protein interactions," 2023.
- [5] E. Golemis, *Protein-protein interactions: A molecular cloning manual*, Cold Spring Harbor, NY, USA: Cold Spring Harbor Laboratory Press, 2002, p. ix, 682.
- [6] E. M. Phizicky, S. Fields, *Protein-Protein Interactions: Methods for Detection and Analysis*. Microbiological Reviews, 1995, Volume 59, No. 1, pp. 94–123, 2-s2.0-0028926048.
- [7] S. Omranian, Z. Nikoloski, and D. G. Grimm, “Computational identification of protein complexes from network interactions: Present state, challenges, and the way forward,” *Comput. Struct. Biotechnol. J.*, vol. 20, pp. 2699–2711, 2022, doi: 10.1016/j.csbj.2022.05.049.
- [8] Z. Wu, Q. Liao, and B. Liu, “A comprehensive review and evaluation of computational methods for identifying protein complexes from protein–protein interaction networks,” *Briefings in Bioinformatics*, vol. 21, no. 5, pp. 1531–1548, Sep. 2020, doi: 10.1093/bib/bbz085. Published: 18 October 2019.
- [9] J. Zahiri, A. Emamjomeh, S. Bagheri, A. Ivazeh, G. Mahdevar, H. S. Tehrani, M. Mirzaie, B. A. Fakheri, and M. Mohammad-Noori, “Protein complex prediction: A survey,” *Genomics*, vol. 112, no. 1, pp. 174–183, Jan. 2020, doi: 10.1016/j.ygeno.2019.01.011.
- [10] Y. Jin, D. Turaev, T. Weinmaier, T. Rattei, and H. A. Makse, “The Evolutionary Dynamics of Protein-Protein Interaction Networks Inferred from the Reconstruction of Ancient Networks,” *PLOS ONE*, vol. 8, no. 3, e58134, Mar. 2013, doi: 10.1371/journal.pone.0058134.

- [11] R. Wang, H. Ma, and C. Wang, "An Improved Memetic Algorithm for Detecting Protein Complexes in Protein Interaction Networks", **Frontiers in Genetics**, vol. 12, Dec. 2021. doi: 10.3389/fgene.2021.794354.
- [12] Wang, R., Wang, C., Sun, L., & Liu, G. (2019). A seed-extended algorithm for detecting protein complexes based on density and modularity with topological structure and GO annotations. *BMC Genomics*, 20(637). <https://doi.org/10.1186/s12864-019-5956-y>
- [13] C. O'Connor, "Cell Division: Stages of Mitosis," *Nature Education*, vol. 1, no. 1, 2008.
- [14] Regulation of Tissue Oxygenation, in "Oxygen Transport", NCBI Bookshelf, 2013. [En ligne]. Disponible: [NCBI Bookshelf :contentReference\[oaicite:2\]index=2](#)
- [15] Doody R. S., et al., "A phase 3 trial of semagacestat for treatment of Alzheimer's disease," **New England Journal of Medicine**, vol. 369, pp. 341–350, 2013. :contentReference[oaicite:3]index=3
- [16] Hopkins C. R., "ACS Chemical Neuroscience Molecule Spotlight on Semagacestat (LY450139)" **ACS Chemical Neuroscience**, vol.1, no.8, pp.533–534, 2010. :contentReference[oaicite:4]index=4
- [17] Proteomics International, "Proteomics services," 2024. [En ligne]. Disponible: <https://www.proteomics.com.au/>
- [18] University of Pittsburgh Mass Spectrometry Core, "Sample submission and pricing," 2024. [En ligne]. Disponible: <https://www.chem.pitt.edu/mass-spec>
- [19] Reddi, P. P., "A practical guide to co-immunoprecipitation and mass spectrometry," *Methods in Molecular Biology*, vol. 2051, pp. 365–377, 2020.
- [20] A. J. Enright, S. Van Dongen, and C. A. Ouzounis, "An efficient algorithm for large-scale detection of protein families," *Nucleic Acids Research*, vol. 30, no. 7, pp. 1575–1584, 2002.
- [21] G. D. Bader and C. W. Hogue, "An automated method for finding molecular complexes in large protein interaction networks," *BMC Bioinformatics*, vol. 4, no. 1, p. 2, 2003.
- [22] R. Sharan, I. Ulitsky, and R. Shamir, "Network-based prediction of protein function," *Molecular Systems Biology*, vol. 3, p. 88, 2007.
- [23] Zhang, Y. (2018). "Multi-criteria complex detection". *Nucleic Acids Res.*
- [24] S. Srihari and H. W. Leong, "A Survey of Computational Methods for Protein Complex Prediction from Protein Interaction Networks," *Journal of Bioinformatics and Computational Biology*, vol. 11, no. 2, pp. 1230002, Apr. 2013, doi: 10.1142/S021972001230002X.

- [25] Wang R, Liu G, Wang C. Identifying protein complexes based on an edge weight algorithm and core-attachment structure. *BMC Bioinformatics*. 2019;20(471). <https://doi.org/10.1186/s12859-019-3007-y>.
- [26] I. M. A. Nooren, *NEW EMBO MEMBER'S REVIEW: Diversity of protein-protein interactions*. The EMBO Journal, Volume 22, July.
- [27] T. Nepusz, H. Yu, and A. Paccanaro, "Detecting overlapping protein complexes in protein-protein interaction networks," *Nat. Methods*, vol. 9, no. 5, pp. 471–472, May 2012.
- [28] Harris, C. R., Millman, K. J., van der Walt, S. J. et al. (2020). *Array programming with NumPy*. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>.
- [29] Moritz, P., Nishihara, R., Wang, S. et al. (2018). *Ray: A Distributed Framework for Emerging AI Applications*. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI'18), pp. 561-577. ISBN: 978-1-931971-47-8.
- [30] Shannon, P., Markiel, A., Ozier, O. et al. (2003). *Cytoscape: A Software Environment for Integrated Models of Biomolecular Interaction Networks*. *Genome Research*, 13(11), 2498-2504. <https://doi.org/10.1101/gr.1239303>.
- [31] B. Korte and J. Vygen, *Combinatorial Optimization: Theory and Algorithms*, 5th ed., Springer, 2012. Disponible en ligne (PDF) : <https://www.mathematik.uni-muenchen.de/~kpanagio/KombOpt/book.pdf> :contentReference[oaicite:1]index=1
- [32] Pedregosa, F., Varoquaux, G., Gramfort, A. et al. (2011). *Scikit-learn: Machine Learning in Python*. *Journal of Machine Learning Research*, 12, 2825-2830.
- [33] McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. In Proceedings of the 9th Python in Science Conference (SciPy 2010), pp. 51-56. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- [34] Szklarczyk D. et al. (2019). "STRING v11: protein-protein association networks with increased coverage, supporting functional discovery in genome-wide experimental datasets". *Nucleic Acids Research*, 47(D1), D607-D613.
- [35] Druker B.J. et al. (2001). "Efficacy and safety of a specific inhibitor of the BCR-ABL tyrosine kinase in chronic myeloid leukemia". *New England Journal of Medicine*, 344(14), 1031-1037.
- [36] Szklarczyk D. et al. (2021). "The STRING database in 2021: customizable protein-protein networks, and functional characterization of user-uploaded gene/measurement sets". *Nucleic Acids Research*, 49(D1), D605-D612.

- [37] Mering C. et al. (2002). “Comparative assessment of large-scale data sets of protein-protein interactions”. *Nature*, 417(6887), 399-403.
- [38] Ouellette B.F. et al. (2021). “BioGRID: a resource for studying biological interactions in yeast”. *Nucleic Acids Research*, 49(D1), D632-D639.
- [39] Park Y. et al. (2023). “Benchmarking algorithms for protein complex detection from PPI networks”. *Nature Methods*, 20(3), 215-221.
- [40] Wang J. et al. (2022). “Hybrid computational methods for protein complex prediction: recent advances and challenges”. *Briefings in Bioinformatics*, 23(1), bbab452.
- [41] K. Sörensen and F. Glover, “Metaheuristics,” in *Encyclopedia of Operations Research and Management Science*, 3rd ed., S. I. Gass and M. C. Fu, Eds. New York, NY, USA: Springer, in press.
- [42] F. Glover and M. Laguna, “Tabu search I,” *INFORMS Journal on Computing*, vol. 1, no. 3, pp. 190–206, 1999.
- [43] S. Luke, *Essentials of Metaheuristics*, 2nd ed. Department of Computer Science, George Mason University, 2016.
- [44] M. J. Kochenderfer and T. A. Wheeler, *Algorithms for Optimization*. MIT Press, 2019.
- [45] M. Bierlaire, *Optimization: Principles and Algorithms*. EPFL Press, 2010.
- [46] N. Kherici, *Optimisation Combinatoire*. Université Badji Mokhtar Annaba, 2020.
- [47] D. C. Hyde, *Introduction to the Principles of Parallel Computation*. Bucknell University, 1998.
- [48] F. Gebali, *Algorithms and Parallel Computing*. Wiley, 2011.
- [49] M. J. Quinn, *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill, 2003.
- [50] V. V. Dimakopoulos, *Parallel Programming Models*. University of Ioannina, Greece, n.d.
- [51] D. B. Kirk and W. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, 2nd ed. Morgan Kaufmann, 2012.
- [52] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2013.
- [53] N. Sadashiv and S. M. D. Kumar, “Cluster, Grid and Cloud Computing: A Detailed Comparison,” in *Proc. 6th Int. Conf. Comput. Sci. Educ. (ICCSE)*, 2011.

- [54] F. Berman, G. Fox, and T. Hey, *The Grid as the Future Computing Infrastructure*. Elsevier, 2003.
- [55] J. Diaz, C. Muñoz-Caro, and A. Niño, "A survey of parallel programming models and tools in the multi and many-core era," *IEEE Transactions on Parallel and Distributed Systems*, vol. 23, no. 8, pp. 1369–1386, Aug. 2012, doi: 10.1109/TPDS.2011.308.
- [56] Lawrence Livermore National Laboratory (LLNL), "Introduction to Parallel Computing Tutorial," 2023. [En ligne]. Disponible : <https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial>. [Consulté le : 24-02-2025].
- [57] A. Grama, A. Gupta, G. Karypis, et V. Kumar, *Introduction to Parallel Computing*, 2e éd. Boston, MA, USA: Addison-Wesley, 2003.
- [58] NVIDIA Corporation, *CUDA C++ Programming Guide*, Release 12.8, Feb. 14, 2025. [En ligne]. Disponible : <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [59] M. Mehdi, *Atelier de programmation parallèle sur le Cluster IBN-Badis (CERIST)*. CERIST, n.d.
- [60] Z. Liu and Y. Tang, "Research on protein complex recognition method based on genetic algorithm," in *2024 International Conference on Computer and Intelligent Control (ICCIC)*, Kuala Lumpur, Malaysia, June 29–30, 2024.
- [61] P. Moscato, "Memetic Algorithms: A Short Introduction", in **New Ideas in Optimization**, McGraw-Hill, 1999.
- [62] I. Sharafuddin, M. Mirzaei, M. Rahgozar, A. Masoudi-Nejad, "Protein-Protein Interaction Network Clustering Using Particle Swarm Optimization," School of Electrical and Computer Engineering, University of Tehran, Iran.
- [63] A. Chowdhury, P. Rakshit, A. Konar, and A. K. Nagar, "A Meta-Heuristic Approach to Predict Protein-Protein Interaction Network," in **Proceedings of [Nom de la conférence ou journal]**, [Lieu], [Année], pp. [Numéros de pages].
- [64] J. Sallim, R. Abdullah, A. T. Khader, "An Improved Ant Colony Optimization Algorithm for Clustering Proteins in Protein Interaction Network," School of Computer Sciences, Universiti Sains Malaysia.
- [65] V. Cutello, G. Narzisi, G. Nicosia, "A multi-objective evolutionary approach to the protein structure prediction problem," *J. R. Soc. Interface*, vol. 3, pp. 139-151, 2006. doi:10.1098/rsif.2005.0083.

- [66] N. Shirmohammady, H. Izadkhah, A. Isazadeh, "PPI-GA: A Novel Clustering Algorithm to Identify Protein Complexes within Protein-Protein Interaction Networks Using Genetic Algorithm," *Hindawi Complexity*, vol. 2021, Article ID 2132516, 14 pages. doi:10.1155/2021/2132516.
- [67] D. Szklarczyk et al., "STRING v11: Protein-protein association networks with increased coverage, supporting functional discovery in genome-wide experimental datasets", *Nucleic Acids Res.*, vol. 47, no. D1, pp. D607-D613, 2019.
- [68] C. Stark et al., "BioGRID: a general repository for interaction datasets", *Nucleic Acids Res.*, vol. 34, pp. D535-D539, 2006.
- [69] I. Xenarios et al., "DIP: the Database of Interacting Proteins", *Nucleic Acids Res.*, vol. 30, no. 1, pp. 303-305, 2002.
- [70] L. Giurgiu et al., "CORUM: the comprehensive resource of mammalian protein complexes-2019", *Nucleic Acids Res.*, vol. 47, no. D1, pp. D559-D563, 2019.
- [71] S. Velankar et al., "Complex Portal: an EBI resource for protein complex data", *Nucleic Acids Res.*, vol. 41, no. D1, pp. D1124-D1130, 2013.
- [72] Gene Ontology Consortium, "The Gene Ontology (GO) database and informatics resource", *Nucleic Acids Res.*, vol. 32, pp. D258-D261, 2004.
- [73] J. X. Binder et al., "COMPARTMENTS: unification and visualization of protein subcellular localization evidence", *Database*, vol. 2014, p. bau012, 2014.
- [74] T. Barrett et al., "NCBI GEO: archive for functional genomics data sets—update", *Nucleic Acids Res.*, vol. 41, no. D1, pp. D991-D995, 2013.
- [75] UniProt Consortium, "UniProt: the universal protein knowledgebase in 2021", *Nucleic Acids Res.*, vol. 49, no. D1, pp. D480-D489, 2021.
- [76] Li XL et al. (2008) *Bioinformatics* 24(13):i197-i205
- [77] Nepusz, T., Yu, H., & Paccanaro, A. (2012). Detecting overlapping protein complexes in protein-protein interaction networks. *Nature methods*, 9(5), 471-472.
- [78] Sharan, R., Ideker, T., & Kelley, B. (2007). Identification of protein complexes by comparative analysis of yeast and bacterial protein interaction data. *Journal of Computational Biology*, 12(6), 835-846.
- [79] Pritykin, Y., & Singh, M. (2015). Simple topological features reflect dynamics and modularity in protein interaction networks. *PLoS computational biology*, 11(10), e1004450.

-
- [80] Cho, Y. R., & Zhang, A. (2016). Predicting protein function by frequent functional association pattern mining in protein interaction networks. *IEEE/ACM transactions on computational biology and bioinformatics*, 13(4), 578-591.
 - [81] Talbi, E. G. (2009). *Metaheuristics: from design to implementation*. John Wiley & Sons.
 - [82] Alba, E. (2005). *Parallel metaheuristics: a new class of algorithms*. John Wiley & Sons.
 - [83] Brooks, P. et al. (2013). "Structure of the 20S proteasome". *Cell Reports*.
 - [84] Perkins, J. R. et al. (2010). "Transient PPIs in signaling". *Nature Rev. Bio.*
 - [85] Blum, C., & Roli, A. (2003). Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM Computing Surveys (CSUR)*, 35(3), 268-308.
 - [86] Boussaid, I., Lepagnot, J., & Siarry, P. (2013). A survey on optimization metaheuristics. *Information Sciences*, 237, 82-117.
 - [87] Peng, X. et al. (2017). *Nature Methods*, 14(2), 125-134.
 - [88] Liu, Z. P. et al. (2016). *Bioinformatics*, 32(12), 1781-1788.
 - [89] Chua, H. N. et al. (2006). *Genome Biology*, 7(Suppl 1), S16.
 - [90] Pesquita, C. et al. (2009). *Briefings in Bioinformatics*, 10(4), 408-421.
 - [91] Zhang, Q. C. et al. (2017). *Nature Communications*, 8(1), 1-12.
 - [92] Scott, M. S. et al. (2013). *PLoS Computational Biology*, 9(5), e1003067.
 - [93] Pearson, K. (1895). *VII. Note on regression and inheritance in the case of two parents*. *Proceedings of the Royal Society of London*, 58, 240-242.
 - [94] Ashburner, M., Ball, C. A., Blake, J. A., et al. (2000). *Gene ontology: tool for the unification of biology*. *Nature genetics*, 25(1), 25-29.
 - [95] Pesquita, C., Faria, D., Falcão, A. O., Lord, P., & Couto, F. M. (2009). *Semantic similarity in biomedical ontologies*. *PLoS computational biology*, 5(7), e1000443.
 - [96] Binder, J. X. et al. (2014). *COMPARTMENTS: unification and visualization of protein subcellular localization evidence*. *Database*, 2014, bau012.
 - [97] Thul, P. J. et al. (2017). *A subcellular map of the human proteome*. *Science*, 356(6340), eaal3321.
 - [98] Uhlén, M. et al. (2015). *Tissue-based map of the human proteome*. *Science*, 347(6220), 1260419.

-
- [99] Wang, J., Peng, J., Ma, J., & Xu, J. (2019a). Identification of protein complexes with core-attachment structure based on edge weight and clustering coefficient. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 16(2), 553–564.
 - [100] Wang, J., Wang, Z., Zhao, Z., & Zhang, Y. (2019b). Detection of protein complexes using a seed-expanding algorithm based on density and modularity. *BMC Bioinformatics*, 20(1), 1–13.
 - [101] Wang, J., Liu, Y., Zhang, Y., & Zhou, X. (2020). Prediction of dynamic and static protein complexes based on multi-source information fusion. *IEEE Access*, 8, 119354–119365.
 - [102] Kevin Drew, John B. Wallingford, Edward M. Marcotte hu.MAP 2.0: integration of over 15,000 proteomic experiments builds a global compendium of human multiprotein assemblies *Mol Syst Biol* (2021)17:e10016 <https://doi.org/10.15252/msb.202010016>.
 - [103] Vogel, C., & Marcotte, E. M. (2012). *Insights into the regulation of protein abundance...* *Nature Reviews Genetics*, 13(4), 227–232.
 - [104] Brohée, S., & van Helden, J. (2006). Evaluation of clustering algorithms for protein-protein interaction networks. *BMC Bioinformatics*, 7, 488. <https://doi.org/10.1186/1471-2105-7-488>
 - [105] Cherry, J. M., Hong, E. L., Amundsen, C., Balakrishnan, R., Binkley, G., Chan, E. T., ...& Dwight, S. S. (2012). *Saccharomyces Genome Database: the genomics resource of budding yeast. Nucleic Acids Research*, 40(D1), D700–D705. <https://doi.org/10.1093/nar/gkr1029>
 - [106] Tomohiro Harada & Enrique Alba. 2018. Parallel Genetic Algorithms: A Useful Survey. *ACM Comput. Surv.* 9, 4, Article 39 (November 2018), 35 pages.
 - [107] Talbi, E.-G. (2009). *Metaheuristics: From Design to Implementation*. Wiley.
 - [108] E. Alba, B. Dorronsoro, *The Exploration/Exploitation Tradeoff in Dynamic Cellular Genetic Algorithms*, *IEEE Transactions on Evolutionary Computation*, vol. 9, no. 2, pp. 126–142, 2005.
 - [109] T.G. Crainic, M. Toulouse, M. Gendreau, *Towards a Taxonomy of Parallel Tabu Search Heuristics*, *ORSA Journal of Computing*, 1995.
 - [110] C. Mering et al., *Cell*, vol. 111, p. 258, 2002.
 - [111] Scott Chacon and Ben Straub. *Pro Git*, 2^e édition. Apress, 2014. Disponible en ligne : <https://git-scm.com/book/en/v2>

- [112] T. Kluyver et al., “Jupyter Notebooks – a publishing format for reproducible computational workflows”, in *Positioning and Power in Academic Publishing*, pp. 87–90, 2016.
- [113] Kefsi N. et Mekhazni R., `hpc_ts` : Framework modulaire pour la recherche tabou parallèle, USTHB, 2025. <https://pypi.org/project/hpc-tabu/0.2.32/>
- [114] Kefsi N. et Mekhazni R., `hpc_ga` : Framework modulaire pour l’algorithme génétique parallèle, USTHB, 2025. <https://pypi.org/project/hpc-ga/0.2.26/>
- [115] Python Software Foundation. *multiprocessing — Process-based parallelism*. Disponible sur <https://docs.python.org/3/library/multiprocessing.html>
- [116] Sadashiv, N., & Kumar, S. M. D. (2011). Cluster, grid and cloud computing: A detailed comparison. *2011 6th International Conference on Computer Science & Education (ICCSE)*, IEEE.
- [117] Berman, F., Fox, G., & Hey, A. J. (2003). *Grid Computing: Making the Global Infrastructure a Reality*. Wiley.
- [118] Gendreau, M., & Potvin, J.-Y. (2010). *Handbook of Metaheuristics*. Springer.
- [119] Zhou, Y., and Tang, Y. (2018). MLPCD: A parallel multi-level protein community detection algorithm for large-scale protein-protein interaction networks. *Journal of Bioinformatics and Computational Biology*, 16(05), 1840012.
- [120] You, J., Liu, C., and Zhang, M. (2017). A MapReduce-based method for community detection in large-scale protein-protein interaction networks. *BMC Bioinformatics*, 18(Suppl 5), 157.
- [121] Bolze, R., Cappello, F., Germain, C., Magniette, F., & Vicat-Blanc, P. (2006). Grid’5000: A large scale, reconfigurable, controllable and monitorable Grid platform. *Future Generation Computer Systems*, 21(4), 555–573.
- [122] Malika Mehdi. PARALLEL HYBRID OPTIMIZATION METHODS FOR PERMUTATION BASED PROBLEMS. Distributed, Parallel, and Cluster Computing [cs.DC]. Université des Sciences et Technologie de Lille - Lille I, 2011. English. fftel-00841962ff.

Annexe A : Initialization Methods, Comparison and their Implementation

A.1 Protein Complex Detection Methods

A.1.1 Markov Clustering - MCL

Dongen et al introduced in 2002 the *MCL* algorithm, one of the most widely used graph clustering algorithms. It was first applied to clustering protein families and orthologous groups, before proving effective for detecting complexes from protein-protein interaction networks. MCL simulates flow in PPI (Protein-Protein Interaction) networks through two steps: expansion and inflation. The expansion parameter allows flow to connect different regions of the graph, while the inflation parameter is used to strengthen or weaken the flow, adjusting cluster granularity. Consequently, cluster size heavily depends on the inflation parameter. The algorithm repeats expansion and inflation steps until convergence, then partitions the PPI network into non-overlapping clusters.

To develop the MCL method code, there are two approaches:

- Use the Python library `markov_clustering`, which provides an optimized implementation with efficient vectorized versions of key matrix operations (expansion/inflation), plus it's simple to use:

```
result = mc.run_mcl(adj_matrix, inflation=2.0)
clusters = mc.get_clusters(result)
```

- A manual implementation (From Scratch) allowing full control over each step and deep understanding of the algorithm (see Algorithm A.1)

Algorithm A.1: Markov Clustering (MCL)**Data:** Graph $G = (V, E)$, $r \in [1.4, 2.5]$, $\epsilon = 10^{-6}$, max_iter **Result:** Protein clusters C

```

1 Initialization ;
2  $M \leftarrow$  Adjacency matrix of  $G$  ;
3  $change \leftarrow \infty$ ,  $iter \leftarrow 0$  ;
4 while  $change > \epsilon$  and  $iter < max\_iter$  do
5    $M_{old} \leftarrow M$  ;
6   Expansion:  $M \leftarrow M \times M$  ;
7   Inflation:  $M_{ij} \leftarrow \frac{M'_{ij}}{\sum_k M'_{kj}}$  ;
8    $change \leftarrow \|M - M_{old}\|_F$  ;
9    $iter \leftarrow iter + 1$  ;
10 end
11 Post-processing ;
12  $C \leftarrow \{\text{connected components where } M_{ij} > 0\}$  ;
13  $C \leftarrow \{c \in C \mid |c| \geq 2\}$  return  $C$ 

```

A.1.2 Molecular Complex Detection Method - MCODE

This is a graph theory-based clustering algorithm that detects densely connected regions in large protein-protein interaction networks that may represent molecular complexes. The MCODE algorithm works in three stages: vertex weighting, complex prediction, and optionally post-processing to filter or add proteins to obtained complexes based on connectivity criteria:

- Vertex weighting: For each protein, assign a score based on local density using the highest k -core of its neighborhood. A k -core of a graph is the maximal subgraph where each node has at least k internal connections.

$$\text{score}(v) = \text{density}(k\text{-neighborhood}) \times k_{\max}$$

where density is defined in section 1.2.2

- Complex prediction: Select the unvisited node with highest score as complex seed and recursively expand the complex by adding neighbors verifying

$$\text{score}(u) \geq \text{VWP} \times \text{score}(\text{seed})$$

where VWP is a vertex weight percentage.

- Post-processing: Only complexes containing at least one 2-core subgraph are retained using a boolean parameter **haircut**. A second boolean parameter **fluff** adds neighbors

if local density exceeds the **fluff** threshold (between 0 and 1.0), which may introduce overlaps. If both options are selected, fluff is applied first, then haircut. The steps are shown in Algorithm A.2

Algorithm A.2: MCODE - Molecular Complex Detection

Data: Graph $G = (V, E)$; Vertex Weight Percentage (VWP) $\in [0, 1]$; haircut (bool);
fluff (bool); fluff density $\in [0, 1]$

Result: Set of protein complexes C

- 1 **Step 1: Vertex weighting ;**
- 2 **for** each node $v \in V$ **do**
- 3 Compute $w(v) \leftarrow \text{score}(v)$;
- 4 **end**
- 5 **Step 2: Complex detection ;**
- 6 Sort nodes by $w(v)$ descending ;
- 7 **while** unvisited nodes remain **do**
- 8 Select unvisited node s with maximal $w(s)$;
- 9 $C \leftarrow \{s\}$, threshold $\leftarrow \text{VWP} \times w(s)$;
- 10 **repeat**
- 11 Add to C neighbors u with $w(u) \geq \text{threshold}$;
- 12 **until** no more additions possible;
- 13 Store C if $|C| \geq 2$;
- 14 Mark all nodes in C as visited ;
- 15 **end**
- 16 **Step 3: Post-processing ;**
- 17 **if** haircut = true **then**
- 18 Keep only 2-cores in each complex ;
- 19 **end**
- 20 **if** fluff = true **then**
- 21 **for** each complex C **do**
- 22 Add neighbor nodes if local density $\geq$ fluff threshold ;
- 23 **end**
- 24 **end**
- 25 **return** C sorted by decreasing size

A.1.3 Clustering with Overlapping Neighborhood Expansion - ClusterONE

ClusterONE is an algorithm designed to detect overlapping protein complexes within PPI networks. It consists of three main steps:

- Greedy cluster growth: Starting from an initial seed protein, ClusterONE uses a greedy

strategy to expand the cluster. This expansion is guided by a cohesion measure balancing internal edge weights and boundary edge weights:

$$\text{cohesion}(G, C, n, \alpha) = \frac{\text{Internal}(n, C) - \alpha \times \text{External}(n, C)}{\sqrt{|C|}} \quad (\text{A.1.1})$$

where:

- $\text{Internal}(n, C) = \sum_{m \in C} w(n, m)$ (sum of edge weights between n and cluster C members)
- $\text{External}(n, C) = \sum_{m \notin C} w(n, m)$ (sum of edge weights between n and external nodes)
- α is the penalty factor for external connections
- $|C|$ is current cluster size

Proteins are iteratively added/removed to maximize this cohesion, producing preliminary clusters.

- **Cluster merging:** After initial clustering, the algorithm evaluates overlap between clusters.

$$\text{overlap}(C_1, C_2) = \frac{|C_1 \cap C_2|}{\min(|C_1|, |C_2|)} \quad (\text{A.1.2})$$

Clusters with high overlap degree (determined by a specified threshold) are merged into unified clusters.

- **Post-processing:** In the final step, the algorithm filters clusters based on predefined criteria like minimum cluster size and density thresholds. Clusters failing these criteria are eliminated, refining results to include only those most likely representing true protein complexes.

These steps are formulated in Algorithm A.3

A.1.4 Core-Attachment Based Method - COACH

COACH is a method that detects protein complexes in two steps. It first detects core proteins forming the "heart" of complexes from each vertex's neighborhood graph in the PPI network. It then extracts peripheral information for each core and selects reliable attachments cooperating with them to form protein complexes. See Algorithm ??

A.1.5 Edge Weight Algorithm and Core Attachment Structure - EWCA

EWCA uses a core-attachment architecture where peripheral and overlapping attachment proteins surround a dense protein core, forming protein complexes. The steps are detailed below:

Algorithm A.3: ClusterONE - Protein Complex Detection

Data: Weighted interaction graph $G = (V, E, w)$, density_threshold ρ , penalty α , overlap_threshold

Result: Set of protein complexes C

- 1 **Phase 1: Greedy Cluster Growth;**
- 2 $C \leftarrow \emptyset$;
- 3 **foreach** $v \in V$ **do**
- 4 **if** v not in any $c \in C$ **then**
- 5 $cluster \leftarrow \{v\}$;
- 6 $neighbors \leftarrow N(v)$;
- 7 **while** $neighbors \neq \emptyset$ **do**
- 8 $u \leftarrow \arg \max_{n \in neighbors} \text{cohesiveness}(G, cluster, n, \alpha)$;
- 9 $new_cluster \leftarrow cluster \cup \{u\}$;
- 10 $density \leftarrow \text{density}(new_cluster)$;
- 11 **if** $density \geq \rho$ **then**
- 12 $cluster \leftarrow new_cluster$;
- 13 $neighbors \leftarrow (neighbors \setminus \{u\}) \cup (N(u) \setminus cluster)$;
- 14 **end**
- 15 **else**
- 16 **break**;
- 17 **end**
- 18 **end**
- 19 **end**
- 20 **end**
- 21 **Phase 2: Hierarchical Merging;**
- 22 $merged \leftarrow \emptyset$;
- 23 **foreach** $c \in C$ **do**
- 24 $found \leftarrow false$;
- 25 **foreach** $m \in merged$ **do**
- 26 **if** $\text{overlap}(c, m) > \text{overlap_threshold}$ **then**
- 27 $m \leftarrow m \cup c$;
- 28 $found \leftarrow true$;
- 29 **break**;
- 30 **end**
- 31 **end**
- 32 **if not found then**
- 33 $merged \leftarrow merged \cup \{c\}$;
- 34 **end**
- 35 **end**
- 36 **Phase 3: Post-processing;**
- 37 $C \leftarrow \{c \in merged \mid |c| \geq 3\}$;
- 38 **return** C

Algorithm A.4: COACH - Protein Complex Core Mining

Data: Weighted interaction graph $G = (V, E, w)$, density threshold d , similarity threshold t , minimum size min_size

Result: List of protein complexes

```

1 Graph loading;
2  $G \leftarrow$  read weighted graph from file;
3 Initialization;
4  $SC \leftarrow \emptyset$ ;                                     // Final complex list
5 foreach  $v \in G$  do
6    $G_v \leftarrow$  subgraph induced by  $v$  and its neighbors;
7    $avg\_deg \leftarrow$  average degree of  $G_v$ ;
8    $core\_nodes \leftarrow \{u \in G_v \mid deg(u) \geq avg\_deg\}$ ;
9    $CG \leftarrow$  subgraph induced by  $core\_nodes$ ;
10  Core reduction;
11   $S \leftarrow core\_removal(CG, d)$ ;
12  foreach  $sg \in S$  do
13    while  $density(sg) < d$  and  $|sg| > 2$  do
14      Remove lowest degree vertex from  $sg$ ;
15    end
16    if  $|sg| < 2$  then
17      continue
18    end
19    Adding strongly connected nodes;
20     $candidates \leftarrow$  nodes in  $CG$  but not in  $sg$ ;
21    while  $candidates \neq \emptyset$  do
22       $w \leftarrow$  candidate with maximal degree;
23       $sg\_temp \leftarrow sg \cup \{w\}$  with its edges;
24      if  $density(sg\_temp) \geq d$  then
25         $sg \leftarrow sg\_temp$ ;
26        Update  $candidates$ ;
27      end
28      else
29        Remove  $w$  from  $candidates$ ;
30      end
31    end
32    Redundancy filtering;
33    Apply  $redundancy\_filtering(sg, SC, t)$ ;
34  end
35 end
36 Size filtering;
37  $Complexes \leftarrow \{c \in SC \mid |c| \geq min\_size\}$ ;
38 return  $Complexes$ 

```

- Edge weighting: EWCA starts by assessing reliability of protein interactions in the PPI network. Based on common neighbors, it evaluates similarity between interacting proteins using a Jaccard similarity coefficient (JCS) approach with these equations:

$$\text{JCS}(v, u) = \begin{cases} \frac{|CN(v, u)|}{|N(v) \cup N(u)|}, & \text{if } |N(v) \cap N(u)| \geq 1 \\ 0, & \text{otherwise} \end{cases} \quad (\text{A.1.3})$$

$$\text{HOCN}(v, u) = \frac{\text{JCS}(v, u) + \text{CNS}(v, u)}{|CN(v, u)| + 1} \quad (\text{A.1.4})$$

$$\text{CNS}(v, u) = \sum_{w \in CN(v, u)} (\text{JCS}(v, w) \times \text{JCS}(w, u)) \quad (\text{A.1.5})$$

This ensures only high-confidence interactions are considered by filtering inconsistent ones and assigning edge weights.

- Core complex identification: It determines which proteins should form the core based on structural similarity calculated as:

$$SN(v) = \{v\} \cup N(v) \quad (\text{A.1.6})$$

$$SS(v, w) = \frac{|SN(v) \cap SN(w)|}{\sqrt{|SN(v)| \cdot |SN(w)|}} \quad (\text{A.1.7})$$

The core is formed of proteins showing high structural similarity with an original protein, ensuring central proteins are functionally and closely related.

- Attachment protein detection: After identifying the core, EWCA detects attachment proteins interacting with the core without being part of it. These fall into two categories:
 - Peripheral proteins: Belonging to a single protein complex and closely interacting with the core
 - Overlapping proteins: Belonging to multiple protein complexes and playing a role in connecting different complexes

EWCA uses weighted connectivity measures to distinguish these two attachment protein types.

- Protein complex formation: EWCA creates complete protein complexes by combining core proteins with corresponding attachments. Redundant or overlapping complexes are filtered to ensure accuracy of final non-redundant results.

Algorithm A.5: EWCA - Protein Complex Detection**Data:** $G(V,E)$, structural similarity threshold $ss_threshold$ **Result:** Set of protein complexes C

```

1 Step 1: Weight calculation;
2 foreach neighboring ( $u, v$ ) do
3   | compute Jaccard similarity; compute HOCN;  $Ws[u, v] \leftarrow (JCS + HOCN)/2$ 
4 end
5 Step 2: Core detection;
6 foreach protein  $u$  do
7   |  $core \leftarrow \{u\}$ ;
8   | foreach  $v \in relations[u]$  do
9     | if  $SS(u, v) > ss\_threshold$  then
10    | | Add  $v$  to  $core$ 
11    | end
12  | end
13  | if  $|core| \geq 2$  then
14  | | Add  $core$  to  $C$ 
15  | end
16 end
17 Step 3: Complex extension (attachments);
18 foreach  $core \in C$  do
19   | Determine candidate neighbor proteins;
20   | Compute internal/external weights;
21   | Add local and overlapping proteins based on criteria
22 end
23 Step 4: Redundant complex filtering;
24 Merge complexes with overlap  $\geq 0.8$ ;
25 Step 5: Result saving;
26 foreach  $complex \in C$  with  $|complex| \geq 3$  do
27 end
28 return  $C$ 

```

A.1.6 Seed Extended Algorithm Based on Density and Modularity with Topological Structure and GO Annotations - SEDMTG

SE-DMTG proposes a new model combining modularity and density, enriched with additional biological information. The algorithm generally proceeds through four steps:

- Construction of a weighted PPIN network using common neighbors and GO annotations:

$$CN(v, u) = \begin{cases} \frac{|N(v) \cap N(u)|^2}{|N(v)| \cdot |N(u)|}, & \text{if } |N(v)| \geq 1 \text{ and } |N(u)| \geq 1 \\ 0, & \text{otherwise} \end{cases} \quad (\text{A.1.8})$$

$$GO(v, u) = \begin{cases} \frac{|GO(v) \cap GO(u)|}{\max(\min(|GO(v)|, |GO(u)|), \text{Average}(GO))}, & \text{if } |GO(v) \cap GO(u)| > 0 \\ 0, & \text{otherwise} \end{cases} \quad (\text{A.1.9})$$

$$w(v, u) = \begin{cases} \frac{GO(v, u) + CN(v, u)}{2}, & \text{if } GO(v, u) + CN(v, u) > 0 \\ 0, & \text{otherwise} \end{cases} \quad (\text{A.1.10})$$

- Seed node selection: Compute scores for all nodes and select the node with maximal score as seed. The score is calculated as:

$$Score(v) = dw(v) \times NGCC(v) \quad (\text{A.1.11})$$

- Protein complex detection: Starting from the seed node, a greedy approach is applied to add/remove nodes to cluster SG , aiming to obtain a subgraph with high fitness. The growth process is repeated from different seeds to form multiple potentially overlapping subgraphs. Once a cluster SG is formed, its nodes are recorded to avoid reuse as seeds. A new seed node is then selected from remaining nodes in queue SQ to generate another SG cluster. Additionally, candidate complexes smaller than 3 are eliminated, and unreliable complexes are removed.
- Redundant complex elimination.

See Algorithm **A.6**

Algorithm A.6: SEDMTG - Protein Complex Detection**Data:** Graph $G = (V, E, w)$, number of iterations I **Result:** Set of protein complexes C

```

1  $C \leftarrow \emptyset$ ; // List of detected complexes
2 for  $i \leftarrow 1$  to  $I$  do
3   Step 1: Seed search;
4   foreach  $v \in V$  do
5     if  $\deg(v) \geq 2$  then
6       Compute  $v$ 's score;
7     end
8   end
9   Randomly shuffle seeds;
10  Step 2: Seed-based growth;
11  foreach seed  $v$  do
12    if  $v$  unvisited and initial cluster size  $\geq 3$  then
13      if  $\text{score}(\text{initial cluster}) \geq \text{global average}$  then
14         $\text{cluster} \leftarrow \{v\}$ ;
15         $\text{cluster} \leftarrow \text{extension\_and\_correction}(\text{cluster})$ ;
16        if cluster score improved then
17           $C \leftarrow C \cup \{\text{cluster}\}$ ;
18        end
19      end
20    end
21  end
22 end
23 Step 3: Redundant complex filtering;
24 Sort  $C$  by decreasing size;
25 foreach pairs  $(c_1, c_2) \in C$  do
26   if  $\text{overlap}(c_1, c_2) \geq \text{threshold}$  then
27     Remove less dense or smaller cluster;
28   end
29 end
30 return  $C$ 

```

A.1.7 MPC-C

The **MPC-C** method was introduced as a solution combining topology and biological data. This method relies on a clustering model applied to both dynamic and static PPI networks, using a greedy heuristic search algorithm for exploring protein complexes. First, MPC-C constructs

weighted dynamic and static PPI networks by combining biological and topological information. Then, initial clusters are obtained using core and multifunctional proteins. These clusters are then expanded using a greedy heuristic search algorithm to form candidate protein complexes. Finally, unreliable and highly overlapping protein complexes were discarded. The method's operation is formulated in Algorithm A.7

Algorithm A.7: MPCC - Weighted Protein Complex Detection

Data: Weighted PPI graph $G = (V, E, w)$
Result: Final list of non-redundant complexes

- 1 **1. Topological weight calculation;**
- 2 **foreach** $(u, v) \in E$ **do**
- 3 Compute $TO[u, v]$ (topological overlap score)
- 4 **end**
- 5 **2. Weight merging;**
- 6 **foreach** $(u, v) \in E$ **do**
- 7 **if** $w[u, v] > 0$ **and** $TO[u, v] > 0$ **then**
- 8 $W[u, v] \leftarrow$ harmonic mean of $w[u, v]$ and $TO[u, v]$
- 9 **end**
- 10 **end**
- 11 **3. Seed detection;**
- 12 **foreach** *protein* u **do**
- 13 **if** $\deg(u) \geq 2$ **then**
- 14 Identify a strongly connected *core*;
- 15 Add essential proteins if close to core;
- 16 **if** $\text{density}(\text{core}) > \text{average density}$ **then**
- 17 Add as seed
- 18 **end**
- 19 **end**
- 20 **end**
- 21 **4. Complex growth;**
- 22 **foreach** *seed* **do**
- 23 Expand and refine cluster by additions/removals;
- 24 Keep if size ≥ 3
- 25 **end**
- 26 **5. Redundancy filtering;**
- 27 Merge complexes with overlap > 0.8 ;
- 28 **6. Ranking and output;**
- 29 Compute complex entropy;
- 30 Sort by decreasing entropy;
- 31 Save valid complexes
- 32 **return** *Final set* C

A.2 Implementation and results

The implementation was conducted on a MacBook Pro (macOS Ventura 13.6.7, Intel Core i9) with 16 GB RAM. The algorithms of the methods presented in the previous section were reimplemented in Python for complete parameter control. The BIOGRID, STRING and DIP networks were pre-filtered to keep only high-confidence interactions. The scripts are available on

A.2.1 Markov clustering - MCL

We used the Python libraries **markov_clustering** and **networkx** for graph management to implement the Markov clustering (MCL) method. The process begins by reading the protein interaction file, from which an undirected graph is created. This graph is then converted into an adjacency matrix, on which MCL is executed with a configurable inflation value (here, we tested several values to maximize results). The algorithm adds loops to each node (self-loops), then applies matrix expansion and inflation until convergence. The cluster extracts are then filtered to keep only those with a minimum size (≥ 3 proteins). We tried several inflation values whose results are illustrated in Table A.1 and Figure ??

Table A.1: Network performance for different inflation values

Inflation	Network	Precision	Recall	F-measure	Accuracy
1.5	BIOGRID_human	0.2499	0.6780	0.3652	0.4116
	BIOGRID_yeast	0.1716	0.7506	0.2794	0.3589
	DIP_yeast	0.1487	0.7160	0.2462	0.3263
	STRING_human	0.1845	0.7883	0.2990	0.3814
	STRING_yeast	0.1700	0.9100	0.2865	0.3934
2.5	BIOGRID_human	0.6031	0.4347	0.5052	0.5120
	BIOGRID_yeast	0.3481	0.4765	0.4023	0.4073
	DIP_yeast	0.3774	0.3947	0.3859	0.3859
	STRING_human	0.3084	0.6681	0.4220	0.4539
	STRING_yeast	0.2438	0.8595	0.3799	0.4578

A.2.2 Molecular Complex Detection method - MCODE

The MCODE method was implemented in Python following the three main steps of the original algorithm: vertex weighting based on the density of k-core subgraphs of their neighborhood, complex detection by exploring highly weighted nodes according to a threshold determined by the vwp parameter, and optional post-processing including the haircut (removal of poorly

connected nodes) and fluff (addition of dense neighbors) options. We first tested several vwp values to determine the best one with haircut = False and fluff = False, see Figure A.1, then explored different fluff_density value configurations, see Figure A.2 for haircut = True then haircut = False, evaluating the impact of these parameters on complex detection performance.

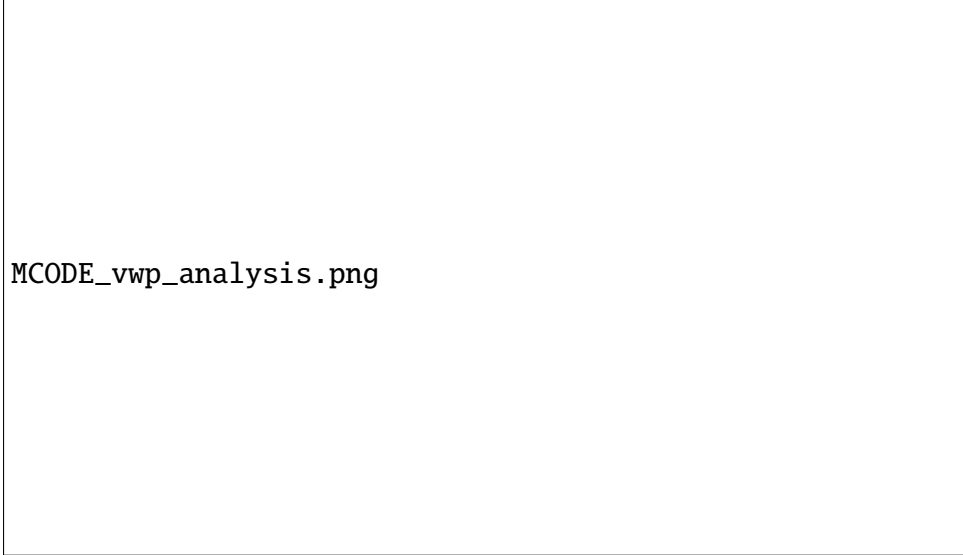


Figure A.1: Variation of total score with respect to VWP

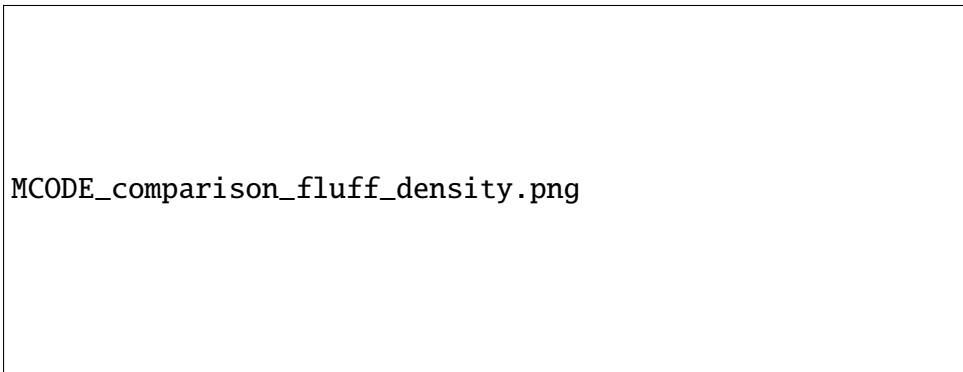


Figure A.2: Variation of total score with respect to fluff_density

A.2.3 Clustering with Overlapping Neighborhood Expansion - ClusterONE

The ClusterONE method was implemented in Python following its main steps. The process depends on several parameters, notably the density threshold and the penalty applied to external connections. We first set the penalty to 2, then varied the density threshold to observe its impact on the number of detected clusters. After setting a consistent threshold, we evaluated the total score of performance metrics for different penalty values to determine the most relevant configuration, see Figure A.3.

Table A.2: Variation of *Total Score* according to MCODE parameters

	Conf.	Network	Prec.	Recall	F-measure	Acc.
<i>Fluff 0.0 (T)</i>						
		BIOGRID H.	0.3632	0.3122	0.3358	0.3367
		BIOGRID L.	0.2816	0.2808	0.2812	0.2812
		DIP L.	0.3544	0.2719	0.3077	0.3105
		STRING H.	0.2717	0.4488	0.3385	0.3492
		STRING L.	0.2642	0.5553	0.3580	0.3830
<i>Fluff 0.9 (T)</i>						
		BIOGRID H.	0.4043	0.3675	0.3850	0.3855
		BIOGRID L.	0.2972	0.3501	0.3215	0.3226
		DIP L.	0.4020	0.3553	0.3772	0.3779
		STRING H.	0.2895	0.5940	0.3893	0.4147
		STRING L.	0.2581	0.7256	0.3808	0.4328
<i>Fluff 0.1 (F)</i>						
		BIOGRID H.	0.1049	0.8373	0.1865	0.2964
		BIOGRID L.	0.1002	0.8943	0.1802	0.2994
		DIP L.	0.0743	0.8432	0.1365	0.2503
		STRING H.	0.1322	0.9470	0.2321	0.3539
		STRING L.	0.1201	0.9968	0.2144	0.3460
<i>Fluff 0.9 (F)</i>						
		BIOGRID H.	0.3269	0.4781	0.3883	0.3954
		BIOGRID L.	0.2432	0.5103	0.3294	0.3523
		DIP L.	0.2602	0.4901	0.3399	0.3571
		STRING H.	0.2852	0.6839	0.4026	0.4417
		STRING L.	0.2338	0.8207	0.3640	0.4381

VWP	Network	Prec.	Recall	F-measure	Acc.
0.1	BIOGRID_H	0.0196	0.7800	0.0381	0.1235
	BIOGRID_L	0.0272	0.8150	0.0526	0.1488
	DIP_L	0.0126	0.7961	0.0247	0.1000
	STRING_H	0.0250	0.8227	0.0486	0.1434
	STRING_L	0.0503	0.8615	0.0951	0.2082
0.9	BIOGRID_H	0.2633	0.3909	0.3146	0.3208
	BIOGRID_L	0.2113	0.3931	0.2748	0.2882
	DIP_L	0.2267	0.3860	0.2856	0.2958
	STRING_H	0.2387	0.4936	0.3218	0.3432
	STRING_L	0.2374	0.6129	0.3423	0.3815

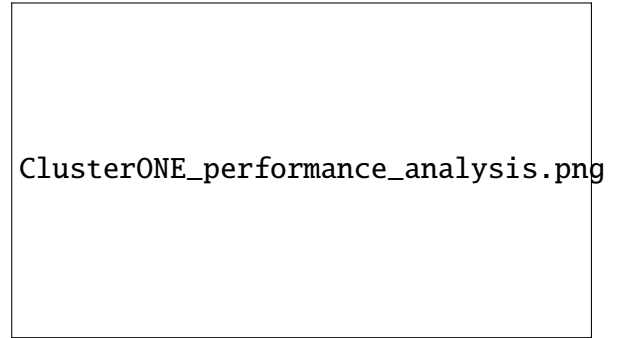
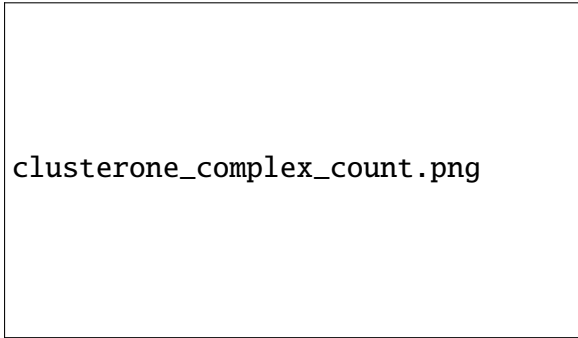


Figure A.3: Visualization of ClusterONE performance from two complementary perspectives

A.2.4 Core Attachment Based Method - COACH

The COACH method was implemented following the steps of the original algorithm. We tested several parameters to be able to choose an optimal value. Initially, we set the similarity threshold to $t = 0.225$ and evaluated the minimum density, then we did the reverse. The results obtained are illustrated in Figure A.4

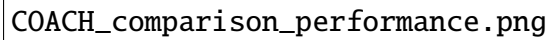


Figure A.4: Variation of total score with respect to COACH parameters

A.2.5 An Extended Algorithm Based on Density and Modularity with Topological Structure and GO Annotations - SEDMTG

The SEDMTG method relies on a multi-seed iterative strategy: at each iteration, proteins are scored according to a score combining density and interaction weights, then selected as seeds. Since this method doesn't really have parameters, we decided to experiment with the number of iterations and observed that there isn't really any change. The results obtained are illustrated graphically in Figure A.5.

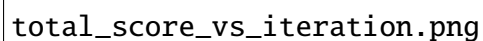


Figure A.5: Variation of total score with respect to number of iterations of SEDMTG

A.2.6 Edge Weight Algorithm and Core Attachment Structure - EWCA

EWCA detects core complexes formed of proteins based on structural similarity, then adds accessory proteins (local or overlapping), a parameter used by EWCA. To find the optimal value of this threshold, we tested several values whose results are presented in Figure A.6

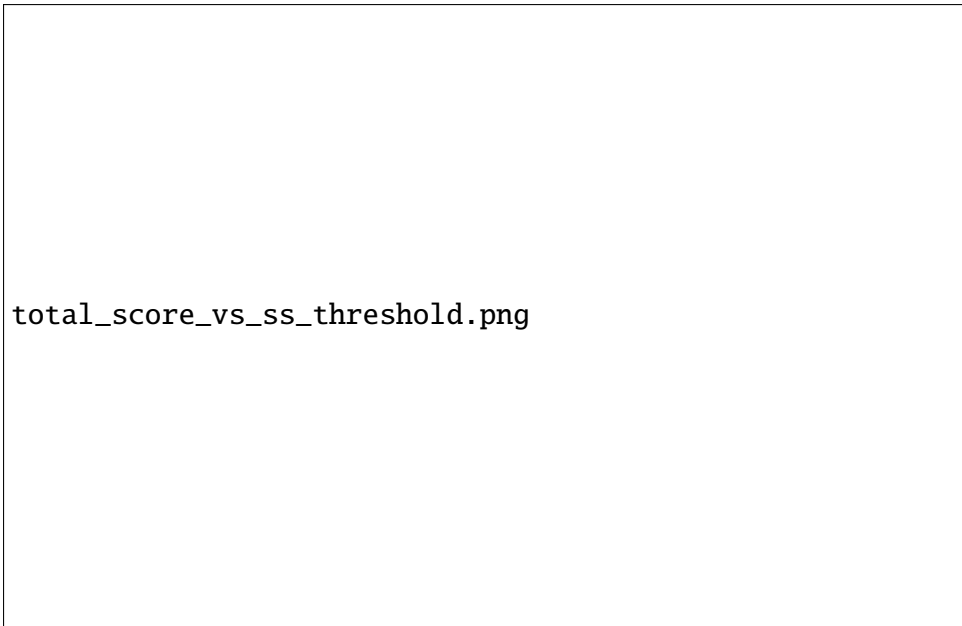


Figure A.6: Variation of total score with respect to COACH parameters

A.2.7 Protein Complex Extraction Using a Clustering Model - MPCC

The MPCC algorithm uses the overlap threshold to detect overlapping complexes. The result of testing the values is presented in Figure A.7

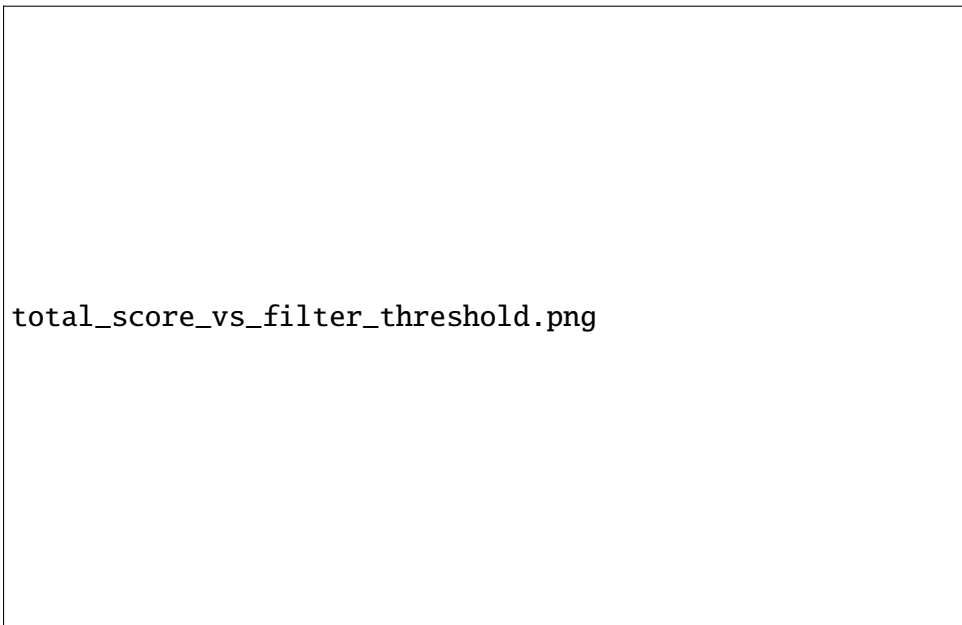


Figure A.7: Variation of total score with respect to COACH parameters

Annexe B : Design, Implementation and Results

B.1 Detailed Functions

Algorithm B.2: GENERATEINDIVIDUAL

Result: An individual (chromosome) or None

```
1  $ref \leftarrow \_shared\_max\_complexes$  ;
2 Randomly draw  $k \sim \mathcal{U}(0.75 \cdot ref, 1.25 \cdot ref)$  ;           // number of complexes
3 Initialize  $complexes \leftarrow []$ ,  $used \leftarrow \emptyset$ ;
4 for  $i \leftarrow 1$  to  $k$  do
5    $C \leftarrow \text{CREATECONNECTEDCOMPLEX}()$  (see algorithm B.3);
6   if  $C \neq \emptyset$  and  $\frac{|C \cap used|}{|C|} < 0.2$  then
7     Add  $C$  to  $complexes$  and update  $used$ ;
8   end
9 end
10 if  $complexes \neq \emptyset$  then
11   return  $\text{Chromosome}(\_shared\_graph, complexes)$ 
12 end
13 else
14   return  $None$ 
15 end
```

Algorithm B.1: RANDOMINITIALIZATION

Data: G : PPI graph, R : reference complexes, N : population size, $max_complexes$, min_size , max_size

Result: Population P composed of N individuals

```

1  $P \leftarrow \emptyset$ ,  $V \leftarrow$  nodes of  $G$ ;
2 while  $|P| < N$  do
3    $complexes \leftarrow []$ ,  $valid \leftarrow \mathbf{true}$ ;
4   for  $i \leftarrow 1$  to  $max\_complexes$  do
5      $T \leftarrow \mathcal{U}(min\_size, max\_size)$ ;
6     Select  $v_0 \in V$  such that  $\deg(v_0) > 0$  (with max attempts);
7     if no valid  $v_0$  then
8        $valid \leftarrow \mathbf{false}$ ; break;
9     end
10     $C \leftarrow \{v_0\}$ ; // complex under construction
11    while  $|C| < T$  do
12       $F \leftarrow$  neighbors of nodes in  $C$  not yet in  $C$ ;
13      if  $F = \emptyset$  then
14        break
15      end
16      Add a random node  $u \in F$  to  $C$ ;
17    end
18    while  $|C| < 3$  do
19      Add a random node  $u \in V$  to  $C$ ;
20    end
21    Add  $C$  to  $complexes$ ;
22  end
23  if  $valid = \mathbf{false}$  then
24    continue
25  end
26   $chrom \leftarrow \text{CHROMOSOME COMPLEXES}(G, complexes)$ ;
27  if  $chrom$  is distinct from all individuals in  $P$  then
28    Add  $chrom$  to  $P$ ;
29  end
30 end
31 foreach  $ind \in P$  do
32   Calculate  $\text{ACCURACY}(ind.genes, R)$ ;
33 end
34 return  $P$ 

```

Algorithm B.3: CREATECONNECTEDCOMPLEX**Result:** A connected complex (set of proteins)

```

1 Randomly choose a node  $v$  with at least one neighbor in  $\_shared\_graph$ ;
2 Initialize  $C \leftarrow \emptyset$ ,  $Q \leftarrow [v]$ ;
3 Draw  $t \sim \mathcal{U}(\min\_size, \max\_size)$ ; // target complex size
4 while  $Q$  not empty and  $|C| < t$  do
5   Extract  $u \leftarrow Q.pop()$ ;
6   if  $u \notin C$  then
7     Add  $u$  to  $C$ ;
8     Get neighbors  $N(u)$  not yet in  $C$ , sorted by decreasing weight;
9     Add the top 3 neighbors (if available) to  $Q$ ;
10  end
11 end
12 return  $C$ 

```

Algorithm B.4: INITGLOBALS**Data:** G : PPI graph, $\min$, $\max$: min and max complex sizes, $\max C$: max number of complexes**Result:** Initialized shared variables

```

1  $\_shared\_graph \leftarrow G$ ;
2  $\_shared\_protein\_nodes \leftarrow$  list of nodes of  $G$ ;
3  $\_shared\_min\_size \leftarrow \min$ ;
4  $\_shared\_max\_size \leftarrow \max$ ;
5  $\_shared\_max\_complexes \leftarrow \max C$ ;

```

Algorithm B.5: COMPUTEMETRICS(P, R, τ)**Data:** P : predicted complexes, R : reference complexes, τ : overlap threshold (e.g. : 0.2)**Result:** Dictionary of evaluation metrics

```

1  $ppv \leftarrow \text{COMPUTEPPV}(P, R)$ ;
2  $sn \leftarrow \text{COMPUTERECALL}(P, R)$ ;
3  $f \leftarrow \text{COMPUDEFMEASURE}(ppv, sn)$ ;
4  $acc \leftarrow \sqrt{ppv \cdot sn}$ ; // Accuracy
5  $mmr \leftarrow \text{COMPUTEMMR}(P, R)$ ;
6  $jaccard \leftarrow \text{COMPUTEJACCARD}(P, R)$ ;
7  $cov \leftarrow \text{COMPUTECOVEREDRATE}(P, R, \tau)$ ;
8 return  $\{PPV : ppv, Sn : sn, F\text{-measure} : f, Accuracy : acc, MMR : mmr, Jaccard :$ 
    $jaccard, Covered Rate : cov, Total Score : f + cov + mmr + jaccard + cov\}$ 

```

Algorithm B.6: COMPUTEPPV(P, R)**Result:** Precision (PPV)

```

1  $s \leftarrow 0$  ;
2 foreach  $p \in P$  do
3    $s \leftarrow s + \max_{r \in R} |p \cap r|$  ;
4 end
5  $den \leftarrow \sum_{p \in P} |p|$  ;
6 return  $s/den$  if  $den > 0$ , else 0

```

Algorithm B.7: COMPUTERECALL(P, R)**Result:** Recall (Sn)

```

1  $s \leftarrow 0$  ;
2 foreach  $r \in R$  do
3    $s \leftarrow s + \max_{p \in P} |p \cap r|$  ;
4 end
5  $den \leftarrow \sum_{r \in R} |r|$  ;
6 return  $s/den$  if  $den > 0$ , else 0

```

Algorithm B.8: COMPUTEFMEASURE(ppv, sn)**Result:** F-measure

```

1 return  $\frac{2 \cdot ppv \cdot sn}{ppv + sn}$  if  $(ppv + sn) > 0$ , else 0

```

Algorithm B.9: COMPUTEMMR(P, R)**Result:** Mean of maximum overlap

```

1 return  $\frac{1}{|P|} \sum_{p \in P} \max_{r \in R} \frac{|p \cap r|}{\sqrt{|p| \cdot |r|}}$ 

```

Algorithm B.10: COMPUTEJACCARD(P, R)**Result:** Average Jaccard index

```

1 return  $\frac{1}{|P|} \sum_{p \in P} \max_{r \in R} \frac{|p \cap r|}{|p \cup r|}$ 

```

Algorithm B.11: COMPUTECOVEREDRATE(P, R, τ)

Result: Coverage rate

```

1  $match \leftarrow 0$  ;
2 foreach  $r \in R$  do
3   if  $\exists p \in P$  such that  $\frac{|p \cap r|}{\sqrt{|p| \cdot |r|}} \geq \tau$  then
4      $match \leftarrow match + 1$  ;
5   end
6 end
7 return  $match/|R|$  if  $|R| > 0$ , else 0

```

Algorithm B.12: COMPUTEFITNESSPOPULATION(P, G)

Data: P : Population (set of individuals), G : PPI graph**Result:** List of fitness scores for each individual

```

1  $fits \leftarrow []$  ;
2 foreach  $individual \in P$  do
3    $f \leftarrow \text{FITNESS}(individual, G)$  ;
4    $fits.append(f)$  ;
5 end
6 return  $fits$ 

```

Algorithm B.13: FITNESS(I, G)

Data: I : individual (set of clusters), G : PPI graph**Result:** Average fitness of individual

```

1  $s \leftarrow 0$  ;
2 foreach  $C \in I$  do
3    $s \leftarrow s + \text{FF}(C, G)$  ;
4 end
5 return  $s/|I|$  if  $|I| > 0$ , else 0

```

Algorithm B.14: FF(C, G)

Data: C : cluster, G : PPI graph**Result:** Global fitness score of cluster

```

1  $f \leftarrow \text{COHESION}(C, G) + \text{DENSITY}(C, G) + \text{AIEW}(C, G)$  ;
2  $f \leftarrow f - \text{ABEW}(C, G) + \text{AWM}(C, G)$  ;
3 return  $f$ 

```

Algorithm B.15: COHESION(C, G)

```

1  $W_{in} \leftarrow 0 ; W_{out} \leftarrow 0 ;$ 
2 foreach  $v \in C$  do
3   foreach  $u \in G[v]$  do
4     if  $u \in C$  then
5        $W_{in} \leftarrow W_{in} + w_{vu}$ 
6     else
7        $W_{out} \leftarrow W_{out} + w_{vu}$ 
8     end
9   end
10 end
11 return  $W_{in} / (W_{in} + W_{out})$  if denominator > 0, else 0

```

Algorithm B.16: DENSITY(C, G)

```

1  $W_{in} \leftarrow 0 ;$ 
2 foreach  $v \in C$  do
3   foreach  $u \in G[v]$  do
4     if  $u \in C$  then
5        $W_{in} \leftarrow W_{in} + w_{vu}$ 
6     end
7   end
8 end
9  $n \leftarrow |C| ;$ 
10 return  $2 \cdot W_{in} / (n \cdot (n - 1))$  if  $n > 1$ , else 0

```

Algorithm B.17: AIEW(C, G)

```

1  $W_{in} \leftarrow 0 ; E_C \leftarrow 0 ;$ 
2 foreach  $v \in C$  do
3   foreach  $u \in G[v]$  do
4     if  $u \in C$  then
5        $W_{in} \leftarrow W_{in} + w_{vu} ; E_C \leftarrow E_C + 1$ 
6     end
7   end
8 end
9 return  $W_{in} / E_C$  if  $E_C > 0$ , else 0

```

Algorithm B.18: ABEW(C, G)

```

1  $W_{out} \leftarrow 0 ; B_C \leftarrow 0 ;$ 
2 foreach  $v \in C$  do
3   foreach  $u \in G[v]$  do
4     if  $u \notin C$  then
5        $W_{out} \leftarrow W_{out} + w_{vu} ; B_C \leftarrow B_C + 1$ 
6     end
7   end
8 end
9 return  $W_{out} / B_C$  if  $B_C > 0$ , else 0

```

Algorithm B.19: AWM(C, G)

```

1  $aiew \leftarrow \text{AIEW}(C, G) ;$ 
2  $abew \leftarrow \text{ABEW}(C, G) ;$ 
3 return  $aiew / (aiew + abew)$  if denominator  $> 0$ , else 0

```

Algorithm B.20: NEIGHBORSBYDELETION(C, G)**Data:** Complex C , PPI graph G **Result:** Set of neighbors $\mathcal{V}_s$ by deletion (while remaining connected)

```

1  $\mathcal{V}_s \leftarrow [] ;$ 
2 foreach  $p \in C$  do
3    $C' \leftarrow C \setminus \{p\} ;$ 
4   if  $C'$  is connected in  $G$  then
5      $\mathcal{V}_s.append(C') ;$ 
6   end
7 end
8 return  $\mathcal{V}_s$ 

```

Algorithm B.21: NEIGHBORSBYINSERTION(C, G)**Data:** Complex C , PPI graph G **Result:** Set of neighbors $\mathcal{V}_i$ by insertion

```

1  $\mathcal{V}_i \leftarrow [] ;$ 
2  $\mathcal{N} \leftarrow$  set of neighbors of  $C$  in  $G ;$            // neighbors of all proteins in  $C$ 
3 foreach  $p \in \mathcal{N} \setminus C$  do
4    $\mathcal{V}_i.append(C \cup \{p\}) ;$ 
5 end
6 return  $\mathcal{V}_i$ 

```

Algorithm B.22: NEIGHBORSBYREPLACEMENT(C, G)

Data: Complex C , PPI graph G **Result:** Set of neighbors $\mathcal{V}_r$ by replacement (connected)

```

1  $\mathcal{V}_r \leftarrow []$ ;
2  $\mathcal{N} \leftarrow$  neighbors of  $C$  in  $G$ ;
3 foreach  $p_{out} \in C$  do
4   foreach  $p_{in} \in \mathcal{N} \setminus C$  do
5      $C' \leftarrow (C \setminus \{p_{out}\}) \cup \{p_{in}\}$ ;
6     if  $C'$  is connected in  $G$  then
7        $\mathcal{V}_r.append(C')$ ;
8     end
9   end
10 end
11 return  $\mathcal{V}_r$ 

```

Algorithm B.23: NEIGHBORSBYDENSECOMPLEXES(C, G, R)

Data: Complex C , PPI graph G , reference complexes R **Result:** Set of neighbors $\mathcal{V}_d$ by dense complexes

```

1  $\mathcal{V}_d \leftarrow []$ ;
2 foreach  $r \in R$  do
3   if  $|r \cap C| \geq t$  then
4      $\mathcal{V}_d.append(r)$ ; // if sufficient partial overlap
5   end
6 end
7 return  $\mathcal{V}_d$ 

```

Algorithm B.24: DIVERSIFICATIONBYRARELYVISITEDPROTEINS(G)

Data: Graph G with node usage frequency counter**Result:** New candidate complexes with low coverage

```

1  $\mathcal{C}_{div} \leftarrow []$ ;
2  $\mathcal{P}_{rare} \leftarrow$  proteins with low frequency;
3 foreach  $p \in \mathcal{P}_{rare}$  do
4    $C_{cand} \leftarrow$  induced subgraph centered on  $p$  and its neighbors;
5   if  $C_{cand}$  is connected and dense then
6      $\mathcal{C}_{div}.append(C_{cand})$ ;
7   end
8 end
9 return  $\mathcal{C}_{div}$ 

```

Algorithm B.25: PARALLELPOPULATIONINITIALIZATION**Data:** G : PPI graph, R : references, o : number of individuals per method, n_{rand} **Result:** Initial population P

```

1  $P_1 \leftarrow \text{EWCA}(G, o)$  ; // see algo ??
2  $P_2 \leftarrow \text{SE\_DMTG}(G, o)$  ; // see algo A.6
3  $P_3 \leftarrow \text{MPC\_C}(G, o)$  ; // see algo A.7
4  $P_4 \leftarrow \text{PARALLELRANDOMINITIALIZATION}(G, n_{\text{rand}}, R)$ 
5  $P \leftarrow P_1 \cup P_2 \cup P_3 \cup P_4$ 
6 return  $P$ 

```

Algorithm B.26: DIVIDEPopulationINTOIslands**Data:** P : global population (list of individuals), $nb_islands$: number of islands**Result:** List of subpopulations $\mathcal{P} = \{P_1, P_2, \dots, P_{nb_islands}\}$

```

1 Initialize  $\mathcal{P} \leftarrow$  empty list of size  $nb\_islands$  ;
2 for  $i \leftarrow 0$  to  $|P| - 1$  do
3    $j \leftarrow i \bmod nb\_islands$  ;
4   Add  $P[i]$  to  $\mathcal{P}[j]$  ;
5 end
6 return  $\mathcal{P}$ 

```

Algorithm B.27: SELECTBESTINDIVIDUALS**Data:** P : population of individuals; k : number of individuals to select; G : PPI graph (used in fitness function)**Result:** List B containing the k best individuals

```

1 Initialize  $B \leftarrow []$  ;
2 Sort  $P$  according to decreasing values of  $FF(s, G)$  for each  $s \in P$  ;
3 Select the first  $k$  individuals from the sorted population and add them to  $B$  ;
4 return  $B$ 

```

Algorithm B.28: SENDToRIGHTNEIGHBOR**Data:** i : identifier of current island; ind_to_send : list of individuals to transmit**Result:** None (transfer to neighbor's buffer)

```

1  $right\_neighbor \leftarrow (i + 1) \bmod nb\_islands$  ;
2  $\text{MIGRATIONBUFFER}[right\_neighbor] \leftarrow ind\_to\_send$  ; // shared memory area
   or Ray message

```

Algorithm B.29: RECEIVEFROMLEFTNEIGHBOR

Data: i : identifier of current island**Result:** $ind_received$: list of individuals received from left neighbor

```
1  $left\_neighbor \leftarrow (i - 1 + nb\_islands) \bmod nb\_islands$  ;  
2  $ind\_received \leftarrow \text{MIGRATIONBUFFER}[i]$  ;      // island  $i$  reads what its left  
   neighbor deposited  
3 return  $ind\_received$ 
```

Algorithm B.30: SELECTWORSTINDIVIDUALS

Data: P_i : local population of island i $migration_size$: number of individuals to select G : interaction graph (used in FF function)**Result:** List W containing the $migration_size$ worst individuals

```
1 Sort  $P_i$  according to increasing values of  $FF(s, G)$  for each  $s \in P_i$  ;  
2  $W \leftarrow$  first  $migration\_size$  individuals of the sorted population ;  
3 return  $W$ 
```
